PROLEAN

Project Management Platform

Technical Design Document

Tasks | Timeline | Access Control | Indonesian Working Calendar

Document version	1.1-draft
Document status	Draft for approval
Date	20 October 2026
Prepared for	Product Owner / Stakeholder
Classification	Internal - design baseline
Implementation gate	Coding starts only after written approval

APPROVAL REQUIRED

This document is the design baseline. No application code will be created until the stakeholder approves it or returns it with changes.

Document Control

Table 1 - Revision history

Version	Date	Author	Status	Change summary
1.0-draft	19 October 2026	Engineering	Superseded	Initial technical design baseline.
1.1-draft	20 October 2026	Engineering	Draft for approval	Design review remediation: RS256/JWKS tokens, credentials moved to auth-service, refresh-token families, timeline audit ledger and snapshots, holiday coverage horizon, idempotent creates, per-caller internal keys, scoped coverage gate. Second-pass review: publish/status locking, draft OPEN/0 defaults, JWKS retirement config, replay ordering.

Approval of this document confirms the scope, architecture, data model, access rules, API contracts, business calculations, frontend approach, and testing acceptance criteria. Any material change after approval should be recorded as a new document version.

Approval

Table 2 - Approval sign-off

Name	Role	Decision	Signature	Date
	Product Owner / Stakeholder	Approved / Changes requested		
	Engineering Lead	Reviewed		
	QA Lead	Reviewed		

Approval gate

The coding folder is intentionally empty. Implementation starts only after this document is approved. Changes requested during review will be applied to this document first, and the document version will be updated.

How to read this document

- Sections 1-3 define the problem, scope, assumptions, and decisions.
- Sections 4-6 define the architecture, data ownership, access control, and calculation rules.
- Sections 7-8 contain the API conventions and the feature-by-feature sequence diagrams and API specifications.
- Sections 9-13 define the frontend, non-functional requirements, testing, error handling, and deployment.
- Appendices contain the Indonesian holiday seed, calculation examples, glossary, and references.

Quick answer: API gateway

The request mentions an open-source payment gateway. This design assumes the intended term is API gateway, because no payment capability is part of the scope. Kong is the most widely used open-source API gateway in the market, followed by Apache APISIX and Traefik. For an all-Java stack, Spring Cloud Gateway is the natural fit. No gateway is included in v1, as requested; Section 4.4 records the comparison and the future recommendation.

Table of Contents

Document Control 2
Approval 2
How to read this document 2
Quick answer: API gateway 2
1. Executive Summary 7
1.1 What the first release delivers 7
1.2 Architecture at a glance 7
1.3 Key design decisions 8
2. Scope and Objectives 9
2.1 Objectives 9
2.2 In scope 9
2.3 Out of scope for release 1 9
3. Assumptions and Design Decisions 10
3.1 Time and date conventions 10
3.2 Open items 11
4. Architecture 12
4.1 Logical architecture 12
4.2 Service responsibilities 12
4.3 Technology stack 12
4.4 API gateway recommendation 13
4.5 Communication and consistency 14
4.6 Security architecture 14
4.7 Deployment topology 15
5. Data Model 16
5.1 Data ownership 16
5.2 Conventions 16
5.3 Authentication database 17
5.4 User and project database 18
5.5 Task database 20
5.6 Timeline database 21
5.7 Constraints and indexes 23
5.8 Migrations and seed data 23
5.9 Data lifecycle and deletion 24
6. Access Control and Business Rules 25
6.1 Roles and permission matrix 25
6.2 Project visibility and access check 25

6.3 Task rules 26
6.4 Timeline rules and versioning 27
6.5 Working-day calculation 29
6.6 Plan completion calculation 29
6.7 Holiday rules 30
6.8 User and project rules 31
6.9 Concurrency and transaction boundaries 31
7. API Conventions 33
7.1 Base URLs and ports 33
7.2 Request headers 33
7.3 Response format 33
7.4 Pagination, sorting, and search 34
7.5 Error contract 34
7.6 HTTP status usage 34
7.7 Stable error codes 35
7.8 Internal API security 36
7.9 OpenAPI and compatibility 36
8. Feature Specifications 37
8.1 Authentication 37
8.2 User and Role Management 42
8.3 Project and Membership Management 46
8.4 Task Management 50
8.5 Timeline Management 54
8.6 Holiday Management 63
8.7 Internal Service APIs 65
9. Frontend Design 68
9.1 Technology choice 68
9.2 Pages and routes 68
9.3 Responsive behaviour for laptop and tablet 68
9.4 Application structure 69
9.5 Authentication and API layer 70
9.6 Timeline screen behaviour 70
9.7 Loading, empty, and error states 71
9.8 Accessibility and UX 71
10. Non-Functional Requirements 72
10.1 Performance 72
10.2 Scalability and availability 72

10.3 Security requirements 72
10.4 Observability 73
10.5 Browser and device support 73
10.6 Data volume and retention 73
10.7 Maintainability 73
11. Testing Strategy and Definition of Done 74
11.1 Test principles 74
11.2 Backend test layers 74
11.3 Required unit-test scenarios 74
11.4 Frontend test layers 75
11.5 Coverage gate 75
11.6 Feature acceptance matrix 76
11.7 CI pipeline 76
11.8 Definition of Done 77
12. Error Handling and Observability 78
12.1 Backend error model 78
12.2 Validation errors 78
12.3 Dependency failures 78
12.4 Logging 79
12.5 Health endpoints 79
13. Deployment and Operations 80
13.1 Docker Compose topology 80
13.2 Environment configuration 80
13.3 Database initialisation and migrations 81
13.4 Local startup and URLs 81
13.5 Production notes 82
13.6 Backup and recovery 82
14. Risks, Limitations, and Future Work 83
14.1 Risks and mitigations 83
14.2 Known limitations 83
14.3 Future work 84
15. Approval Checklist 85
15.1 Next steps after approval 85
Appendix A. Indonesian Holiday Seed Data 2025-2027 87
A.1 2025 87
A.2 2026 88
A.3 2027 89

Appendix B. Calculation Examples 90
B.1 Task duration 90
B.2 Timeline working days and plan completion 90
B.3 Timeline revision scenario 90
B.4 Rounding 91
B.5 Zero working days after a holiday edit 91
Appendix C. Glossary 92
Appendix D. References 92

1. Executive Summary

ProLean is a focused project management platform for managing project tasks and a versioned project timeline. The first release is deliberately small: four Spring Boot business services, one React web application, and PostgreSQL. It supports laptop and tablet use, role-based access, automatic task duration, automatic timeline working-day duration, automatic plan completion, and mandatory timeline versioning.

The system separates authentication, user and project administration, task management, and timeline management into four independently deployable services. Each service owns its database and exposes a REST API. There is no API gateway in v1; the browser calls the four services directly through a CORS allow-list. This keeps the deployment small and matches the four-service constraint.

1.1 What the first release delivers

Table 3 - Release 1 capability summary

Capability	Outcome
Login	Username/password login, short-lived JWT access token, rotating refresh token, logout, and current-user endpoint.
Users and roles	Admin-managed users, fixed ADMIN and USER roles, activate/deactivate, password reset, and last-admin protection.
Projects and access	Admin-created projects with member assignment; admin sees all projects, users see only projects where they are members.
Task management	Task CRUD with start/end date, automatic calendar-day duration, PIC, status, and percentage completion.
Timeline management	One timeline per project, mandatory draft/publish versioning, automatic working-day duration, actual and plan completion.
Indonesian calendar	Seeded national holidays and joint leave for 2025-2027, excluding weekends; admin can maintain holidays.
Responsive web UI	React + TypeScript + Ant Design, designed for laptop and tablet viewports.
Quality gate	Automated unit and integration tests with a 100% line coverage gate on production business code.

1.2 Architecture at a glance

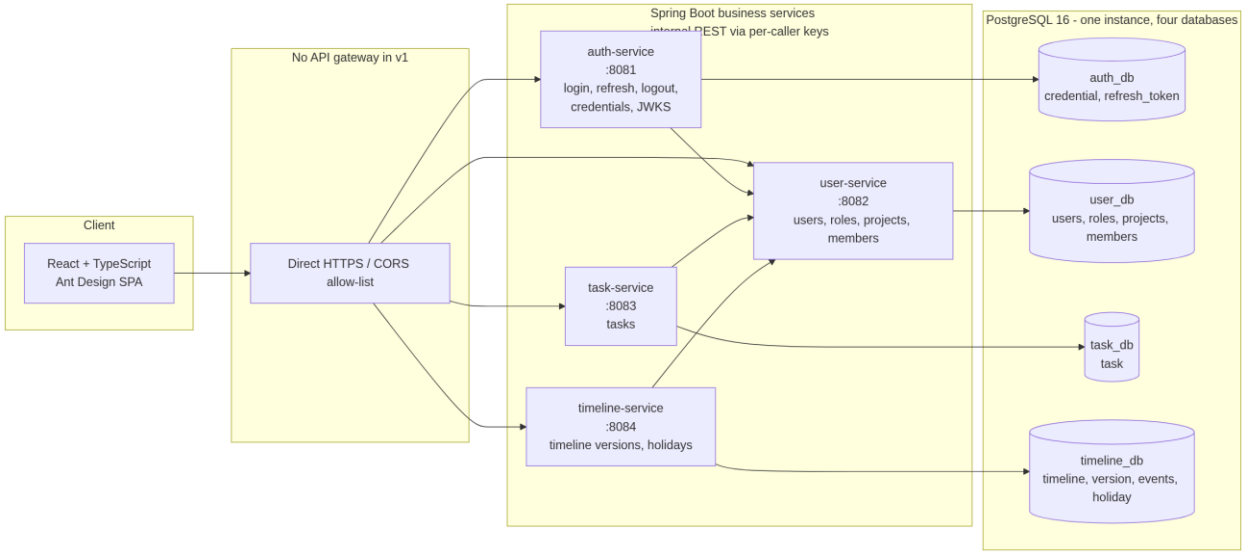


Figure 1 - System context and service boundaries

The four business services are auth-service, user-service, task-service, and timeline-service. auth-service owns credentials, refresh-token families, and the RS256 signing keys; it reads current account status and roles from

user-service. task-service and timeline-service depend on user-service for project access checks and validate access tokens locally with cached JWKS public keys. There are no distributed transactions and no shared tables.

1.3 Key design decisions

- One timeline per project, represented by a timeline aggregate and an ordered set of timeline versions.
- Timeline versions move through DRAFT, PUBLISHED, SUPERSEDED, and CANCELLED. Only one draft and one published version exist per project at any time.
- A timeline revision is created as a draft while the current published version remains active. Publishing promotes the draft, carries forward the current operational status and actual completion, and supersedes the old current version.
- Cancelling the only draft of a never-published timeline removes the timeline identity so the project can start again; published versions are never deleted.
- Task duration is calendar days inclusive. Timeline duration excludes Saturday, Sunday, national holidays, and joint leave.
- Timeline plan completion is calculated from elapsed working days divided by total working days, with a 100% / duration-day daily increment. The value is snapshotted when a version is superseded.
- Access tokens are RS256 JWTs signed by auth-service; every service validates them with cached JWKS public keys. Refresh tokens rotate atomically inside a token family.
- Credentials and password hashes live only in auth-service. user-service is the system of record for profiles, roles, projects, and membership.
- Access is enforced twice: the JWT proves identity and role, and a user-service internal access check proves project membership. Project-scoped writes re-check access immediately before commit.
- Timeline writes are restricted to the seeded holiday coverage horizon (currently 2025-2027). Creating or editing past that horizon requires an admin to add holiday data first.
- Creates use a client-generated requestId with a unique constraint, and publish is replay-safe, so network retries do not duplicate projects, tasks, or timeline versions.
- The frontend keeps the access token in memory and the refresh token in an HttpOnly cookie; it never stores the refresh token in browser storage.
- No API gateway is deployed. Kong is documented as the most widely used open-source option if a gateway is added later.

2. Scope and Objectives

2.1 Objectives

- Give an organisation a simple, auditable way to manage project tasks and a versioned project timeline.
- Ensure users only see projects they belong to, while administrators can see and manage everything.
- Calculate durations and timeline plan completion consistently, using the Indonesian working calendar.
- Keep the operational footprint small: four business services, one PostgreSQL instance, and one web application.
- Make the design testable enough to enforce a 100% line coverage gate on production business code.

2.2 In scope

Table 4 - In-scope capabilities

Area	In scope
Identity	Username/password login, RS256 JWT access token, rotating refresh token family, logout, current user, disabled-account enforcement.
Administration	User CRUD (create, read, update, deactivate), fixed role assignment, admin password reset, project CRUD, project member assignment.
Tasks	Create, list, view, update, change status, change completion, delete. Automatic duration. PIC validation against project membership.
Timeline	Create initial draft, edit draft, cancel draft, publish, revise, publish revision, version history, actual progress, operational status, automatic plan progress.
Calendar	Weekend exclusion, Indonesian national holidays and joint leave, admin-maintained holiday table, 2025-2027 seed.
Frontend	Responsive React application for laptop and tablet, login, dashboard, projects, tasks, timeline, users, holidays, profile.
Quality	Backend unit/slice/integration tests, frontend unit/component tests, 100% line coverage gate on business code, Docker Compose smoke path.

2.3 Out of scope for release 1

Table 5 - Explicitly out of scope

Out of scope	Reason / future option
API gateway	Not required for four services. Kong or Spring Cloud Gateway can be added later; see Section 4.4.
Payment or billing	No payment capability is part of this product. The gateway discussion is about API traffic only.
Email, notifications, comments, attachments	Not required for the first release; can be added as separate capabilities.
Full Gantt dependencies and resource levelling	The timeline is a versioned project-level baseline, not a scheduling optimiser.
Dynamic roles and permissions	Fixed ADMIN and USER roles plus project membership are sufficient for release 1.
SSO, Keycloak, social login	JWT username/password authentication is the approved approach.
Mobile-native applications	Responsive web is the approved client; tablet is a first-class target.
Full cross-entity audit log	Timeline version events, actor UUIDs, and timestamps are stored. A full audit trail for every entity is a future enhancement.
Multi-tenancy	Single organisation deployment.
Reporting and export	Future enhancement; the API is designed to support it.

3. Assumptions and Design Decisions

The following decisions were confirmed before this design was written. They are binding for release 1 unless the approval changes them.

Table 6 - Confirmed design decisions

#	Decision	Consequence
D1	One versioned timeline per project.	A project has one timeline identity and an ordered list of versions. Tasks remain separate project work items.
D2	ADMIN and USER roles only, with project membership.	Admin creates projects and assigns members. Users see only member projects. No per-project role in v1.
D3	Timeline revisions use draft-then-publish.	A draft can be edited while the current published version remains active. Publishing is an explicit, atomic action.
D4	RS256 JWT access token plus rotating refresh token family.	Access token is short-lived, held in memory, and verified through cached JWKS public keys. Refresh tokens are HttpOnly, Secure, SameSite=Lax, stored hashed, and rotated atomically.
D5	No API gateway in v1.	The SPA calls four service origins directly. CORS is configured per service. A gateway comparison is documented.
D6	React + TypeScript + Vite + Ant Design.	The frontend uses Ant Design tables, forms, date pickers, drawers, and responsive grid.
D7	PostgreSQL, one instance, one database per service.	No cross-service joins. Cross-service references are UUIDs validated over internal REST.
D8	100% line coverage gate on production business code.	JaCoCo and Vitest fail the build below the gate. Generated/config/DTO code is explicitly excluded; frontend thresholds are per directory.
D9	Indonesian holidays seeded for 2025-2027, admin-maintained.	Weekend is Saturday/Sunday. National holidays and joint leave both reduce working days. Timeline dates must stay within the covered holiday horizon.
D10	Task duration uses calendar days; timeline duration uses working days.	Task duration = end - start + 1. Timeline duration excludes weekends and holidays.
D11	Plan % is derived on read and snapshotted when a version is superseded.	Current values stay live. Superseded and cancelled versions keep their final snapshot so later holiday corrections do not rewrite history.
D12	PIC is a project member.	Task stores PIC user ID and a display-name snapshot. PIC must be an active member when a task is created or updated.
D13	Credentials are owned by auth-service.	auth_db stores the BCrypt hash, login lock state, refresh-token families, and RS256 signing keys. user-service owns profiles, roles, projects, and membership only.
D14	Create and publish requests are idempotent.	Clients send a UUID requestId; unique constraints and a publish_request_id make retries return the original resource instead of creating duplicates.
D15	Internal REST uses per-caller keys and endpoint scopes.	Each caller has a key ID and secret; receivers enforce the scope matrix. mTLS and network policies are documented for production.
D16	Timeline lifecycle changes are auditable.	timeline_version stores actor/timestamp columns and a timeline_version_event ledger records create, update, publish, supersede, cancel, status, and progress changes.

3.1 Time and date conventions

- Business dates are date-only values in ISO format (yyyy-MM-dd) and have no time zone.
- Business-day calculations use the Asia/Jakarta time zone for determining 'today'.
- Audit timestamps are stored as UTC timestamptz and returned in ISO-8601 UTC.
- Weekends are Saturday and Sunday. Public holidays are data, not hard-coded dates.
- The holiday coverage horizon is derived from the holiday table: 2025-01-01 through 2027-12-31 in the seeded release. Timeline dates outside that horizon are rejected with 409 HOLIDAY_COVERAGE_REQUIRED.
- Historical timeline versions return the plan snapshot taken when they were superseded or cancelled, plus the original planAsOfDate.

3.2 Open items

There are no open design decisions at the time of approval. If the stakeholder wants a full Gantt with activity-level dependencies, dynamic roles, or an API gateway in the first release, this document must be revised before coding starts.

4. Architecture

4.1 Logical architecture

ProLean is a service-oriented system with four business services and one browser client. Each service is a Spring Boot application with its own PostgreSQL database. The services communicate synchronously over REST for the small number of cross-service checks required by release 1. There is no API gateway, message broker, shared table, or distributed transaction. The system context is shown in Figure 1 in Section 1.2.

4.2 Service responsibilities

Table 7 - Service ownership and responsibilities

Service	Port	Owns	Responsibilities	Calls
auth-service	8081	auth_db: auth_credential, refresh_token	Login, credential and password storage, refresh-token families, logout, current-user claims, JWKS and RS256 keys, disabled-account enforcement, login audit fields.	user-service (auth context: status, roles)
user-service	8082	user_db: app_user, role, user_role, project, project_member	User administration (profiles and roles), project administration, project membership, internal access checks, bootstrap profile creation.	auth-service (revoke tokens on deactivation/role change)
task-service	8083	task_db.task	Task CRUD, filtering, automatic calendar-day duration, PIC validation, status and completion updates.	user-service (access, PIC user)
timeline-service	8084	timeline_db: timeline, timeline_version, holiday	Timeline draft/publish versioning, revision history, operational status, actual progress, working-day duration, plan progress, holiday administration.	user-service (access)
frontend	80 (prod) / 5173 (dev)	No database	Responsive SPA: authentication, dashboard, projects, tasks, timeline, users, holidays, profile.	All four services

Why auth-service and user-service are separate

auth-service owns credentials, password hashes, refresh-token families, and the RS256 signing keys. user-service owns the user record, roles, projects, and membership. Login verifies the BCrypt hash inside auth-service and then reads account status and roles from user-service, so password material never crosses a service boundary. User administration and credential administration are separate actions in the UI: create the profile, then set the initial password through auth-service.

4.3 Technology stack

Table 8 - Approved technology stack

Layer	Technology	Why it is used
Backend language	Java 21 LTS	Long-term support, records, pattern matching, virtual threads available if needed.
Backend framework	Spring Boot 3.5.x	Mature REST, security, data, validation, Actuator, and testing ecosystem.
Build	Maven	Simple, reproducible multi-module build and familiar CI integration.
REST API	Spring MVC + springdoc-openapi	Explicit controllers, OpenAPI generation, Swagger UI in non-production.
Security	Spring Security + Nimbus JOSE+JWT	RS256 issue/validation, JWKS key rotation, BCrypt password encoding, method security, per-caller internal keys, standard filters.
Persistence	Spring Data JPA + Hibernate	CRUD and pagination with a familiar Java model.
Migrations	Flyway	Versioned schema and holiday seed data; runs at service startup.
Database	PostgreSQL 16	Reliable relational store, partial unique indexes, generated columns, timestamptz.
Validation	Jakarta Bean Validation	Declarative request validation with consistent error responses.

Layer	Technology	Why it is used
Backend testing	JUnit 5, Mockito, AssertJ, Spring Boot Test, MockMvc, Testcontainers, WireMock	Unit, slice, repository, and integration tests against real PostgreSQL and stubbed internal services.
Coverage	JaCoCo	Line coverage verification bound to the Maven verify phase.
Frontend	React 19 + TypeScript 5 + Vite 7	Fast SPA development, strong typing, simple static deployment.
UI kit	Ant Design 5	Enterprise tables, forms, date pickers, drawers, modals, and responsive grid for laptop/tablet.
Server state	TanStack Query 5	Caching, retries, invalidation, loading/error states without a global store.
Routing	React Router 7	Nested routes, protected routes, project tabs.
Forms	React Hook Form + Zod	Performant forms with schema validation and field-level errors.
HTTP	Axios	Interceptors for bearer token, refresh-once, request ID, and error normalization.
Dates	Day.js	Small date utility with timezone support for Asia/Jakarta display.
Frontend testing	Vitest + React Testing Library + MSW	Fast component and hook tests with mocked HTTP.
Deployment	Docker Compose + Nginx	One-command local stack, static frontend hosting, PostgreSQL volume.
Observability	Spring Boot Actuator + Micrometer + OpenTelemetry + SLF4J/Logback	Liveness/readiness probes, mandatory HTTP server/client metrics, W3C trace propagation, structured logs, request correlation.

4.4 API gateway recommendation

The requirement asks for the most used open-source API gateway, while also stating that no gateway needs to be created. This design therefore keeps the gateway out of the runtime architecture and records the options for a future decision. The term 'payment gateway' in the original requirement is interpreted as 'API gateway'; payment functionality is not in scope.

Table 9 - Open-source API gateway comparison

Gateway	Model	Strengths	Trade-offs	Fit for ProLean
Kong	Open-source API gateway on OpenResty/Nginx; enterprise edition available	Most widely adopted open-source gateway; large plugin ecosystem for authentication, rate limiting, logging, and observability; declarative config; strong community.	Adds a stateful component and its own configuration/deployment; plugin work is mostly Lua/Go, not Java.	Best choice if a full-featured gateway is added later.
Apache APISIX	High-performance gateway with etcd-backed dynamic configuration	Very high throughput; rich plugin set; dynamic routing; good Kubernetes story.	Operational complexity from etcd and its control plane.	Strong alternative for high-traffic or Kubernetes-first deployments.
Traefik	Cloud-native reverse proxy and ingress	Simple configuration, automatic service discovery, good Docker/Kubernetes integration, built-in TLS.	Less comprehensive API-management feature set than Kong or APISIX.	Good for simple routing and TLS if the deployment becomes container-heavy.
Spring Cloud Gateway	Java/Spring reactive gateway	Natural fit for a Java team; route filters, circuit breakers, and security integrate with Spring; one language for the whole backend.	Becomes a fifth deployable; smaller plugin ecosystem; custom code can grow.	Best choice if the team wants an all-Java gateway and only needs routing, auth propagation, and resilience.
Nginx	Reverse proxy / load balancer	Proven, fast, simple static and reverse-proxy configuration.	Not a full API-management gateway; custom Lua or third-party modules for advanced policy.	Suitable as a production edge reverse proxy even if no API gateway is used.

Recommendation

Keep the v1 architecture gateway-free. If a gateway becomes necessary, choose Kong when API management, plugins, and operational tooling matter most; choose Spring Cloud Gateway when the team prefers an all-Java stack and simple routing. The frontend will use four environment-configured service base URLs in v1.

4.5 Communication and consistency

- Service-to-service communication is synchronous REST/JSON over the private Docker network.
- Internal endpoints live under /internal/** and require X-Internal-Key-Id plus X-Internal-Api-Key. Each caller has its own key and an endpoint scope matrix; keys rotate with an overlap window. Internal APIs are never exposed to the browser.
- Access tokens are RS256 JWTs. Resource services fetch auth-service JWKS public keys, cache them, and refresh the cache when an unknown kid appears. The private signing key never leaves auth-service.
- Client-to-service communication uses HTTPS in production and per-service CORS allow-lists.
- No message broker, saga, outbox, or distributed transaction is used in release 1.
- Each service is the only writer of its own database. Cross-service references are UUIDs, not foreign keys.
- The user-service is the system of record for project membership. Access checks are live on every task/timeline request; there is no membership cache in v1.
- Project-scoped writes call the access check again immediately before commit. This narrows the membership-change window to one internal round trip; the residual window is documented in Section 6.9.
- Timeline lifecycle mutations lock the timeline row. Draft edits, publish, and revision creation use expected updated-at values and return 409 rather than overwriting a concurrent change.
- Task rows store a PIC display-name snapshot so task lists do not need a user lookup per row. If a user's name changes, existing tasks keep the snapshot until they are edited.
- Internal GET calls use a 2-second connect timeout and a 3-second read timeout. A failed dependency returns 503 with a stable error code; there is no automatic write retry.

4.6 Security architecture

4.6.1 Token model

Table 10 - Authentication token model

Item	Decision
Access token	JWT signed with RS256 (RSA-2048), 15-minute lifetime, held in frontend memory only. The JOSE header carries kid.
Refresh token	Opaque 256-bit random value, 7-day lifetime, stored as SHA-256 hash in auth_db, delivered in an HttpOnly cookie. Each token belongs to a token family.
Refresh cookie	HttpOnly; Secure in production; SameSite=Lax; Path=/api/v1/auth; Max-Age=604800. Secure is disabled only for local HTTP development.
Rotation	Atomic conditional revoke of the old token and insert of a new token in the same family. If a revoked token is presented, the entire family is revoked (reuse detection).
Multi-tab	The SPA serialises refresh across tabs with a Web Locks API lock and BroadcastChannel fallback, so only one tab rotates a given cookie.
JWT claims	iss=prolean-auth, sub=userId, aud=prolean-api, exp, iat, jti, username, name, roles.
Signing keys	auth-service holds the active RSA private key and serves public keys at GET /.well-known/jwks.json. Retired public keys are supplied through JWT_RETIRED_PUBLIC_KEYS and remain published until all access tokens signed with them have expired. Key rotation is a manual, reviewed operation in release 1.
Validation	Each service validates signature using cached JWKS, issuer, audience, and expiry locally. An unknown kid triggers a rate-limited JWKS refresh.
Revocation	Password set/change/reset, role change, and deactivation revoke all active refresh-token families for the user. Existing access tokens expire within 15 minutes.
Future hardening	MFA, distributed rate limiting, and an external identity provider if the deployment grows.

4.6.2 Authorization model

There are two global roles. ADMIN can manage users, roles, projects, members, holidays, tasks, and timelines. USER can read only projects where they are a member and can create, update, and delete tasks and manage timeline drafts/revisions inside those projects. The JWT carries the global role, but project-level access is always confirmed by user-service so a role alone cannot grant project visibility.

4.6.3 Password and account security

- Passwords are hashed with BCrypt cost 12 inside auth-service and stored only in auth_db.auth_credential. Plaintext passwords are accepted only by auth-service login and password endpoints.
- Password policy: 8-64 characters, at least one letter and one digit.
- An admin resets a user's password through auth-service. The user changes their own password with the current password. Both actions revoke all active refresh-token families for that user.
- Deactivating a user blocks new logins and refresh attempts, and user-service asks auth-service to revoke the user's refresh tokens immediately. An already-issued access token expires within 15 minutes.
- Five consecutive failed logins lock the credential for 15 minutes. A locked account returns 423 AUTH_ACCOUNT_LOCKED.
- The last active ADMIN cannot be deactivated or have the ADMIN role removed. The check is serialized with a PostgreSQL advisory transaction lock. An admin cannot deactivate their own account.
- Login rate limiting beyond the per-credential lockout, MFA, and email verification are future hardening items.

4.6.4 Transport, CORS, and headers

- Production traffic terminates TLS at the Nginx/edge reverse proxy.
- CORS origins are configured per service with an explicit allow-list. Credentials are allowed only for auth-service.
- Allowed headers: Authorization, Content-Type, X-Request-Id. Allowed methods: GET, POST, PUT, PATCH, DELETE, OPTIONS.
- Security headers at the edge: HSTS, X-Content-Type-Options=nosniff, Referrer-Policy, frame-ancestors 'none'.
- Internal API endpoints require X-Internal-Key-Id and X-Internal-Api-Key, enforce the caller scope matrix, and must be blocked from external ingress in production.
- Internal traffic uses TLS in production (mTLS recommended). The local Compose profile uses the private bridge network; per-caller keys still apply.
- A request ID is accepted from X-Request-Id or generated by the service, propagated to internal calls, logged, and returned in error responses.

4.7 Deployment topology

The local and demonstration environment runs one PostgreSQL container with four databases, four Spring Boot containers, and one Nginx container that serves the built React application. Production can split the databases onto managed PostgreSQL and run multiple service replicas; the application contracts do not change. Section 13.1 contains the deployment diagram and Section 13 contains the environment, health check, and startup details.

5. Data Model

5.1 Data ownership

Table 11 - Database ownership map

Database	Owning service	Tables	Notes
auth_db	auth-service	auth_credential, refresh_token	Credentials, login lock state, token families, and token lifecycle. No user profile.
user_db	user-service	app_user, role, user_role, project, project_member	System of record for profiles, roles, projects, and membership. No password hash.
task_db	task-service	task	Task rows reference project and user IDs logically; no cross-database foreign keys.
timeline_db	timeline-service	timeline, timeline_version, timeline_version_event, holiday	One timeline per project; versions, audit events, and holidays drive working-day calculations.

5.2 Conventions

- Primary keys are UUID v4 values generated by the service. Client retry identity is a separate request_id UUID.
- Business dates are DATE values; audit timestamps are TIMESTAMPTZ stored in UTC.
- Enum values are stored as short VARCHAR values with CHECK constraints in PostgreSQL and Java enums in the application.
- Cross-service references are UUID columns without database foreign keys. The owning service validates them over internal REST.
- Users are deactivated, not deleted. Projects are archived, not deleted. Tasks can be hard-deleted. Published timeline versions are never deleted.
- All mutable tables carry created_at and updated_at. Actor columns (created_by, updated_by, published_by) store user UUIDs.
- Create endpoints accept a client-generated request_id (UUID). A unique constraint on request_id makes a retry return the original resource with Idempotent-Replay: true instead of creating a duplicate.
- Timeline lifecycle changes are recorded in timeline_version_event with actor, timestamp, event type, and JSON details. This supplements the immutable version rows and snapshot columns.
- Unique text values are normalised to lowercase in the application (username, email). Project codes are uppercase.

5.3 Authentication database

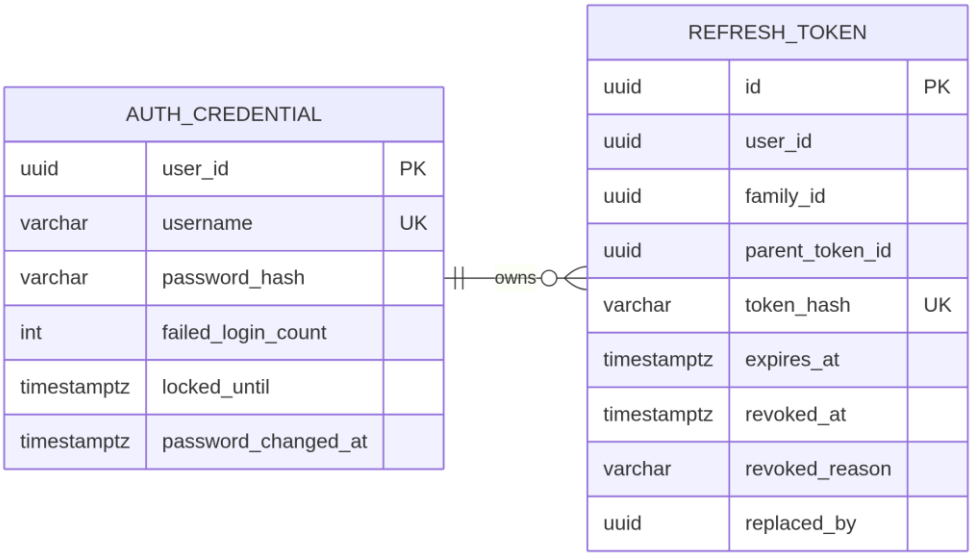


Figure 2 - auth_db entity relationship diagram

5.3.1 auth_credential

Table 12 - auth_db.auth_credential

Column	Type	Null	Rules / notes
user_id	UUID	No	Primary key. Logical reference to user_db.app_user.id.
username	VARCHAR(50)	No	Unique, lowercase. Immutable copy used for login lookup; usernames cannot be changed in release 1.
password_hash	VARCHAR(100)	No	BCrypt hash, cost 12. Never returned by the API or logged.
failed_login_count	INTEGER	No	Consecutive failures; reset on successful login.
locked_until	TIMESTAMPTZ	Yes	Set for 15 minutes after five consecutive failures.
password_changed_at	TIMESTAMPTZ	Yes	When the password was last set.
created_at	TIMESTAMPTZ	No	Audit timestamp.
updated_at	TIMESTAMPTZ	No	Audit timestamp.

5.3.2 refresh_token

Table 13 - auth_db.refresh_token

Column	Type	Null	Rules / notes
id	UUID	No	Primary key.
user_id	UUID	No	Logical reference to user_db.app_user.id. Indexed.
family_id	UUID	No	Token family created at login. Indexed. A reuse detection revokes the whole family.
parent_token_id	UUID	Yes	Token that was rotated into this row; null for the login token.
token_hash	CHAR(64)	No	SHA-256 hex of the opaque refresh token. Unique.
expires_at	TIMESTAMPTZ	No	Seven days after issue.
revoked_at	TIMESTAMPTZ	Yes	Set on rotation, logout, reuse detection, password change, role change, or deactivation.
revoked_reason	VARCHAR(30)	Yes	ROTATED, LOGOUT, REUSE, PASSWORD_CHANGE, ROLE_CHANGE, or DEACTIVATED.
replaced_by	UUID	Yes	ID of the rotated replacement token, for traceability.
created_at	TIMESTAMPTZ	No	Issue time.

Refresh rotation is a single conditional update: the old row is revoked only when revoked_at is null. If two requests race, exactly one rotation succeeds; the loser sees a revoked token and triggers reuse detection, which revokes the entire family. The frontend cross-tab lock prevents this race in normal use.

5.4 User and project database

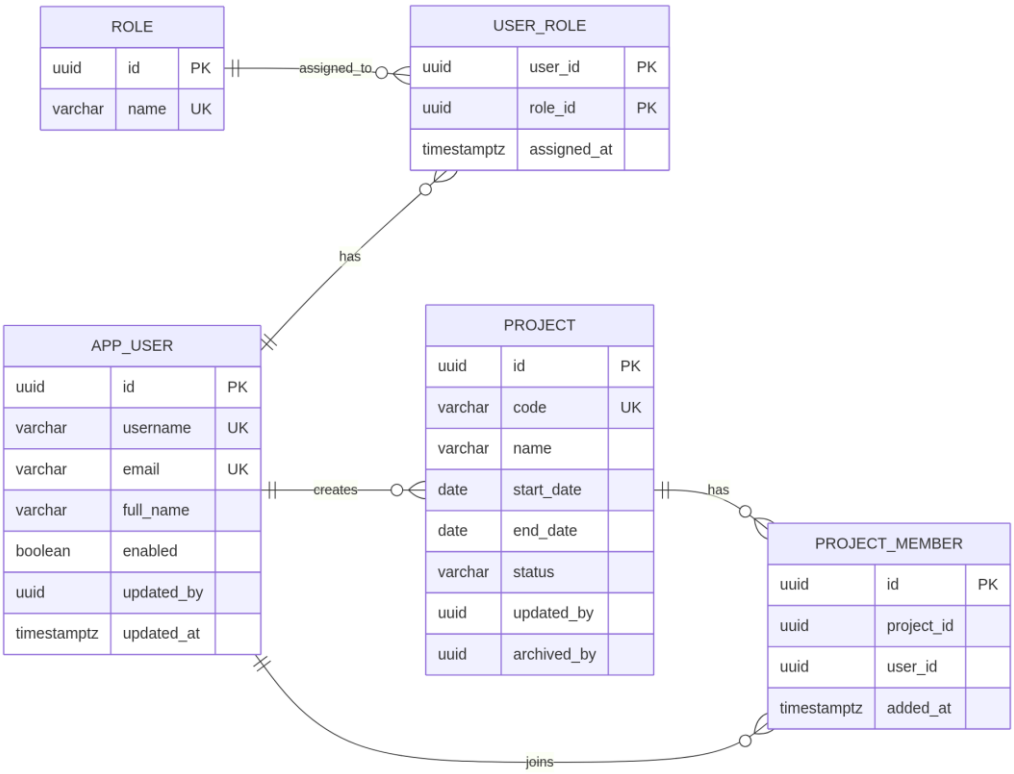


Figure 3 - user_db entity relationship diagram

5.4.1 app_user

Table 14 - user_db.app_user

Column	Type	Null	Rules / notes
id	UUID	No	Primary key.
username	VARCHAR(50)	No	Unique, lowercase, 3-50 characters, pattern [a-z0-9._-].
email	VARCHAR(255)	No	Unique, lowercase, valid email, maximum 255 characters.
full_name	VARCHAR(100)	No	Display name, 1-100 characters.
enabled	BOOLEAN	No	Default true. Deactivated users cannot login or refresh.
created_by	UUID	Yes	Admin who created the user; null for bootstrap admin.
updated_by	UUID	Yes	Last actor.
created_at	TIMESTAMPTZ	No	Audit timestamp.
updated_at	TIMESTAMPTZ	No	Audit timestamp.

The password hash is intentionally not part of user_db. auth_service.auth_credential owns the hash, lock state, and password-change timestamp; user-service never receives or stores a credential through its profile API. The Users screen sets an initial password by calling auth-service after the profile is created.

5.4.2 role and user_role

Table 15 - user_db.role and user_db.user_role

Table.column	Type	Null	Rules / notes
role.id	UUID	No	Primary key.

Table.column	Type	Null	Rules / notes
role.name	VARCHAR(20)	No	Unique. Seeded values: ADMIN, USER.
role.description	VARCHAR(255)	Yes	Human-readable description.
user_role.user_id	UUID	No	FK to app_user.id; part of composite primary key.
user_role.role_id	UUID	No	FK to role.id; part of composite primary key.
user_role.assigned_at	TIMESTAMPTZ	No	When the role was assigned.

5.4.3 project

Table 16 - user_db.project

Column	Type	Null	Rules / notes
id	UUID	No	Primary key.
code	VARCHAR(30)	No	Unique, uppercase, 2-30 characters, pattern [A-Z0-9-].
name	VARCHAR(150)	No	Project name, 1-150 characters.
description	VARCHAR(2000)	Yes	Optional description.
start_date	DATE	No	Project start date.
end_date	DATE	No	Must be on or after start_date.
status	VARCHAR(20)	No	ACTIVE or ARCHIVED. Archived projects are read-only for tasks and timelines.
created_by	UUID	No	Admin who created the project.
updated_by	UUID	Yes	Last actor who updated the project.
archived_by	UUID	Yes	Admin who archived the project; cleared or retained per restore policy.
created_at	TIMESTAMPTZ	No	Audit timestamp.
updated_at	TIMESTAMPTZ	No	Audit timestamp.
archived_at	TIMESTAMPTZ	Yes	Set when status becomes ARCHIVED; cleared on restore.

5.4.4 project_member

Table 17 - user_db.project_member

Column	Type	Null	Rules / notes
id	UUID	No	Primary key.
project_id	UUID	No	FK to project.id. Unique together with user_id.
user_id	UUID	No	FK to app_user.id. Must be an active user when added.
added_by	UUID	No	Admin who added the member.
added_at	TIMESTAMPTZ	No	When membership was granted.

5.5 Task database

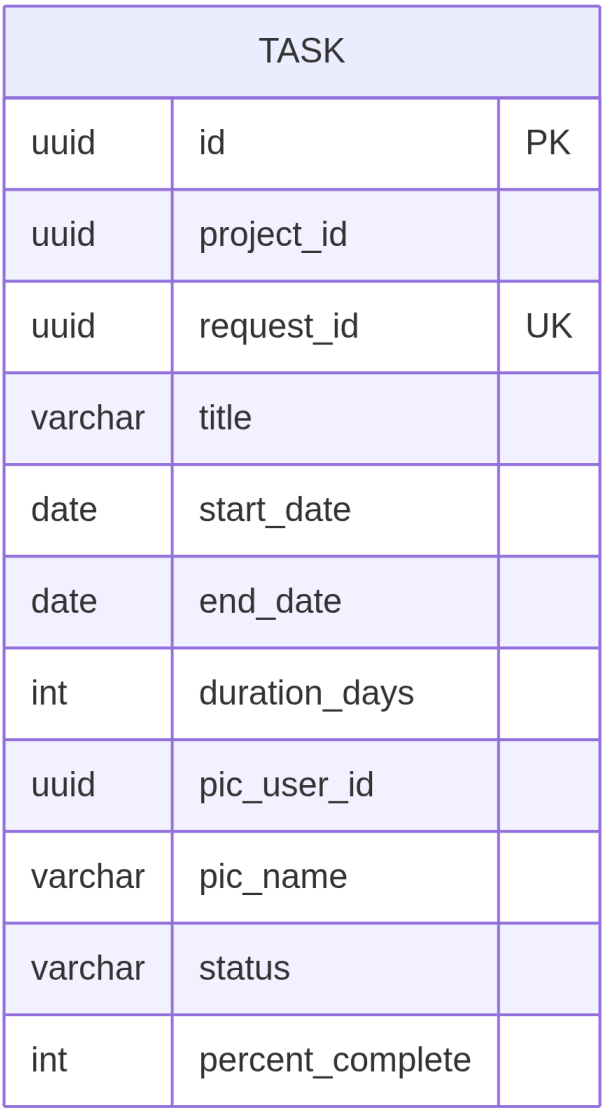


Figure 4 - task_db entity relationship diagram

Table 18 - task_db.task

Column	Type	Null	Rules / notes
id	UUID	No	Primary key.
project_id	UUID	No	Logical reference to user_db.project.id. Indexed with status, PIC, and dates.
request_id	UUID	No	Client-generated idempotency key. Unique across tasks; replay returns the original task.
title	VARCHAR(200)	No	Task title, 1-200 characters.
description	VARCHAR(2000)	Yes	Optional description.
start_date	DATE	No	Inclusive start date.
end_date	DATE	No	Inclusive end date; must be on or after start_date.
duration_days	INTEGER	No	GENERATED ALWAYS AS (end_date - start_date + 1) STORED. Calendar days, inclusive.
pic_user_id	UUID	No	Logical reference to an active project member.
pic_name	VARCHAR(100)	No	Display-name snapshot at create/update time.
status	VARCHAR(20)	No	OPEN, IN_PROGRESS, or CLOSED.
percent_complete	SMALLINT	No	0-100 inclusive. Default 0.
created_by	UUID	No	Actor who created the task.
updated_by	UUID	No	Actor who last updated the task.

Column	Type	Null	Rules / notes
created_at	TIMESTAMPTZ	No	Audit timestamp.
updated_at	TIMESTAMPTZ	No	Audit timestamp.

The duration_days generated column guarantees that task duration cannot drift from the dates. The service also calculates the value for API responses and tests; PostgreSQL remains the final guard. The PIC name snapshot keeps task lists independent from user-service availability during reads. A CHECK constraint enforces status = 'CLOSED' => percent_complete = 100; the service also rejects progress edits on a closed task with 409 TASK_CLOSED.

5.6 Timeline database

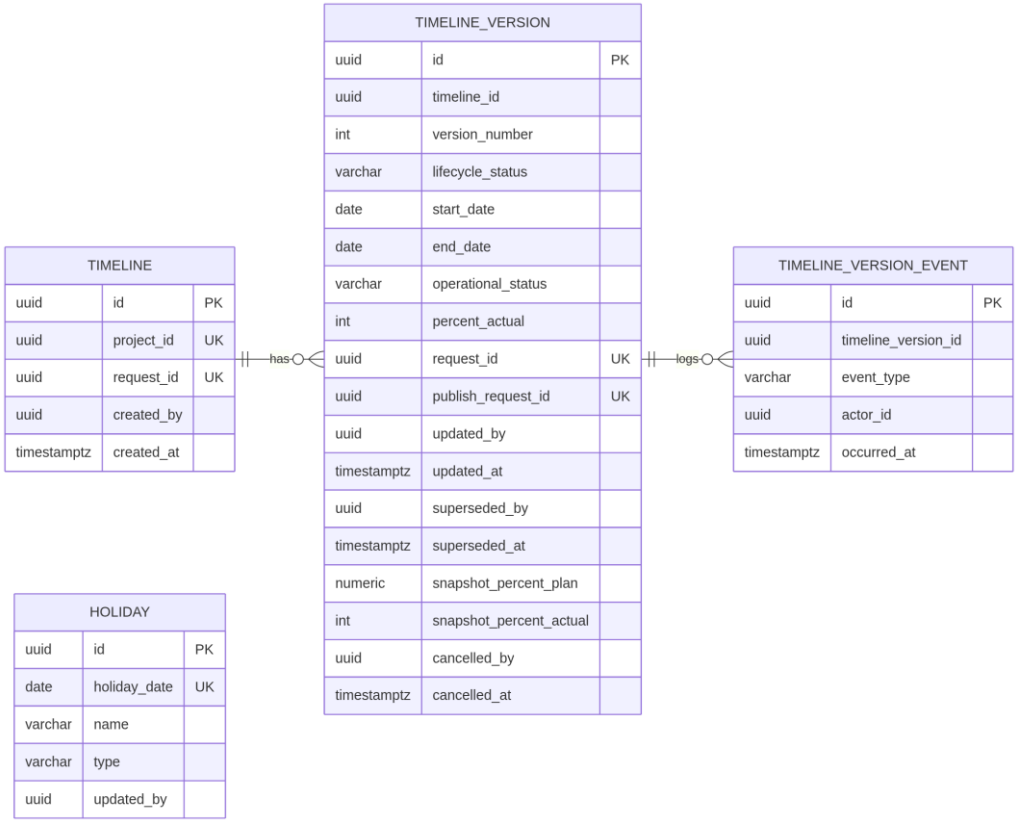


Figure 5 - timeline_db entity relationship diagram

5.6.1 timeline

Table 19 - timeline_db.timeline

Column	Type	Null	Rules / notes
id	UUID	No	Primary key.
project_id	UUID	No	Unique. Logical reference to user_db.project.id. One timeline per project.
request_id	UUID	No	Client-generated idempotency key for the initial create. Unique.
created_by	UUID	No	Actor who created the timeline.
created_at	TIMESTAMPTZ	No	Audit timestamp.
updated_at	TIMESTAMPTZ	No	Audit timestamp.

5.6.2 timeline_version

Table 20 - timeline_db.timeline_version

Column	Type	Null	Rules / notes
id	UUID	No	Primary key.

Column	Type	Null	Rules / notes
timeline_id	UUID	No	FK to timeline.id.
version_number	INTEGER	No	Starts at 1 and increments per project. Unique with timeline_id.
lifecycle_status	VARCHAR(20)	No	DRAFT, PUBLISHED, SUPERSEDED, or CANCELLED.
start_date	DATE	No	Inclusive start date.
end_date	DATE	No	Inclusive end date; must be on or after start_date.
operational_status	VARCHAR(20)	No	OPEN, IN_PROGRESS, or CLOSED. DRAFT and CANCELLED rows store OPEN; PUBLISHED rows carry the operational value.
percent_actual	SMALLINT	No	0-100 inclusive. DRAFT and CANCELLED rows store 0; PUBLISHED rows carry the value. Publish writes the carried-forward value into the new current row.
revision_notes	VARCHAR(1000)	Yes	Reason or summary for the revision.
request_id	UUID	No	Client idempotency key for create/revision. Unique.
publish_request_id	UUID	Yes	Client idempotency key for publish. Unique; a retry returns this version.
created_by	UUID	No	Actor who created the draft.
created_at	TIMESTAMPTZ	No	Draft creation time.
updated_by	UUID	Yes	Actor who last edited the draft or updated the current version.
updated_at	TIMESTAMPTZ	No	Last edit time, used for optimistic concurrency checks.
published_by	UUID	Yes	Actor who published the version.
published_at	TIMESTAMPTZ	Yes	Publish time.
superseded_by	UUID	Yes	Actor who published the newer version.
superseded_at	TIMESTAMPTZ	Yes	When a newer version was published.
snapshot_duration_workdays	INTEGER	Yes	Working-day duration frozen when this version was superseded or cancelled.
snapshot_percent_plan	NUMERIC(5,2)	Yes	Plan completion frozen at snapshot time.
snapshot_percent_actual	INTEGER	Yes	Actual completion frozen at snapshot time.
snapshot_as_of_date	DATE	Yes	As-of date for the frozen plan snapshot.
cancelled_by	UUID	Yes	Actor who cancelled the draft.
cancelled_at	TIMESTAMPTZ	Yes	When the draft was cancelled.

5.6.3 holiday

Table 21 - timeline_db.holiday

Column	Type	Null	Rules / notes
id	UUID	No	Primary key.
holiday_date	DATE	No	Unique. The excluded date.
name	VARCHAR(150)	No	Holiday or joint-leave name.
type	VARCHAR(20)	No	NATIONAL or JOINT_LEAVE. Both types reduce working days.
created_by	UUID	No	Admin who created the row.
updated_by	UUID	Yes	Admin who last changed the row.
created_at	TIMESTAMPTZ	No	Audit timestamp.
updated_at	TIMESTAMPTZ	No	Audit timestamp.

5.6.4 timeline_version_event

Table 22 - timeline_db.timeline_version_event

Column	Type	Null	Rules / notes
id	UUID	No	Primary key.
timeline_version_id	UUID	No	FK to timeline_version.id. Indexed with occurred_at.
event_type	VARCHAR(30)	No	CREATED, UPDATED, PUBLISHED, SUPERSEDED, CANCELLED, STATUS_CHANGED, PROGRESS_CHANGED.

Column	Type	Null	Rules / notes
actor_id	UUID	No	User who performed the action.
occurred_at	TIMESTAMPTZ	No	Event time.
details	JSONB	Yes	Small event payload: previous/new status or percent, requestId, snapshot values.

5.7 Constraints and indexes

Table 23 - Key constraints and indexes

Constraint / index	Purpose
app_user.username unique	Prevents duplicate login names.
app_user.email unique	Prevents duplicate email addresses.
role.name unique	Only one ADMIN and one USER role.
project.code unique	Stable human-readable project identifier.
project_member (project_id, user_id) unique	A user is a member at most once per project.
task.duration_days generated	Database-enforced calendar-day duration.
task (project_id, status)	Task filtering by project and status.
task (project_id, pic_user_id)	Task filtering by PIC.
task (project_id, start_date)	Timeline and date-range listing.
timeline.project_id unique	One timeline per project.
timeline_version (timeline_id, version_number) unique	No duplicate version numbers.
timeline_version partial unique on timeline_id where lifecycle_status = 'DRAFT'	At most one active draft per timeline.
timeline_version partial unique on timeline_id where lifecycle_status = 'PUBLISHED'	At most one current published version per timeline.
holiday.holiday_date unique	No duplicate holiday dates.
refresh_token.token_hash unique	Fast, exact lookup and no duplicate token hash.
refresh_token.user_id	Revoke-all by user and reuse detection.
refresh_token.family_id	Revoke the entire token family on reuse detection.
auth_credential.username unique	Login lookup and one credential per username.
project.request_id unique	Idempotent project creation.
task.request_id unique	Idempotent task creation.
timeline.request_id unique	Idempotent initial timeline creation.
timeline_version.request_id unique	Idempotent revision creation.
timeline_version.publish_request_id unique	Idempotent publish retries.
task CLOSED => 100 CHECK	Database guard that a closed task has percent_complete = 100.
timeline_version CLOSED => 100 CHECK	Database guard that a closed timeline has percent_actual = 100. DRAFT/CANCELLED rows store OPEN/0 and never violate it.
timeline_version_event (timeline_version_id, occurred_at)	Fast history lookup for one version.

5.8 Migrations and seed data

- Each service uses Flyway with its own migration history table in its own database.
- user-service V1 creates the profile, role, project, and membership schema and seeds the fixed roles (ADMIN, USER). It has no password column.
- auth-service V1 creates auth_credential and refresh_token. On startup, if no active admin credential exists and BOOTSTRAP_ADMIN_PASSWORD is provided, auth-service hashes it locally and calls user-service POST

/internal/users/bootstrap (idempotent) to ensure the ADMIN profile and role exist. The password is never logged.

- timeline-service V2 seeds the 2025-2027 Indonesian holiday calendar. The seed is idempotent by holiday_date.
- Holiday data is also supplied as docs/holidays-id-2025-2027.csv for review and reproducible seeding.
- No service runs another service's migrations. Database creation is handled by the PostgreSQL init script in Docker Compose.

5.9 Data lifecycle and deletion

Table 24 - Data lifecycle rules

Entity	Lifecycle rule
User	Deactivate instead of delete. Historical actor UUIDs remain valid. A deactivated user cannot login or refresh.
Project	Archive instead of delete. Archived projects are read-only for tasks and timelines; admin can restore.
Project member	Can be removed. Existing tasks keep the PIC UUID and name snapshot; editing such a task requires reassigning the PIC to a current member.
Task	Hard delete allowed for project members/admin. No soft-delete in v1.
Credential	Owned by auth-service. Deactivated users keep their credential row for audit but cannot authenticate; password changes are recorded with password_changed_at.
Timeline	Created once per project and never deleted after a version is published. If the only draft of a never-published timeline is cancelled, the timeline identity is removed in the same transaction and the project can create a new timeline.
Timeline draft	Can be edited or cancelled. Cancelled drafts remain in history with lifecycle_status CANCELLED, except when cancelling the only draft of a never-published timeline (the identity is removed).
Holiday	Admin can add, update, or delete. Derived timeline values are recalculated on read; superseded versions keep their snapshots.
Refresh token	Revoked on rotation/logout/reuse, password change, role change, or deactivation. Expired rows can be purged by a future maintenance job.

6. Access Control and Business Rules

6.1 Roles and permission matrix

Table 25 - Role and permission matrix

Capability	Public	USER (project member)	USER (not a member)	ADMIN	Internal key
Login / refresh / logout	Yes	Yes	Yes	Yes	No
Current user (GET /auth/me)	No	Yes	Yes	Yes	No
View own profile	No	Yes	Yes	Yes	No
Change own password	No	Yes	Yes	Yes	No
List / create / update / deactivate users	No	No	No	Yes	No
Assign roles / admin password reset	No	No	No	Yes	No
Set initial password after user creation	No	No	No	Yes	No
Internal access / auth context / token revocation	No	No	No	No	Scoped caller key
List / view projects	No	Member only	No	All	No
Create / update / archive projects	No	No	No	Yes	No
View project members	No	Member only	No	All	No
Add / remove project members	No	No	No	Yes	No
Task CRUD and progress	No	Member only	No	All	No
Timeline read / history	No	Member only	No	All	No
Timeline create / revise / publish	No	Member only	No	All	No
Timeline status / actual progress	No	Member only	No	All	No
Holiday read	No	Yes	Yes	Yes	No
Holiday create / update / delete	No	No	No	Yes	No

Backend is authoritative

The frontend hides unavailable actions for usability, but every protected endpoint validates the JWT and project access on the server. Hidden buttons are never treated as security.

6.2 Project visibility and access check

1. The request carries an access JWT. The service validates the RS256 signature via cached JWKS, then checks issuer, audience, expiry, and roles locally.
2. If the route requires ADMIN and the token lacks ROLE_ADMIN, the service returns 403 immediately.
3. For project-scoped routes, task-service or timeline-service calls GET /internal/projects/{projectId}/access?userId={jwt.sub} on user-service using its scoped internal key.
4. user-service returns allowed=true when the user is an active ADMIN or an active project member. It also returns member=true/false and projectStatus.
5. If the project does not exist, the caller returns 404. If it exists but the user is not allowed, the caller returns 403.
6. If projectStatus is ARCHIVED, read requests are allowed and write requests return 409 PROJECT_ARCHIVED.
7. Task creation/update additionally requires the PIC to be an active member (member=true) of the same project.
8. Immediately before committing a project-scoped write, the service repeats the access check. If access was removed during the request, the transaction rolls back with 403. The residual race window is one internal round trip; see Section 6.9.

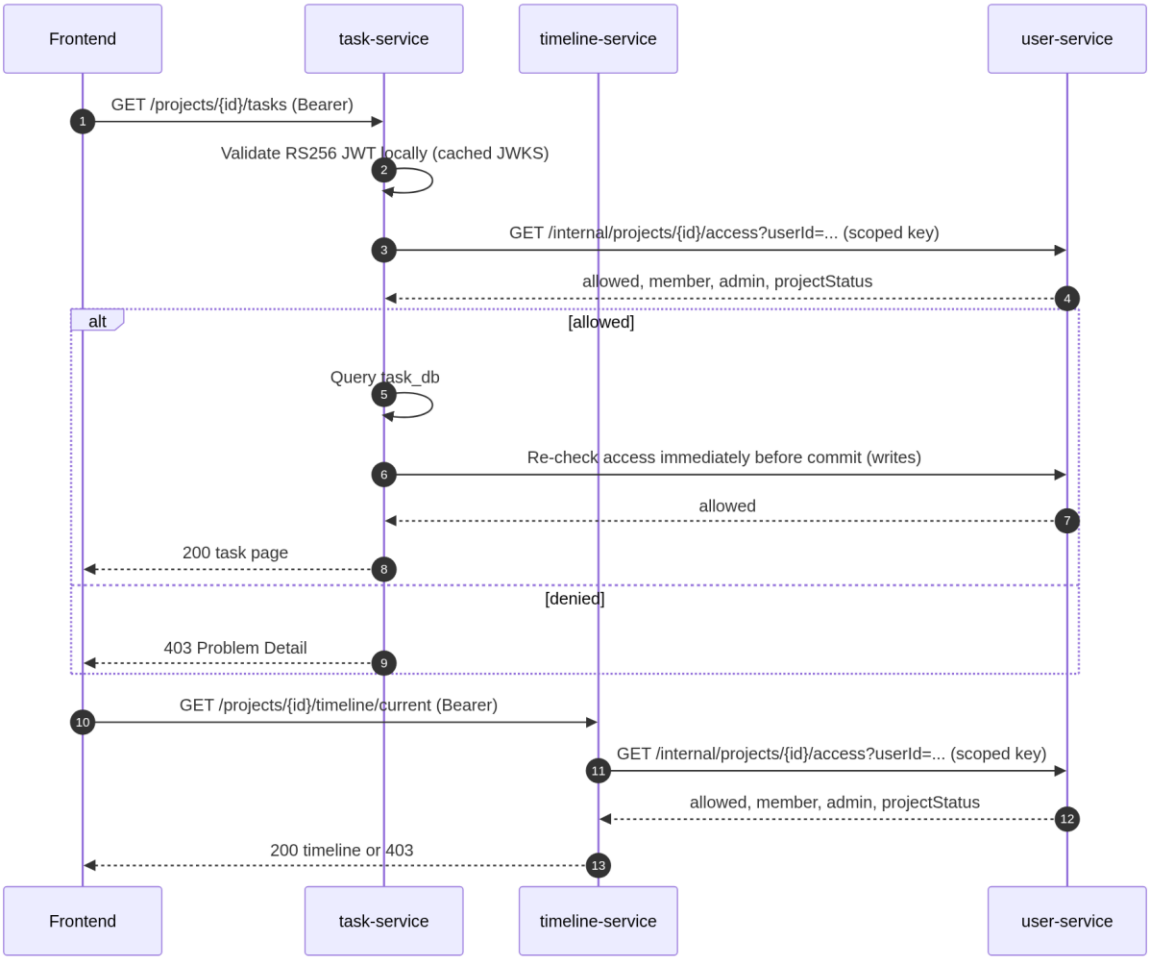


Figure 6 - Project access check used by task-service and timeline-service

6.3 Task rules

6.3.1 Fields and validation

Table 26 - Task field rules

Field	Rule
title	Required, 1-200 characters after trimming.
description	Optional, maximum 2,000 characters.
startDate / endDate	Required, ISO date, endDate >= startDate. Maximum range 10 years (3,650 days inclusive).
durationDays	Read-only, computed as endDate - startDate + 1 calendar days. Stored as a PostgreSQL generated column.
picUserId	Required. Must reference an active user who is a member of the project. Admin must be a member to be PIC.
picName	Read-only in responses. Snapshot of the user's full name at create/update time.
status	OPEN, IN_PROGRESS, or CLOSED. Default OPEN.
percentComplete	Integer 0-100. Default 0.

6.3.2 Duration example

A task from Monday 2 November 2026 to Friday 6 November 2026 has a duration of 5 calendar days. A task from Monday 2 November 2026 to Monday 9 November 2026 has a duration of 8 calendar days, because Saturday and Sunday are included in task duration. Task duration intentionally does not exclude holidays; only timeline duration uses the working calendar.

6.3.3 Status and completion

Table 27 - Task status transitions

Action	Allowed result
OPEN -> IN_PROGRESS	Allowed.
OPEN -> CLOSED	Allowed; percentComplete is set to 100.
IN_PROGRESS -> CLOSED	Allowed; percentComplete is set to 100.
CLOSED -> IN_PROGRESS	Allowed (reopen); percentComplete is not changed automatically.
CLOSED -> OPEN	Not allowed. Reopen to IN_PROGRESS, then reset progress if needed.
IN_PROGRESS -> OPEN	Not allowed. Use CLOSED or continue work.
PATCH progress to 100	Allowed; status is not changed automatically. The user can close the task separately.
PATCH progress while CLOSED	Rejected with 409 TASK_CLOSED. Reopen the task before changing progress.
PUT with CLOSED and percentComplete < 100	Rejected with 400 INVALID_COMPLETION_FOR_CLOSED; a closed task must be 100%.

6.3.4 Delete and archived projects

- Any project member or ADMIN can create, update, and delete tasks in an ACTIVE project.
- Deleting a task is a hard delete and returns 204. There is no restore endpoint in v1.
- When a project is ARCHIVED, task reads remain available but task create/update/delete/status/progress writes return 409 PROJECT_ARCHIVED.
- A removed project member loses access immediately. If that user is still the PIC of a task, the task remains readable; editing it requires the caller to choose a current member as PIC.

6.4 Timeline rules and versioning

6.4.1 Timeline aggregate

- A project has exactly zero or one timeline. The timeline identity is created with the first draft.
- A timeline has ordered versions. Version numbers start at 1 and increase by one, including cancelled drafts.
- At most one DRAFT version and one PUBLISHED version may exist per timeline. This is enforced by partial unique indexes and by lifecycle writes that lock the timeline row.
- A published version's planned dates and revision notes are immutable. Only the current published version's operational status and actual completion can be updated.
- Superseded and cancelled versions are read-only and carry frozen duration, plan, and actual snapshots.
- A draft is editable and is not active for the project until published. The current published version remains active while a draft exists. A draft contains planned dates and notes only; operational status and actual completion are operational fields, not revision fields. DRAFT and CANCELLED rows store the defaults OPEN and 0; the API reports those stored draft values, and the current version's operational values are read from the current version.
- Publishing applies the draft's planned dates and notes, carries forward the operational status and actual completion from the version that is current at publish time, snapshots the outgoing current version, and then supersedes it.
- Cancelling a draft keeps it in version history as CANCELLED, except when it is the only draft and no version has ever been published: in that case the timeline identity is removed in the same transaction so the project can start a new timeline.
- Draft edit, revision create, and cancel accept expectedDraftUpdatedAt. Publish accepts expectedDraftUpdatedAt and requires expectedCurrentUpdatedAt when a current version exists. A concurrent change returns 409 DRAFT_MODIFIED or 409 CURRENT_VERSION_CHANGED instead of overwriting it.

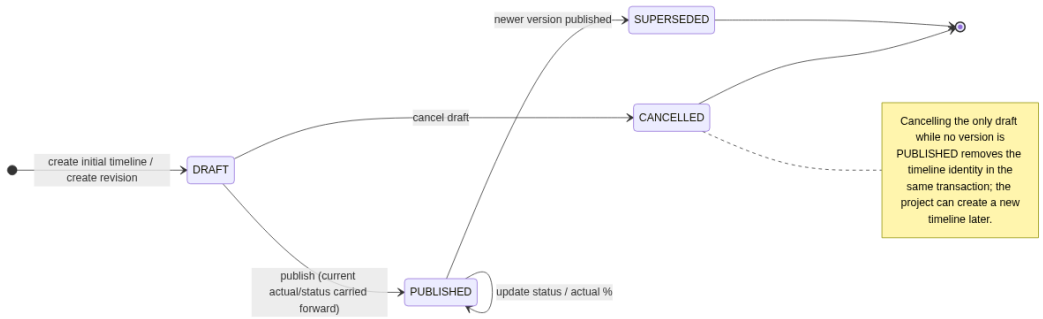


Figure 7 - Timeline version lifecycle

6.4.2 Draft and publish workflow

Table 28 - Timeline draft/publish workflow

Step	Action	Result
1	POST /projects/{id}/timeline with start/end dates and requestId	Create timeline identity and version 1 in DRAFT. Operational status OPEN, actual 0.
2	PUT /projects/{id}/timeline/draft	Edit planned dates and revision notes while in DRAFT. Status and actual completion are not draft fields.
3	POST /projects/{id}/timeline/draft/publish	Version 1 becomes PUBLISHED with OPEN/0 carried forward. It is the current version.
4	POST /projects/{id}/timeline/revisions	Create version N+1 in DRAFT with the current planned dates. Current remains active.
5	PUT /projects/{id}/timeline/draft	Adjust planned dates and notes. The current version is unaffected.
6	POST /projects/{id}/timeline/draft/publish	Draft becomes PUBLISHED/current with the current status/actual carried forward; previous current becomes SUPERSEDED with a snapshot.
7	DELETE /projects/{id}/timeline/draft	Draft becomes CANCELLED; current remains unchanged. If no version was ever published, the timeline identity is deleted.

6.4.3 Timeline fields and validation

Table 29 - Timeline field rules

Field	Rule
startDate / endDate	Required, ISO date, endDate >= startDate. Must contain at least one working day at create/edit time. Must lie inside the holiday coverage horizon (currently 2025-01-01 to 2027-12-31); otherwise 409 HOLIDAY_COVERAGE_REQUIRED.
durationWorkdays	Read-only, inclusive count of working days excluding Saturday, Sunday, national holidays, and joint leave. Zero for an existing timeline whose remaining days were later covered by holidays; reads then set planWarning.
operationalStatus	OPEN, IN_PROGRESS, or CLOSED. Operational field of the current published version. It is carried forward at publish and is not editable on a draft.
percentActual	Integer 0-100. Operational field of the current published version. It is carried forward at publish and is not editable on a draft.
percentPlan	Read-only, derived from elapsed working days and total working days. Rounded to 2 decimals, capped 0-100. Frozen in the snapshot when a version is superseded or cancelled.
planAsOfDate	Read-only. Today in Asia/Jakarta for the current version; snapshot_as_of_date for SUPERSEDED/CANCELLED versions.
planWarning	Read-only. NO_WORKING_DAYS when totalWorkingDays = 0; null otherwise. The API returns durationWorkdays=0 and percentPlan=0.00 instead of dividing by zero.
revisionNotes	Optional, maximum 1,000 characters. Recommended for every revision.
versionNumber	Read-only, assigned by the service.
lifecycleStatus	DRAFT, PUBLISHED, SUPERSEDED, CANCELLED.
createdBy / createdAt, updatedBy / updatedAt	Audit fields for the draft/current version.
supersededBy / supersededAt, cancelledBy / cancelledAt	Actor and timestamp for the terminal transition.

6.4.4 Operational status

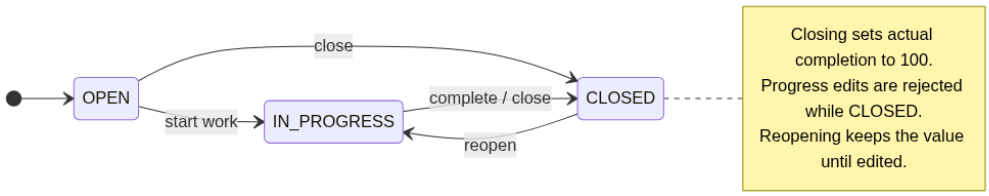


Figure 8 - Task and timeline operational status transitions

Operational status is separate from version lifecycle. A timeline can be PUBLISHED and OPEN, PUBLISHED and IN_PROGRESS, or PUBLISHED and CLOSED. Closing a task or the current timeline sets actual completion to 100. Progress edits are rejected while CLOSED with 409 TASK_CLOSED or 409 TIMELINE_CLOSED; reopen first. A full update that sets CLOSED with a percentage below 100 is rejected with 400 INVALID_COMPLETION_FOR_CLOSED. PostgreSQL CHECK constraints repeat the rule as a final guard. Reopening keeps the value until the user edits it. Plan completion is always calculated from dates and is not forced to 100 by closing early.

6.5 Working-day calculation

Timeline duration and plan completion use the Indonesia working calendar. The calculation is inclusive of both start and end dates and excludes every Saturday, every Sunday, every NATIONAL holiday, and every JOINT_LEAVE row in the holiday table.

```
countWorkingDays(startDate, endDate):
    if endDate < startDate: reject
    holidays = holidayRepository.findByHolidayDateBetween(startDate, endDate)  # one query
    count = 0
    for each date d from startDate to endDate inclusive:
        if dayOfWeek(d) is Saturday or Sunday: continue
        if d in holidays: continue
        count = count + 1
    return count
```

- Creating or editing a timeline whose range contains zero working days is rejected with 400 NO_WORKING_DAYS.
- Holiday edits can later reduce an existing timeline to zero working days. The service does not divide by zero: reads return durationWorkdays=0, percentPlan=0.00, and planWarning=NO_WORKING_DAYS.
- The holiday set for the requested range is loaded in one query and checked with a set membership test. The date loop is single-pass.
- Official holidays that fall on a weekend remain in the holiday table but do not reduce the working-day count further.
- The calculation uses date-only values in Asia/Jakarta. There is no time-of-day component.
- Timeline creates and edits are bounded by the holiday coverage horizon, so the maximum range is currently three years and the loop is cheap.

6.6 Plan completion calculation

Plan completion answers: at the end of the as-of date, how much of the planned working duration should have elapsed? The daily increment is 100% divided by the total number of working days. The result is capped at 0 before the start date and at 100 after the end date.

```
totalWorkingDays = countWorkingDays(startDate, endDate)
if totalWorkingDays == 0:
    return { durationWorkdays: 0, percentPlan: 0.00, planWarning: "NO_WORKING_DAYS" }

effectiveAsOf = min(asOfDate, endDate)
elapsedWorkingDays = 0
if asOfDate >= startDate:
    elapsedWorkingDays = countWorkingDays(startDate, effectiveAsOf)

dailyIncrement = 100 / totalWorkingDays
percentPlan = min(100, elapsedWorkingDays * dailyIncrement)
percentPlan = roundHalfUp(percentPlan, 2)
```

- If asOfDate is before startDate: percentPlan = 0.00.
- If asOfDate is on or after endDate: percentPlan = 100.00.
- If asOfDate is a weekend or holiday: it does not add a working day; the value stays at the last working day's increment.
- If totalWorkingDays = 0 after a holiday edit: percentPlan = 0.00, durationWorkdays = 0, planWarning = NO_WORKING_DAYS. The service never divides by zero.
- The as-of date is today in Asia/Jakarta for the current version. For superseded and cancelled versions, the API returns the stored snapshot values and snapshot_as_of_date.
- Percent plan is calculated on read for the current version. When a version is superseded, duration, plan, actual, and as-of date are snapshotted so later holiday corrections do not rewrite history.

6.6.1 Worked example with holidays

Timeline: Monday 16 March 2026 to Friday 27 March 2026. The date range includes Nyepi on 19 March, joint leave on 18, 20, 23, and 24 March, and the weekend of 21-22 March. Working days are 16, 17, 25, 26, and 27 March: 5 days.

Table 30 - Plan completion example with the 2026 Indonesian calendar

As-of date	Elapsed working days	Calculation	Plan %
2026-03-15 (Sunday, before start)	0	0 x 20.00	0.00%
2026-03-16 (Monday)	1	1 x 20.00	20.00%
2026-03-17 (Tuesday)	2	2 x 20.00	40.00%
2026-03-18 (joint leave)	2	2 x 20.00	40.00%
2026-03-19 (Nyepi)	2	2 x 20.00	40.00%
2026-03-20 (joint leave)	2	2 x 20.00	40.00%
2026-03-23 (joint leave)	2	2 x 20.00	40.00%
2026-03-25 (Wednesday)	3	3 x 20.00	60.00%
2026-03-27 (Friday, end)	5	5 x 20.00	100.00%
2026-03-30 (after end)	5	capped	100.00%

6.7 Holiday rules

- Holiday data is owned by timeline-service and applies to all projects.
- Both NATIONAL and JOINT_LEAVE rows are excluded from working days, matching the Indonesian government's joint-leave practice.
- holiday_date is unique. Duplicate dates return 409 HOLIDAY_DATE_TAKEN.
- A holiday on a Saturday or Sunday is allowed because official calendars contain weekend holidays; it simply has no additional effect on the working-day count.
- Only ADMIN can create, update, or delete holidays. Any authenticated user can read them. Every write records created_by or updated_by.

- The coverage horizon is derived from the holiday table: currently 2025-01-01 through 2027-12-31. Timeline creates and edits outside the horizon return 409 HOLIDAY_COVERAGE_REQUIRED until an admin adds holiday data for the missing year.
- Deleting or adding a holiday changes derived duration and plan percentages on the next read. Superseded versions keep their snapshots.
- A holiday edit can reduce an existing timeline to zero working days. The edit is allowed; affected reads return durationWorkdays=0, percentPlan=0.00, and planWarning=NO_WORKING_DAYS.
- The seed covers 2025, 2026, and 2027. Future years are added by an administrator or a new Flyway migration.

6.8 User and project rules

Table 31 - User and project validation rules

Area	Rule
Username	Required, unique, lowercase, 3-50 characters, [a-z0-9._-].
Email	Required, unique, lowercase, valid format, maximum 255 characters.
Password	8-64 characters, at least one letter and one digit, BCrypt cost 12 inside auth-service. Admin reset and self change revoke all refresh-token families.
Credential	Stored only in auth_db.auth_credential. A profile can exist before its initial password is set; such a user cannot log in until an admin sets it.
Roles	At least one role. Fixed roles ADMIN and USER. The service prevents removal of the last active ADMIN.
Self-deactivation	An admin cannot deactivate their own account.
Project code	Required, unique, uppercase, 2-30 characters, [A-Z0-9-].
Project dates	endDate >= startDate. Project dates do not drive task or timeline calculations; they are descriptive boundaries.
Project members	An admin can add active users and remove members. Removing a member does not delete their historical task references.
Archived project	Reads allowed, writes blocked for tasks and timelines. Restore is an admin action.
Bootstrap admin	On first startup, auth-service creates the ADMIN credential only if no active admin credential exists and BOOTSTRAP_ADMIN_PASSWORD is provided. It calls user-service to ensure the profile and ADMIN role exist.

6.9 Concurrency and transaction boundaries

- Timeline create, revision create, draft edit, cancel, publish, and the current status/progress updates (TL10/TL11) all lock the timeline row (SELECT ... FOR UPDATE) for the transaction. A progress update and a publish therefore serialize: whichever transaction commits first is seen by the other, so publish cannot overwrite a concurrently committed progress change. The partial unique indexes on DRAFT and PUBLISHED are the final guard against two concurrent drafts or two current versions.
- Publish requires expectedCurrentUpdatedAt whenever a current version exists (null is accepted only for the first publish of a timeline), and accepts expectedDraftUpdatedAt. A missing, null, or mismatched current timestamp returns 409 CURRENT_VERSION_CHANGED; a mismatched draft timestamp returns 409 DRAFT_MODIFIED, rather than overwriting a concurrent change.
- Create and publish endpoints are replay-safe through unique request_id and publish_request_id columns. A retry returns the original resource with 200 and Idempotent-Replay: true.
- The last-active-ADMIN check and the role/status update run under pg_advisory_xact_lock in user-service, so two concurrent admin changes cannot remove the last administrator.
- Project-scoped writes perform the user-service access check again immediately before commit and roll back with 403 if access was removed. The residual race window is one internal round trip; membership changes are infrequent and the window is accepted for release 1.

- Deactivation and role changes call auth-service to revoke refresh tokens after the local commit. If that call fails, the refresh flow still rejects the user because auth-service reads the current enabled state and roles from user-service on every refresh.
- Task updates use a normal read-modify-write transaction. Release 1 does not implement optimistic locking or ETags for tasks; simultaneous edits follow last-write-wins. Timeline versions do use optimistic checks because they are the audit baseline.
- User, role, project, and membership changes are single-database transactions inside user-service.
- There are no distributed transactions. Cross-service operations are synchronous reads (auth context, access check, PIC lookup) and one idempotent revoke-tokens write.

7. API Conventions

All external APIs use JSON over HTTPS and are versioned under /api/v1. Internal service APIs use /internal and are protected by per-caller keys with an endpoint scope matrix. The browser never calls internal APIs.

7.1 Base URLs and ports

Table 32 - Local service URLs

Service	Local base URL	API docs (non-production)
auth-service	http://localhost:8081/api/v1	http://localhost:8081/swagger-ui.html
user-service	http://localhost:8082/api/v1	http://localhost:8082/swagger-ui.html
task-service	http://localhost:8083/api/v1	http://localhost:8083/swagger-ui.html
timeline-service	http://localhost:8084/api/v1	http://localhost:8084/swagger-ui.html
frontend (dev)	http://localhost:5173	Vite dev server
frontend (container)	http://localhost:3000 or http://localhost	Nginx static host

7.2 Request headers

Table 33 - Request headers

Header	Required	Purpose
Authorization: Bearer <jwt>	Yes for protected endpoints	Access token issued by auth-service.
Content-Type: application/json	Yes when a body is present	JSON request body.
Accept: application/json	Recommended	JSON response.
X-Request-Id	Optional	Client correlation ID. Generated by the service if absent and propagated internally.
X-Internal-Key-Id	Yes for /internal/**	Identifies the caller key during rotation. Never used by the browser.
X-Internal-Api-Key	Yes for /internal/**	Per-caller secret compared in constant time. Never used by the browser.
Cookie: refresh_token=...	Refresh/logout	HttpOnly refresh token cookie. The browser sends it automatically with credentials.

Project, task, timeline create, revision create, and publish requests also carry a client-generated requestId (UUID) in the JSON body. A replay of the same requestId returns the original resource with the response header Idempotent-Replay: true and does not apply the body again. User, membership, and holiday creates do not carry a requestId in release 1: a retried create returns the normal duplicate error (409), and the caller should re-read the existing resource instead of retrying blindly.

7.3 Response format

- Single-resource success responses return the resource object directly with HTTP 200 or 201.
- Delete and logout success responses return 204 with no body.
- List responses use a consistent page envelope: content, page, size, totalElements, totalPages, first, last.
- Timeline version history and holiday list also use the page envelope. Superseded and cancelled versions return frozen snapshot fields, not live values.
- Errors use RFC 7807 Problem Details with application/problem+json and additional code, requestId, timestamp, and field errors.
- All timestamps are ISO-8601 UTC strings. All business dates are yyyy-MM-dd strings.
- UUIDs are lowercase strings.

7.3.1 Page response

```
{
  "content": [
    { "id": "4f4c2d7a-...", "title": "Design database" }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 42,
  "totalPages": 3,
  "first": true,
  "last": false
}
```

7.4 Pagination, sorting, and search

- Query parameters: page (0-based, default 0), size (default 20, maximum 100), sort (field,direction).
- Sort direction is asc or desc. Only allow-listed fields are accepted; unknown sort fields return 400.
- Search parameter q is trimmed, maximum 100 characters, and escaped before use in a JPQL LIKE or lower() expression.
- User list filters: q, enabled, role. Project list filters: q, status. Task list filters: q, status, picUserId.
- Timeline version history uses the same envelope with default size 50 and sort versionNumber,desc. The holiday list uses the envelope with a year filter and default size 100.
- Invalid page or size values return 400 VALIDATION_ERROR.

7.5 Error contract

Every service uses the same error shape. The code field is stable and is the value the frontend uses for branching. The detail field is human-readable and safe to display.

```
HTTP/1.1 400 Bad Request
Content-Type: application/problem+json

{
  "type": "https://prolean.local/errors/validation-error",
  "title": "Validation failed",
  "status": 400,
  "detail": "One or more fields are invalid.",
  "instance": "/api/v1/projects/8f.../tasks",
  "code": "VALIDATION_ERROR",
  "requestId": "01J...",
  "timestamp": "2026-10-19T10:15:30Z",
  "errors": [
    { "field": "endDate", "message": "must be on or after startDate" }
  ]
}
```

7.6 HTTP status usage

Table 34 - HTTP status conventions

Status	Use
200 OK	Successful read or update.
201 Created	Successful create. Location header points to the resource where applicable.
204 No Content	Successful delete, logout, or membership removal.
400 Bad Request	Malformed JSON, validation failure, invalid date range, unknown sort field.
401 Unauthorized	Missing/invalid/expired access token, invalid credentials, invalid refresh token.
403 Forbidden	Authenticated but not allowed, disabled account, missing role.

Status	Use
404 Not Found	Unknown resource, no timeline, or no draft. Use 409, not 404, when a current version is required but none exists.
409 Conflict	Duplicate unique value, invalid lifecycle state, archived project, last-admin guard, holiday coverage horizon, concurrent version conflict.
423 Locked	Credential is locked after repeated failed logins.
500 Internal Server Error	Unexpected server error; no stack trace in the response.
503 Service Unavailable	A required internal dependency is unavailable.

7.7 Stable error codes

Table 35 - Stable error code catalogue

Error code	HTTP	Meaning
VALIDATION_ERROR	400	Request body or query parameter failed validation.
INVALID_DATE_RANGE	400	endDate is before startDate.
NO_WORKING_DAYS	400	Timeline range excludes every day as weekend/holiday.
AUTH_INVALID_CREDENTIALS	401	Username or password is incorrect.
AUTH_ACCOUNT_DISABLED	403	Account is deactivated.
AUTH_ACCOUNT_LOCKED	423	Credential is locked for 15 minutes after five consecutive failed logins.
AUTH_REFRESH_INVALID	401	Refresh token is missing, expired, revoked, or unknown.
AUTH_REFRESH_REUSED	401	A revoked refresh token was reused; all active tokens are revoked.
UNAUTHORIZED	401	Access token is missing, invalid, or expired.
FORBIDDEN	403	Authenticated user lacks the required role or project access.
INTERNAL_UNAUTHORIZED	401	Internal API key ID is missing or the secret is invalid.
INTERNAL_FORBIDDEN	403	A valid internal caller key was used outside its allowed endpoint scope.
USER_NOT_FOUND	404	User does not exist.
USERNAME_TAKEN	409	Username already exists.
EMAIL_TAKEN	409	Email already exists.
LAST_ADMIN	409	Operation would remove the last active ADMIN.
SELF_DEACTIVATION	409	An admin cannot deactivate their own account.
ROLE_NOT_FOUND	404	Requested role does not exist.
PROJECT_NOT_FOUND	404	Project does not exist.
PROJECT_CODE_TAKEN	409	Project code already exists.
PROJECT_ARCHIVED	409	Project is archived and read-only.
MEMBER_NOT_FOUND	404	User is not a project member.
TASK_NOT_FOUND	404	Task does not exist.
PIC_NOT_MEMBER	400	PIC is not an active member of the project.
TIMELINE_EXISTS	409	Project already has a timeline.
TIMELINE_NOT_FOUND	404	Project has no timeline.
DRAFT_EXISTS	409	A draft already exists for this timeline.
DRAFT_NOT_FOUND	404	No editable draft exists.
NO_CURRENT_VERSION	409	A revision was requested but no published version exists.
CURRENT_VERSION_CHANGED	409	The current published version changed, or expectedCurrentUpdatedAt was missing when required; reload before publishing.
DRAFT_MODIFIED	409	The draft changed since it was loaded; reload before saving or publishing.
INVALID_COMPLETION_FOR_CLOSED	400	A closed task or timeline was submitted with completion below 100.
TASK_CLOSED	409	Progress cannot be edited while the task is CLOSED; reopen first.
TIMELINE_CLOSED	409	Progress cannot be edited while the current timeline is CLOSED; reopen first.
INVALID_TIMELINE_STATE	409	Requested operation is not allowed for the current timeline lifecycle state.
INVALID_STATUS_TRANSITION	409	Requested task or timeline status transition is not allowed.

Error code	HTTP	Meaning
HOLIDAY_DATE_TAKEN	409	A holiday already exists on that date.
HOLIDAY_COVERAGE_REQUIRED	409	Timeline dates fall outside the covered holiday horizon; add holiday data for that year first.
HOLIDAY_NOT_FOUND	404	Holiday does not exist.
INTERNAL_DEPENDENCY_UNAVAILABLE	503	user-service could not be reached or timed out.
INTERNAL_ERROR	500	Unexpected server error.

7.8 Internal API security

- Internal endpoints are under /internal/** and are not part of the public API contract.
- Every caller has a key ID and a 256-bit secret. The caller sends X-Internal-Key-Id and X-Internal-Api-Key; the receiver looks up the key ID, compares the secret in constant time, and checks that the caller is allowed to call that endpoint.
- Keys are configured per receiving service as a map of keyId to {caller, secret}. Rotation uses two active key IDs per caller: the new key is deployed to receivers first, then to the caller, then the old key is removed.
- The caller scope matrix below is enforced by each receiving service. A valid key for the wrong scope returns 403 INTERNAL_FORBIDDEN.
- In production, the edge reverse proxy must block /internal/** from external traffic and internal traffic should use TLS (mTLS recommended). In Docker Compose, the internal endpoints are reachable only on the private bridge network and are key-protected.
- Internal responses use the same Problem Details error shape. Internal callers map dependency failures to 503 INTERNAL_DEPENDENCY_UNAVAILABLE.
- Internal endpoints never return password hashes, refresh-token values, or signing keys.

Table 36 - Internal caller scope matrix

Caller	Allowed internal targets and endpoints
auth-service	user-service: GET /internal/users/{userId}, GET /internal/users/{userId}/auth-context, POST /internal/users/bootstrap. No project-access or task endpoints.
task-service	user-service: GET /internal/projects/{projectId}/access, GET /internal/users/{userId}. No auth endpoints.
timeline-service	user-service: GET /internal/projects/{projectId}/access. No user-detail or auth endpoints.
user-service	auth-service: POST /internal/users/{userId}/revoke-tokens. No other endpoints.
frontend or external caller	None. /internal/** is blocked at the edge and no browser key exists.

7.9 OpenAPI and compatibility

- Each service exposes an OpenAPI 3.1 document generated by springdoc-openapi.
- Swagger UI is enabled in local/dev profiles and disabled or role-protected in production.
- This document is the design baseline. During implementation, the generated OpenAPI documents must match the endpoint tables and schemas in Section 8.
- Additive changes (new optional fields, new endpoints) are backward compatible. Removing fields, renaming fields, or changing status semantics requires a versioned API.
- The frontend uses explicit service base URLs from runtime configuration; no gateway URL rewriting is assumed.

8. Feature Specifications

Every feature below includes its actors, sequence diagram, API contract, business rules, and error cases. The API tables are the approval baseline for the generated OpenAPI documents.

8.1 Authentication

8.1.1 Purpose and actors

Authentication proves who the user is and issues the access and refresh credentials used by the SPA. The user enters a username and password. auth-service loads the credential from auth_db, verifies the BCrypt hash locally, reads current account status and roles from user-service, issues an RS256 access JWT, and stores a hashed refresh token in a token family. The browser receives the access token in the response body and the refresh token in an HttpOnly cookie.

- Actors: any registered active user.
- Preconditions: the user profile exists in user-service, a credential exists in auth-service, the account is enabled, and the credential is not locked.
- Postconditions: the browser has an access token in memory and a refresh cookie; auth_db has a hashed refresh-token row in a new family.

8.1.2 Login sequence

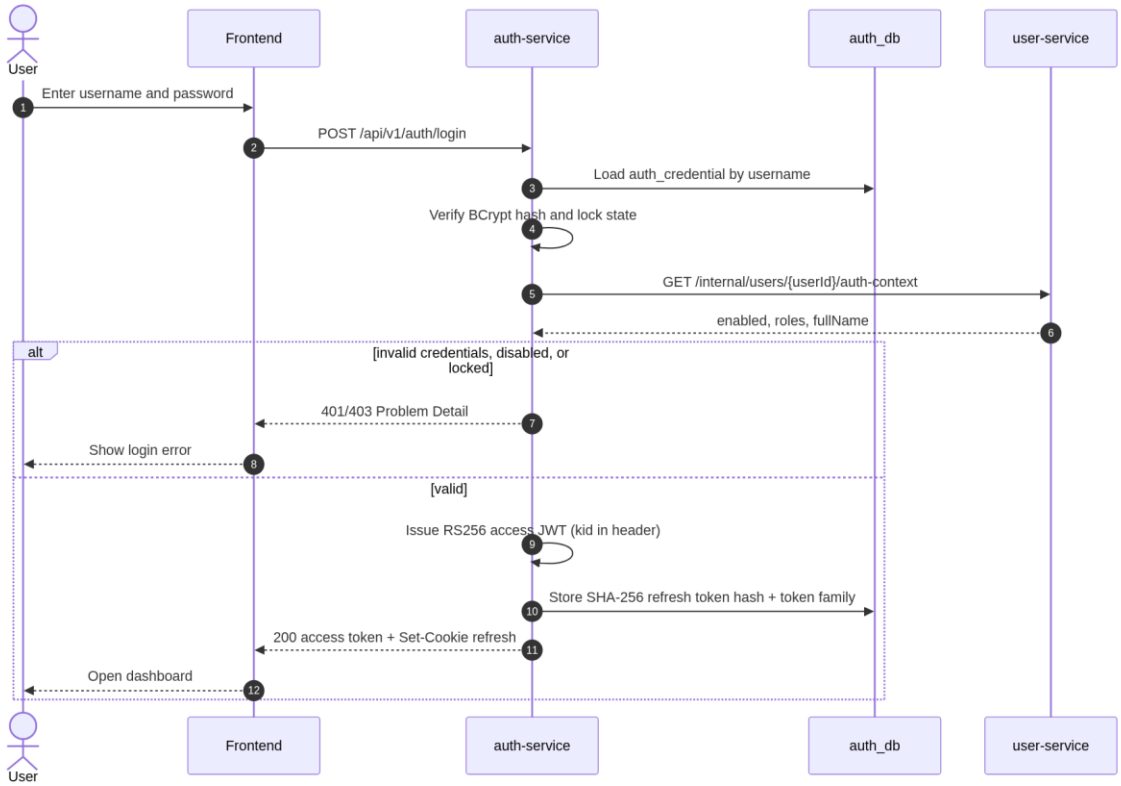


Figure 9 - Login sequence

8.1.3 Authentication endpoints

Table 37 - Authentication API endpoints

#	Method	Path	Auth	Purpose
A1	POST	/api/v1/auth/login	Public	Verify credentials and create a session.
A2	POST	/api/v1/auth/refresh	Refresh cookie	Rotate the refresh token and issue a new access token.
A3	POST	/api/v1/auth/logout	Refresh cookie	Revoke the refresh token and clear the cookie.

#	Method	Path	Auth	Purpose
A4	GET	/api/v1/auth/me	Bearer	Return the current user's claims.
A5	PUT	/api/v1/auth/users/{userId}/password	ADMIN	Set or reset a user's password. Revokes all refresh-token families.
A6	PUT	/api/v1/auth/me/password	Bearer	Change own password with the current password. Revokes all refresh-token families.
A7	GET	/.well-known/jwks.json	Public	Publish the RS256 public keys used to validate access tokens.

8.1.4 POST /api/v1/auth/login

Table 38 - Login request fields

Field	Type	Required	Validation
username	string	Yes	1-50 characters; trimmed; matched case-insensitively after lowercase normalisation.
password	string	Yes	1-128 characters in the request; verified inside auth-service with BCrypt and never forwarded to another service.

```
POST /api/v1/auth/login
Content-Type: application/json

{
  "username": "admin",
  "password": "S3cretPass!"
}

HTTP/1.1 200 OK
Content-Type: application/json
Set-Cookie: refresh_token=...; HttpOnly; Secure; SameSite=Lax; Path=/api/v1/auth; Max-Age=604800

{
  "accessToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6IjIwMjYtMTAiLCJ0eXAiOiJKV1QifQ...",
  "tokenType": "Bearer",
  "expiresIn": 900,
  "user": {
    "id": "0f4e0d3a-...",
    "username": "admin",
    "fullName": "System Administrator",
    "roles": ["ADMIN"]
  }
}
```

- The refresh token is never returned in the JSON body.
- Secure is omitted only in the local HTTP profile. SameSite=Lax is used because the SPA and auth-service may be different localhost ports during development.
- The cookie Path is /api/v1/auth so it is not sent to business endpoints.
- Every failed login returns a generic 401 AUTH_INVALID_CREDENTIALS for unknown users and wrong passwords. A disabled account with correct credentials returns 403 AUTH_ACCOUNT_DISABLED. A locked credential returns 423 AUTH_ACCOUNT_LOCKED.
- Five consecutive failures lock the credential for 15 minutes. A successful login resets the counter. Unknown usernames are not counted because there is no credential row.
- A profile without an auth_credential row (for example, created but initial password not yet set) always returns the generic 401; it never reveals that the credential is missing.

8.1.5 POST /api/v1/auth/refresh

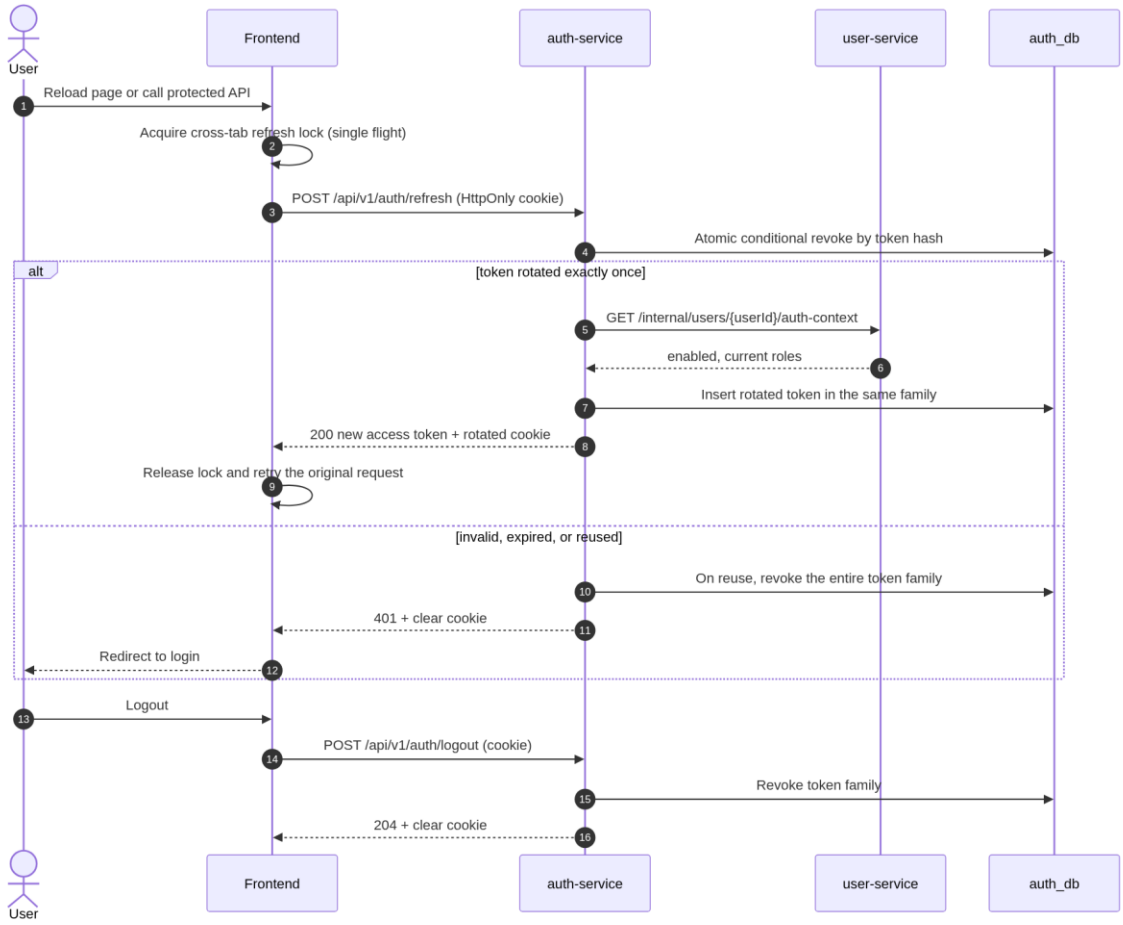


Figure 10 - Refresh and logout sequences

The request has no JSON body. The browser sends the refresh cookie automatically because Axios is configured with withCredentials=true. auth-service performs an atomic conditional revoke of the old token by SHA-256 hash, confirms the user is still enabled and gets current roles from user-service, and inserts the rotated token in the same family.

```
POST /api/v1/auth/refresh
Cookie: refresh_token=...

HTTP/1.1 200 OK
Content-Type: application/json
Set-Cookie: refresh_token=<new value>; HttpOnly; Secure; SameSite=Lax; Path=/api/v1/auth; Max-Age=604800

{
  "accessToken": "eyJhbGciOiJSUzI1NiIsImtpZCI6IjIwMjYtMTAiLCJ0eXAiOiJKV1QifQ...",
  "tokenType": "Bearer",
  "expiresIn": 900
}
```

- A missing, expired, revoked, or unknown token returns 401 AUTH_REFRESH_INVALID and clears the cookie.
- If a revoked token is presented, auth-service treats it as reuse, revokes the entire token family, and returns 401 AUTH_REFRESH_REUSED.
- Only one concurrent rotation of the same token can succeed because the revoke update is conditional on revoked_at being null. The frontend cross-tab lock prevents the losing request in normal use.
- The new access token uses the user's current roles, so a role change takes effect at the next refresh without waiting for the old access token to expire.
- The refresh cookie is rotated on every successful refresh.

- Password change, role change, and deactivation revoke all active families; the next refresh fails and the user must log in again.

8.1.6 POST /api/v1/auth/logout

```
POST /api/v1/auth/logout
Cookie: refresh_token=...

HTTP/1.1 204 No Content
Set-Cookie: refresh_token=; HttpOnly; Secure; SameSite=Lax; Path=/api/v1/auth; Max-Age=0
```

Logout is idempotent. A missing or already-revoked cookie still returns 204 and clears the cookie. The access token is discarded by the frontend; it remains cryptographically valid until its 15-minute expiry.

8.1.7 GET /api/v1/auth/me

```
GET /api/v1/auth/me
Authorization: Bearer <access token>

HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": "0f4e0d3a-...",
  "username": "admin",
  "fullName": "System Administrator",
  "roles": ["ADMIN"]
}
```

The values come from the validated JWT claims. A name change is reflected after the next refresh or login. /me does not query user-service, so it remains available if user-service is temporarily down.

8.1.8 Authentication errors

Table 39 - Authentication error cases

Condition	HTTP	Code
Missing username or password	400	VALIDATION_ERROR
Unknown username or wrong password	401	AUTH_INVALID_CREDENTIALS
Disabled account with valid password	403	AUTH_ACCOUNT_DISABLED
Credential locked after five failures	423	AUTH_ACCOUNT_LOCKED
Missing or invalid access token	401	UNAUTHORIZED
Missing/invalid refresh cookie	401	AUTH_REFRESH_INVALID
Reused revoked refresh token	401	AUTH_REFRESH_REUSED
user-service unavailable during login/refresh	503	INTERNAL_DEPENDENCY_UNAVAILABLE

8.1.9 Password set, reset, and change

A5 and A6 are the only endpoints other than login that accept a plaintext password. Both run inside auth-service, hash with BCrypt cost 12, update auth_credential, reset the failure counter, and revoke every active refresh-token family for the user.

Table 40 - Password endpoint fields

Endpoint	Field	Type	Required	Rule
A5 admin set/reset	newPassword	string	Yes	8-64 chars, at least one letter and one digit. The target user must exist; a disabled user can have a password set but remains disabled.
A6 self change	currentPassword	string	Yes	Must match the stored BCrypt hash.

Endpoint	Field	Type	Required	Rule
A6 self change	newPassword	string	Yes	Same policy and must differ from currentPassword.

- A5 is the second step of user creation: the admin creates the profile in user-service, then sets the initial password in auth-service.
- A5 requires ROLE_ADMIN and validates the target user with GET /internal/users/{userId}. It returns 404 USER_NOT_FOUND when the profile does not exist.
- A5 and A6 return 204 No Content. They never return the hash or the plaintext.
- A6 returns 400 VALIDATION_ERROR when the current password is wrong or the new password equals the current one.
- After A5 or A6, every refresh token is revoked. The frontend clears its session and navigates to login. Existing access tokens expire within 15 minutes.

8.2 User and Role Management

8.2.1 Purpose and actors

ADMIN manages user profiles and fixed role assignments. Password credentials are managed separately by auth-service: after creating a profile, the admin sets the initial password through auth-service, and users change their own password through auth-service. There is no user self-registration, email invitation, or dynamic permission editor in release 1. Deactivation is used instead of deletion so historical actor IDs remain valid.

8.2.2 User administration sequence

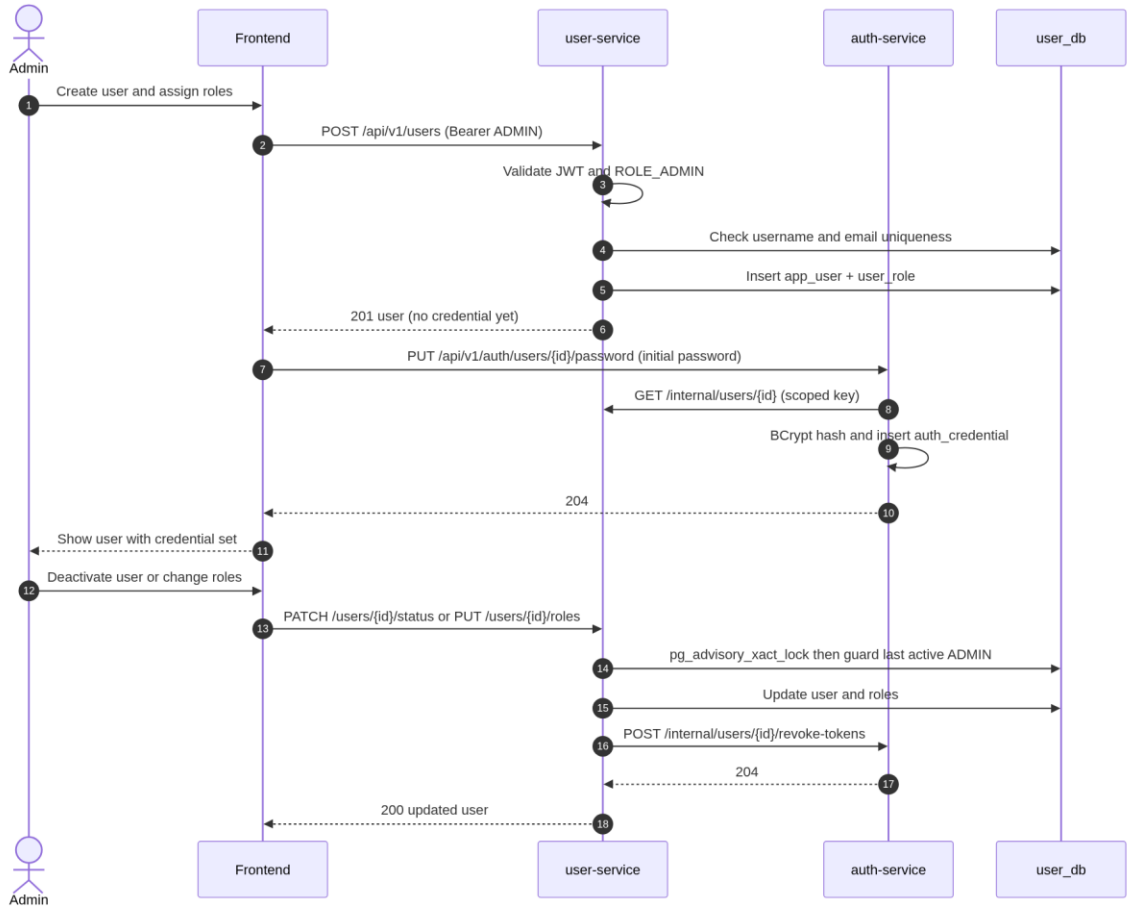


Figure 11 - User creation and role administration sequence

8.2.3 User and role endpoints

Table 41 - User and role API endpoints

#	Method	Path	Role	Purpose
U1	GET	/api/v1/users	ADMIN	Paginated user list with search, enabled, and role filters.
U2	POST	/api/v1/users	ADMIN	Create a user and assign initial roles.
U3	GET	/api/v1/users/{id}	ADMIN	User detail.
U4	PUT	/api/v1/users/{id}	ADMIN	Update full name and email.
U5	PATCH	/api/v1/users/{id}/status	ADMIN	Activate or deactivate a user.
U6	PUT	/api/v1/users/{id}/roles	ADMIN	Replace the user's roles.
U7	PUT	/api/v1/auth/users/{id}/password	ADMIN	Owned by auth-service (A5). Reset a password and revoke refresh tokens.
U8	GET	/api/v1/users/me	Authenticated	Current user's profile.

#	Method	Path	Role	Purpose
U9	PUT	/api/v1/auth/me/password	Bearer	Owned by auth-service (A6). Change own password and revoke refresh tokens.
U10	GET	/api/v1/roles	ADMIN	List fixed roles.

8.2.4 User create and update contracts

Table 42 - User create/update request fields

Field	Type	Required	Validation
username	string	Create	3-50 chars, lowercase, unique, pattern [a-z0-9._-].
email	string	Create/update	Valid email, lowercase, unique, max 255.
fullName	string	Create/update	1-100 chars.
roles	string[]	Create	At least one of ADMIN, USER. Default USER when omitted on create.
enabled	boolean	Create	Default true. Status changes use U5.

```
POST /api/v1/users
Authorization: Bearer <ADMIN token>
Content-Type: application/json

{
  "username": "budi.santoso",
  "email": "budi.santoso@example.com",
  "fullName": "Budi Santoso",
  "roles": ["USER"]
}

HTTP/1.1 201 Created
Location: /api/v1/users/2b2e...

{
  "id": "2b2e...",
  "username": "budi.santoso",
  "email": "budi.santoso@example.com",
  "fullName": "Budi Santoso",
  "enabled": true,
  "roles": ["USER"],
  "createdAt": "2026-10-19T08:00:00Z",
  "updatedAt": "2026-10-19T08:00:00Z"
}

// The admin UI then sets the initial credential in auth-service:
PUT /api/v1/auth/users/2b2e.../password
Authorization: Bearer <ADMIN token>
{ "newPassword": "InitialPass1" }

HTTP/1.1 204 No Content
```

8.2.5 User list and detail

GET /api/v1/users returns a page envelope. Profile responses never contain credential fields; password hashes live only in auth-service. The list supports q (username, email, full name), enabled, role, page, size, and sort. Allowed sort fields are username, fullName, email, enabled, createdAt. GET /api/v1/users/{id} returns the same detail shape for one user.

```
GET /api/v1/users?q=budi&enabled=true&role=USER&page=0&size=20&sort=username,asc
Authorization: Bearer <ADMIN token>

HTTP/1.1 200 OK

{
```

```
  "content": [
    {
      "id": "2b2e...",
      "username": "budi.santoso",
      "email": "budi.santoso@example.com",
      "fullName": "Budi Santoso",
      "enabled": true,
      "roles": ["USER"],
      "createdAt": "2026-10-19T08:00:00Z"
    }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 1,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

8.2.6 Role assignment and status changes

```
PUT /api/v1/users/{id}/roles
{ "roles": ["USER", "ADMIN"] }

PATCH /api/v1/users/{id}/status
{ "enabled": false }

PUT /api/v1/auth/users/{id}/password
{ "newPassword": "ResetPass1" }
```

- Role names are fixed. Only ADMIN and USER exist in release 1. Unknown roles return 404 ROLE_NOT_FOUND.
- The last active ADMIN cannot be deactivated or have ADMIN removed. The check runs under pg_advisory_xact_lock and returns 409 LAST_ADMIN.
- An ADMIN cannot deactivate their own account. The service returns 409 SELF_DEACTIVATION.
- Deactivation and role changes revoke the user's refresh-token families through an internal auth-service call. Existing access tokens remain valid for at most 15 minutes.
- Admin password reset is a auth-service endpoint; it revokes all refresh-token families for the user. The user must log in again on every device.

8.2.7 Self-service profile and password

```
GET /api/v1/users/me
Authorization: Bearer <token>

{
  "id": "2b2e...",
  "username": "budi.santoso",
  "email": "budi.santoso@example.com",
  "fullName": "Budi Santoso",
  "enabled": true,
  "roles": ["USER"],
  "createdAt": "2026-10-19T08:00:00Z"
}

PUT /api/v1/auth/me/password
{
  "currentPassword": "InitialPass1",
  "newPassword": "NewPass1"
}

HTTP/1.1 204 No Content
```

The current password must match. The new password follows the same policy and must differ from the current password. All refresh-token families are revoked after a successful change; the user must log in again on every device. The current access token is discarded by the frontend and expires within 15 minutes.

8.2.8 User and role errors

Table 43 - User and role error cases

Condition	HTTP	Code
Duplicate username	409	USERNAME_TAKEN
Duplicate email	409	EMAIL_TAKEN
Unknown role	404	ROLE_NOT_FOUND
User not found	404	USER_NOT_FOUND
Last active admin protection	409	LAST_ADMIN
Self-deactivation	409	SELF_DEACTIVATION
Current password mismatch	400	VALIDATION_ERROR
Admin sets password for unknown user	404	USER_NOT_FOUND
Non-admin calls admin endpoint	403	FORBIDDEN

8.3 Project and Membership Management

8.3.1 Purpose and actors

Projects are the access boundary. ADMIN creates projects and assigns members. A USER can see a project only when they are a member. Tasks and timelines are always addressed through a project, which makes the access rule consistent across the two downstream services.

- Actors: ADMIN for create/update/archive/member changes; ADMIN or project member for reads.
- Preconditions: members are active users; project code is unique.
- Postconditions: membership changes take effect on the next access check; no cache is used in v1.

8.3.2 Project administration sequence

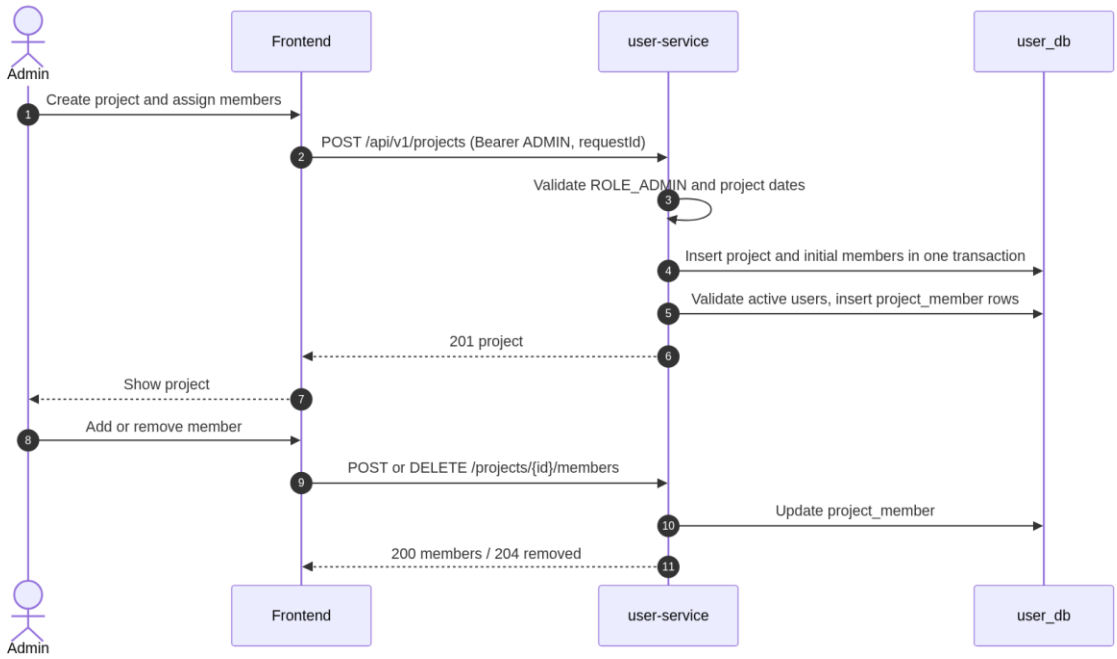


Figure 12 - Project and membership administration sequence

8.3.3 Project and membership endpoints

Table 44 - Project and membership API endpoints

#	Method	Path	Role	Purpose
P1	GET	/api/v1/projects	ADMIN / USER	Paginated projects. Admin sees all; user sees member projects.
P2	POST	/api/v1/projects	ADMIN	Create a project and optionally add initial members.
P3	GET	/api/v1/projects/{id}	ADMIN / member	Project detail.
P4	PUT	/api/v1/projects/{id}	ADMIN	Update name, description, and dates. Code is immutable.
P5	PATCH	/api/v1/projects/{id}/status	ADMIN	Archive or restore a project.
P6	GET	/api/v1/projects/{id}/members	ADMIN / member	List project members.
P7	POST	/api/v1/projects/{id}/members	ADMIN	Add one or more active users.
P8	DELETE	/api/v1/projects/{id}/members/{userId}	ADMIN	Remove a project member.

8.3.4 Project create request

Table 45 - Project create request fields

Field	Type	Required	Validation
code	string	Yes	2-30 chars, uppercase, unique, pattern [A-Z0-9-]. Immutable after create.
requestId	uuid	Yes	Client-generated idempotency key. Unique; a replay returns the original project.
name	string	Yes	1-150 chars.
description	string	No	Max 2,000 chars.
startDate	date	Yes	ISO date.
endDate	date	Yes	ISO date, on or after startDate.
memberIds	uuid[]	No	Active users. Duplicates are ignored. Maximum 200 per request.

```
POST /api/v1/projects
Authorization: Bearer <ADMIN token>
Content-Type: application/json

{
  "code": "PRJ-001",
  "name": "Website Revamp",
  "description": "Corporate website redesign and content migration.",
  "startDate": "2026-11-02",
  "endDate": "2027-03-31",
  "requestId": "6a1f5e0e-6f2a-4a34-9c5d-2b1f0c9a7e11",
  "memberIds": [
    "2b2e...",
    "7c1a..."
  ]
}

HTTP/1.1 201 Created
Location: /api/v1/projects/8f4c...

{
  "id": "8f4c...",
  "code": "PRJ-001",
  "name": "Website Revamp",
  "description": "Corporate website redesign and content migration.",
  "startDate": "2026-11-02",
  "endDate": "2027-03-31",
  "status": "ACTIVE",
  "memberCount": 2,
  "createdBy": "0f4e...",
  "createdAt": "2026-10-19T08:00:00Z",
  "updatedAt": "2026-10-19T08:00:00Z"
}
```

8.3.5 Project list and detail

GET /api/v1/projects is role-sensitive. ADMIN receives all projects. A USER receives only projects where a project_member row exists. The query supports q (code or name), status (ACTIVE, ARCHIVED), page, size, and sort. Allowed sort fields are code, name, startDate, endDate, status, createdAt.

```
GET /api/v1/projects?status=ACTIVE&q=website&page=0&size=20&sort=code,asc
Authorization: Bearer <USER token>

HTTP/1.1 200 OK

{
  "content": [
    {
      "id": "8f4c...",
      "code": "PRJ-001",
      "name": "Website Revamp",
```

```
      "status": "ACTIVE",
      "startDate": "2026-11-02",
      "endDate": "2027-03-31",
      "memberCount": 2
    }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 1,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

GET /api/v1/projects/{id} returns the same detail as create plus memberCount. A USER who is not a member receives 403 FORBIDDEN; an unknown project returns 404 PROJECT_NOT_FOUND.

8.3.6 Update, archive, and restore

```
PUT /api/v1/projects/{id}
{
  "name": "Website Revamp Phase 2",
  "description": "Updated scope.",
  "startDate": "2026-11-09",
  "endDate": "2027-04-15"
}

PATCH /api/v1/projects/{id}/status
{ "status": "ARCHIVED" }

PATCH /api/v1/projects/{id}/status
{ "status": "ACTIVE" }
```

- Project code is immutable after creation because it is a stable human identifier.
- Project create is idempotent by requestId. A replay returns the original project with 200 OK and Idempotent-Replay: true.
- Changing project dates does not automatically change task or timeline dates. Each update sets updated_by.
- Archiving sets status=ARCHIVED, archived_at, and archived_by. Reads remain available; task and timeline writes return 409 PROJECT_ARCHIVED.
- Restoring clears archived_at, sets status=ACTIVE, and records the acting admin in updated_by.
- There is no project delete endpoint in release 1.

8.3.7 Membership

```
GET /api/v1/projects/{id}/members

HTTP/1.1 200 OK
{
  "content": [
    {
      "userId": "2b2e...",
      "username": "budi.santoso",
      "fullName": "Budi Santoso",
      "email": "budi.santoso@example.com",
      "addedAt": "2026-10-19T08:05:00Z"
    }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 1,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

```
POST /api/v1/projects/{id}/members
{ "userIds": ["2b2e...", "7c1a..."] }

HTTP/1.1 200 OK
{ "added": 2, "alreadyMembers": 0 }

DELETE /api/v1/projects/{id}/members/{userId}
HTTP/1.1 204 No Content
```

- Adding a member is idempotent: existing members are reported in alreadyMembers instead of failing the whole request.
- Only active users can be added. An inactive user returns 404 USER_NOT_FOUND.
- Removing a member takes effect immediately for access checks. Their access token remains valid, but project-scoped calls return 403.
- If the removed user is the PIC of a task, the task remains readable with the stored PIC name snapshot. Editing that task requires selecting a current member as PIC.
- Members can read the member list of their own project; only ADMIN can change it.

8.3.8 Project errors

Table 46 - Project and membership error cases

Condition	HTTP	Code
Duplicate project code	409	PROJECT_CODE_TAKEN
Project not found	404	PROJECT_NOT_FOUND
Non-member reads project	403	FORBIDDEN
Archived project write via task/timeline	409	PROJECT_ARCHIVED
Adding an existing member	200	Not an error; reported in alreadyMembers
Member not found on removal	404	MEMBER_NOT_FOUND
Inactive user added as member	404	USER_NOT_FOUND

8.4 Task Management

8.4.1 Purpose and actors

Tasks are the detailed work items inside a project. Each task has a title, optional description, inclusive start and end dates, an automatically calculated calendar-day duration, a PIC, a status, and a completion percentage. A project member or ADMIN can manage tasks in an ACTIVE project.

8.4.2 Task create/update sequence

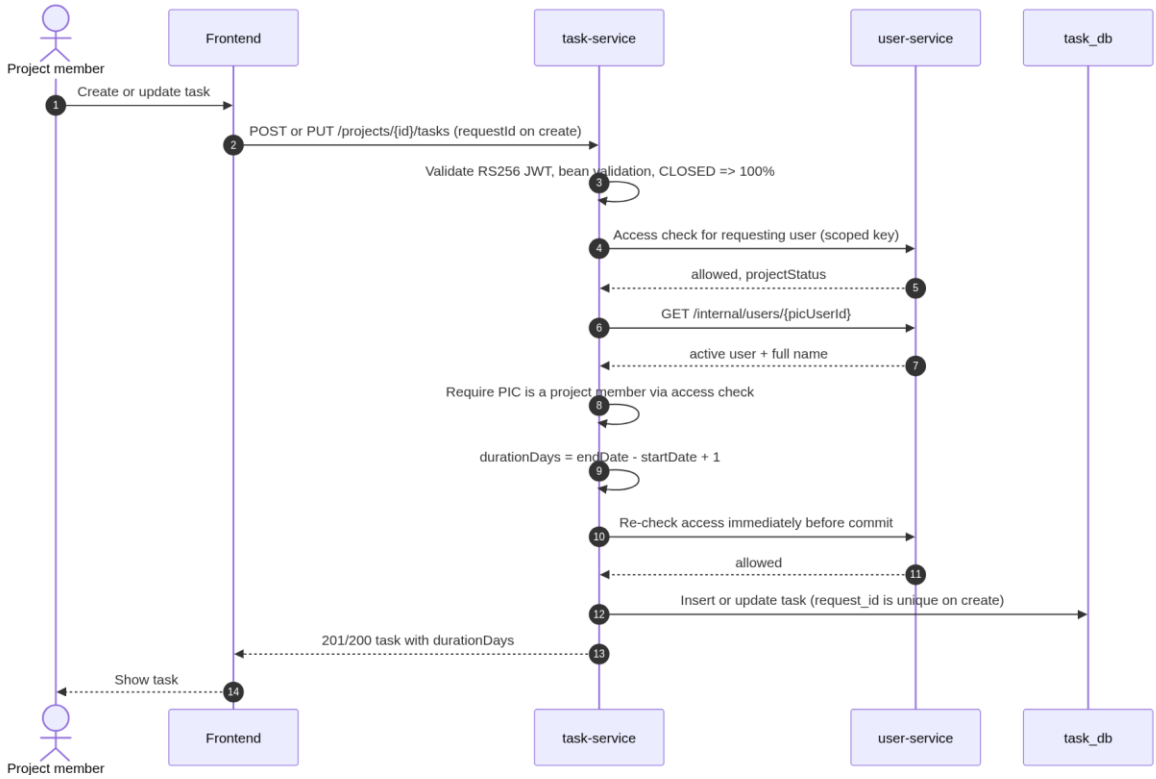


Figure 13 - Task create and update sequence

8.4.3 Task endpoints

Table 47 - Task API endpoints

#	Method	Path	Role	Purpose
T1	GET	/api/v1/projects/{projectId}/tasks	ADMIN / member	Paginated, filtered task list.
T2	POST	/api/v1/projects/{projectId}/tasks	ADMIN / member	Create a task.
T3	GET	/api/v1/projects/{projectId}/tasks/{taskId}	ADMIN / member	Task detail.
T4	PUT	/api/v1/projects/{projectId}/tasks/{taskId}	ADMIN / member	Full task update.
T5	PATCH	/api/v1/projects/{projectId}/tasks/{taskId}/status	ADMIN / member	Change operational status.
T6	PATCH	/api/v1/projects/{projectId}/tasks/{taskId}/progress	ADMIN / member	Change completion percentage.
T7	DELETE	/api/v1/projects/{projectId}/tasks/{taskId}	ADMIN / member	Hard-delete a task.

8.4.4 Task create request

Table 48 - Task create/update request fields

Field	Type	Required	Validation
title	string	Yes	1-200 chars after trim.
requestId	uuid	Yes	Client-generated idempotency key. Unique; a replay returns the original task.

Field	Type	Required	Validation
description	string	No	Max 2,000 chars.
startDate	date	Yes	ISO date.
endDate	date	Yes	ISO date, on or after startDate; range max 10 years.
picUserId	uuid	Yes	Active user and member of the project.
status	enum	No	OPEN, IN_PROGRESS, CLOSED. Default OPEN.
percentComplete	integer	No	0-100. Default 0.

```
POST /api/v1/projects/8f4c.../tasks
Authorization: Bearer <member token>
Content-Type: application/json

{
  "title": "Design database schema",
  "description": "Logical and physical model for the project module.",
  "startDate": "2026-11-02",
  "endDate": "2026-11-06",
  "picUserId": "2b2e...",
  "status": "OPEN",
  "percentComplete": 0,
  "requestId": "b3c9d2f4-8e17-4f60-9a2b-7d5c1e8f0a42"
}

HTTP/1.1 201 Created
Location: /api/v1/projects/8f4c.../tasks/9a7d...

{
  "id": "9a7d...",
  "projectId": "8f4c...",
  "title": "Design database schema",
  "description": "Logical and physical model for the project module.",
  "startDate": "2026-11-02",
  "endDate": "2026-11-06",
  "durationDays": 5,
  "pic": {
    "userId": "2b2e...",
    "name": "Budi Santoso"
  },
  "status": "OPEN",
  "percentComplete": 0,
  "createdBy": "0f4e...",
  "updatedBy": "0f4e...",
  "createdAt": "2026-10-19T08:10:00Z",
  "updatedAt": "2026-10-19T08:10:00Z"
}
```

8.4.5 Task query sequence

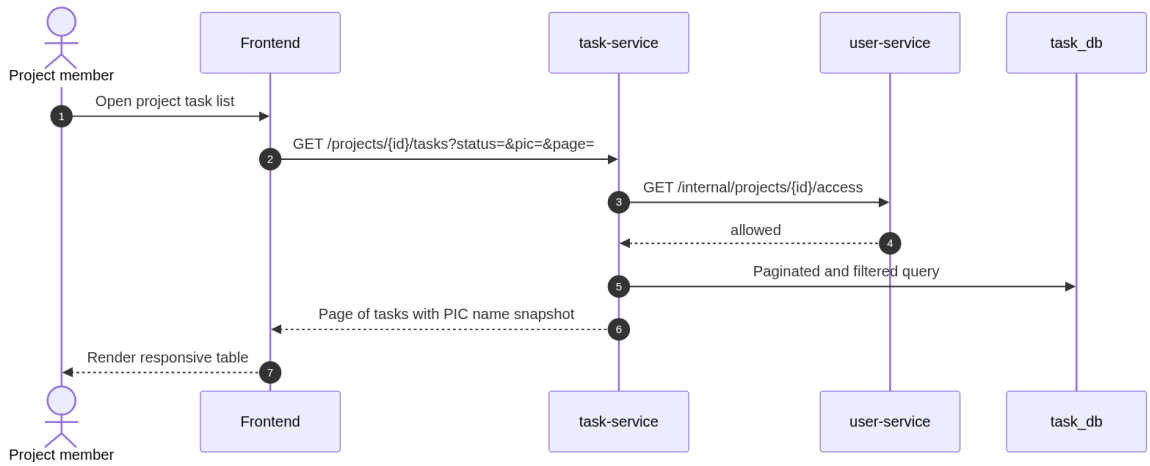


Figure 14 - Task list query sequence

GET /api/v1/projects/{projectId}/tasks returns a page envelope. Filters: status, picUserId, q, page, size, sort. q searches title and description case-insensitively. Allowed sort fields: title, startDate, endDate, durationDays, status, percentComplete, createdAt, updatedAt.

```
GET /api/v1/projects/8f4c.../tasks?status=IN_PROGRESS&picUserId=2b2e...&page=0&size=20&sort=endDate,asc
Authorization: Bearer <member token>

HTTP/1.1 200 OK

{
  "content": [
    {
      "id": "9a7d...",
      "projectId": "8f4c...",
      "title": "Design database schema",
      "startDate": "2026-11-02",
      "endDate": "2026-11-06",
      "durationDays": 5,
      "pic": { "userId": "2b2e...", "name": "Budi Santoso" },
      "status": "IN_PROGRESS",
      "percentComplete": 40
    }
  ],
  "page": 0,
  "size": 20,
  "totalElements": 1,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

8.4.6 Status, progress, and delete

```
PATCH /api/v1/projects/{projectId}/tasks/{taskId}/status
{ "status": "CLOSED" }

PATCH /api/v1/projects/{projectId}/tasks/{taskId}/progress
{ "percentComplete": 75 }

DELETE /api/v1/projects/{projectId}/tasks/{taskId}
HTTP/1.1 204 No Content
```

- Changing status to CLOSED sets percentComplete to 100 in the same transaction.
- A progress edit on a CLOSED task returns 409 TASK_CLOSED; reopen the task before changing progress.
- A PUT that sets CLOSED with percentComplete below 100 returns 400 INVALID_COMPLETION_FOR_CLOSED. The database CHECK constraint is the final guard.
- Task create is idempotent by requestId. A replay returns the original task with 200 OK and Idempotent-Replay: true.
- Changing progress to 100 does not automatically change status. The user can close the task separately.
- Allowed status transitions are OPEN -> IN_PROGRESS, OPEN -> CLOSED, IN_PROGRESS -> CLOSED, and CLOSED -> IN_PROGRESS. Other transitions return 409 INVALID_STATUS_TRANSITION; the API contract is the table in Section 6.3.3.
- PUT is a full update and must include title, description, startDate, endDate, picUserId, status, and percentComplete. Missing fields are validation errors.
- Delete is hard delete. The task is removed; there is no restore.
- Task duration is calculated and returned; clients must not send durationDays. Sending a read-only field is ignored or rejected with 400 depending on the DTO configuration; the documented contract is that it is read-only.

8.4.7 Task access and PIC rules

- The requesting user must be ADMIN or a member of the project. The access check is performed on every request.
- The PIC must be an active user and a member of the same project. The service checks access with member=true and loads the user's full name for the snapshot.
- An ADMIN who is not a project member can manage tasks but cannot be selected as PIC until added as a member.
- Reading a task requires project access; there is no per-task assignee restriction. All project members can see all tasks.
- Archived project: reads allowed, writes return 409 PROJECT_ARCHIVED.

8.4.8 Task errors

Table 49 - Task error cases

Condition	HTTP	Code
Task not found in project	404	TASK_NOT_FOUND
Project not found	404	PROJECT_NOT_FOUND
Non-member task access	403	FORBIDDEN
PIC not an active project member	400	PIC_NOT_MEMBER
endDate before startDate	400	INVALID_DATE_RANGE
Title/description/percent invalid	400	VALIDATION_ERROR
Archived project write	409	PROJECT_ARCHIVED
Invalid status transition	409	INVALID_STATUS_TRANSITION
Progress edit on a CLOSED task	409	TASK_CLOSED
PUT sets CLOSED with percent below 100	400	INVALID_COMPLETION_FOR_CLOSED
user-service unavailable during access/PIC check	503	INTERNAL_DEPENDENCY_UNAVAILABLE

8.5 Timeline Management

8.5.1 Purpose and actors

A project has one timeline that represents the approved project schedule. The timeline is versioned. A new timeline starts as a draft, is published, and can later be revised through a new draft. The current published version remains active while a revision is being prepared. A draft contains planned dates and notes only. Publishing a revision promotes the draft, carries forward the current operational status and actual completion, snapshots and supersedes the previous current version, and records an event.

- Actors: ADMIN or a member of the project.
- Preconditions: the project exists and is ACTIVE for writes; the planned dates lie inside the holiday coverage horizon.
- Postconditions: timeline versions are ordered, auditable, and never physically deleted after publication; lifecycle events are recorded in timeline_version_event.
- Calculated fields: durationWorkdays and percentPlan are derived on every read for the current version and frozen in snapshots for superseded/cancelled versions.

8.5.2 Initial draft and publish sequence

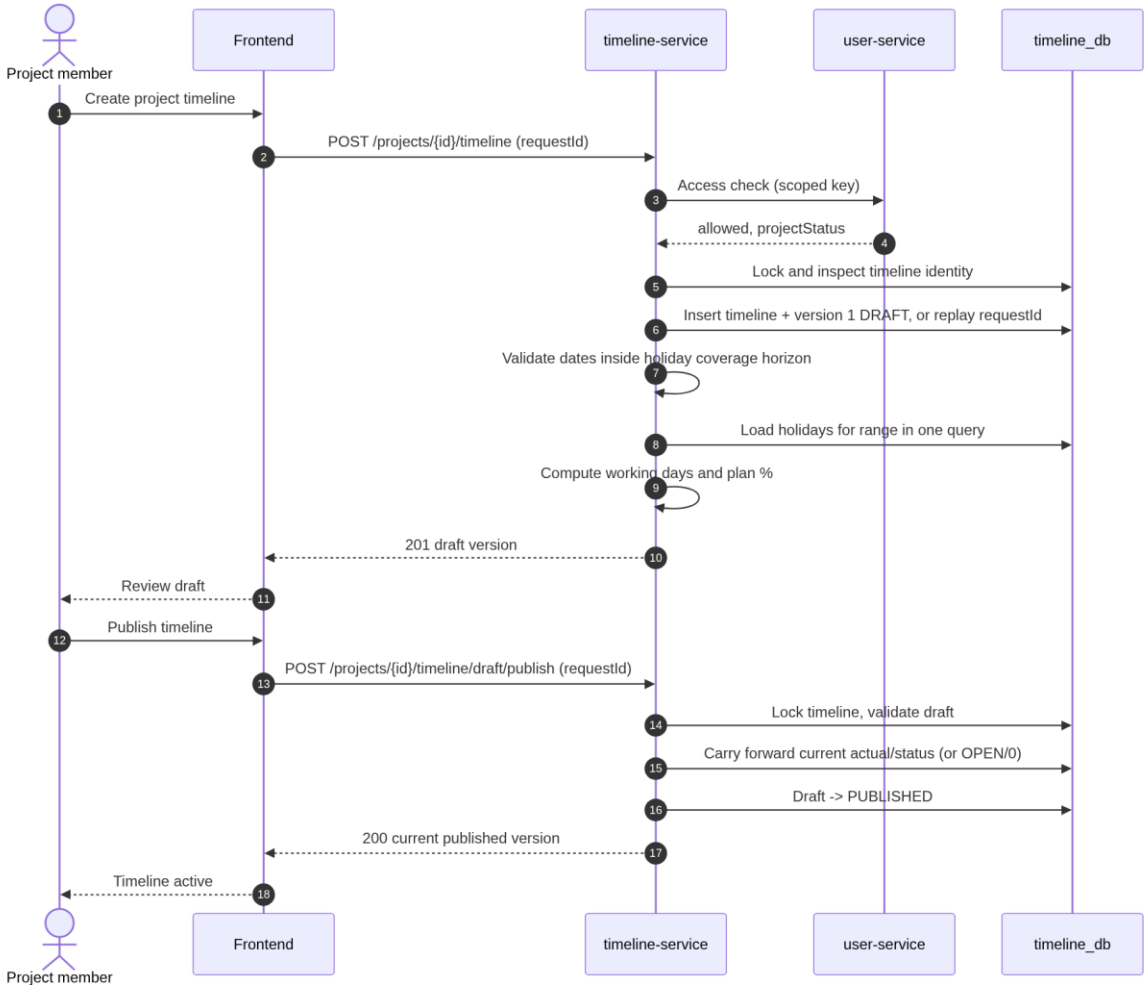


Figure 15 - Initial timeline draft and publish sequence

8.5.3 Revision and publish sequence

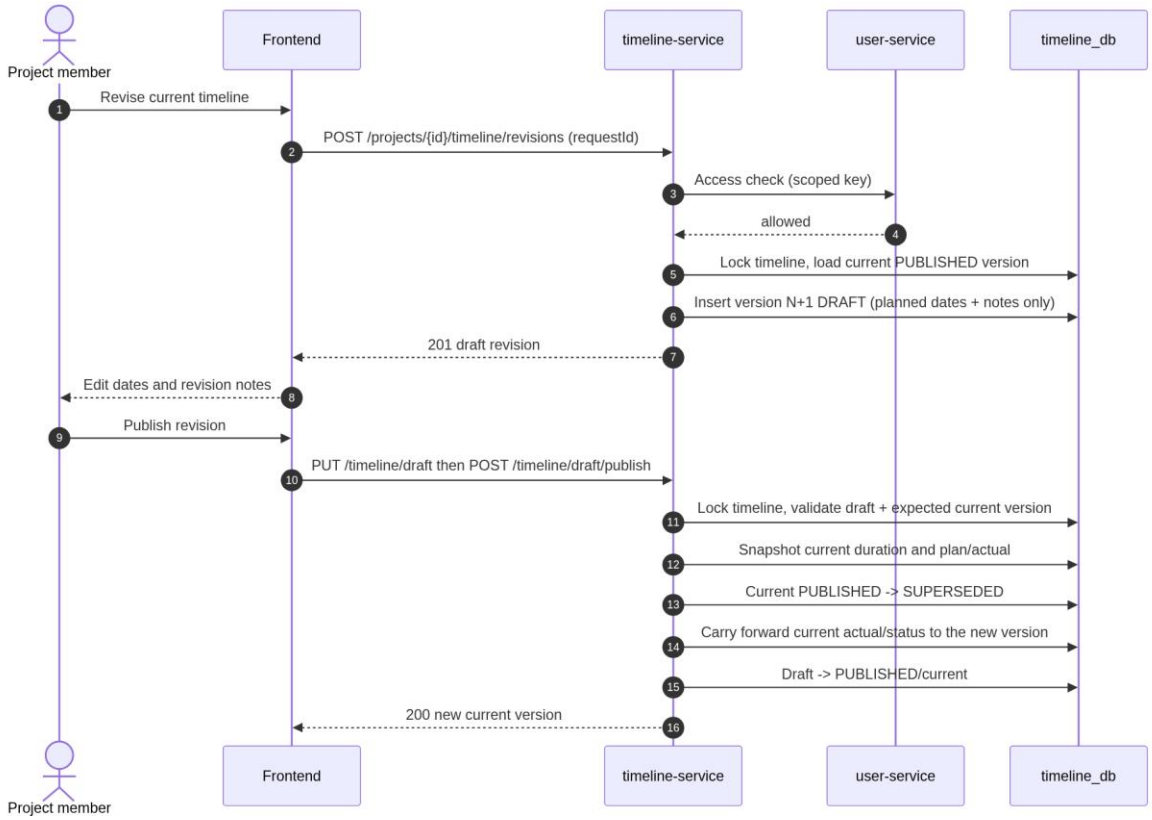


Figure 16 - Timeline revision draft and publish sequence

8.5.4 Timeline endpoints

Table 50 - Timeline API endpoints

#	Method	Path	Role	Purpose
TL1	GET	/api/v1/projects/{projectId}/timeline	ADMIN / member	Aggregate: current version, draft, version count.
TL2	GET	/api/v1/projects/{projectId}/timeline/current	ADMIN / member	Current published version. Optional asOf for plan preview.
TL3	GET	/api/v1/projects/{projectId}/timeline/versions	ADMIN / member	Paginated version history, newest first, with snapshots for past versions.
TL4	GET	/api/v1/projects/{projectId}/timeline/versions/{versionId}	ADMIN / member	One version with snapshot semantics for past versions.
TL5	POST	/api/v1/projects/{projectId}/timeline	ADMIN / member	Create the timeline and version 1 DRAFT; requestId is idempotent.
TL6	PUT	/api/v1/projects/{projectId}/timeline/draft	ADMIN / member	Edit the active draft's planned dates and notes; optional expectedDraftUpdatedAt.
TL7	DELETE	/api/v1/projects/{projectId}/timeline/draft	ADMIN / member	Cancel the active draft; removes the identity if nothing was ever published.
TL8	POST	/api/v1/projects/{projectId}/timeline/draft/publish	ADMIN / member	Publish the active draft; requestId, optional expected draft timestamp, required expected current timestamp when a current version exists.
TL9	POST	/api/v1/projects/{projectId}/timeline/revisions	ADMIN / member	Create a draft from the current planned dates; requestId is idempotent.
TL10	PATCH	/api/v1/projects/{projectId}/timeline/current/status	ADMIN / member	Update current operational status; CLOSED sets actual to 100.
TL11	PATCH	/api/v1/projects/{projectId}/timeline/current/progress	ADMIN / member	Update current actual completion; rejected while CLOSED.

TL2 accepts an optional asOf query parameter (ISO date). If it is omitted, the server uses today in Asia/Jakarta. If it is supplied, it is used for the calculation and echoed as planAsOfDate. The before-start (0.00) and on/after-end (100.00) caps from Section 6.6 still apply.

8.5.5 Timeline version response

Table 51 - Timeline version response fields

Field	Type	Description
id	uuid	Version ID.
timelineId	uuid	Parent timeline ID.
projectId	uuid	Owning project.
versionNumber	integer	1, 2, 3, ... per project.
lifecycleStatus	enum	DRAFT, PUBLISHED, SUPERSEDED, CANCELLED.
startDate / endDate	date	Inclusive planned dates.
durationWorkdays	integer	Working days excluding weekends and holidays.
operationalStatus	enum	OPEN, IN_PROGRESS, CLOSED. DRAFT and CANCELLED responses report the stored default OPEN; the current version reports the live value.
percentActual	integer	0-100. DRAFT and CANCELLED responses report the stored default 0; the current version reports the live value.
percentPlan	decimal	0.00-100.00, rounded half-up to 2 decimals.
planAsOfDate	date	Today in Asia/Jakarta for the current version; frozen snapshot_as_of_date for superseded/cancelled versions.
planWarning	string or null	NO_WORKING_DAYS when the range has no working days; otherwise null.
revisionNotes	string or null	Max 1,000 chars.
createdBy / createdAt	uuid / timestamp	Draft creation audit.
updatedBy / updatedAt	uuid / timestamp	Last edit or operational update; used for optimistic checks.
publishedBy / publishedAt	uuid / timestamp or null	Publish audit.
supersededBy / supersededAt	uuid / timestamp or null	Actor and time when a newer version was published.
snapshotDurationWorkdays	integer or null	Frozen working-day duration for a superseded/cancelled version.
snapshotPercentPlan	decimal or null	Frozen plan % for a superseded/cancelled version.
snapshotPercentActual	integer or null	Frozen actual % for a superseded/cancelled version.
snapshotAsOfDate	date or null	As-of date used for the frozen plan %.
cancelledBy / cancelledAt	uuid / timestamp or null	Actor and time when a draft was cancelled.


```
{
  "id": "c1a2...",
  "timelineId": "e9b1...",
  "projectId": "8f4c...",
  "versionNumber": 2,
  "lifecycleStatus": "PUBLISHED",
  "startDate": "2026-11-02",
  "endDate": "2027-04-15",
  "durationWorkdays": 106,
  "operationalStatus": "IN_PROGRESS",
  "percentActual": 35,
  "percentPlan": 30.19,
  "planAsOfDate": "2026-12-15",
  "planWarning": null,
  "revisionNotes": "Extended UAT and content migration by two weeks.",
  "createdBy": "0f4e...",
  "createdAt": "2026-11-20T03:00:00Z",
  "updatedBy": "2b2e...",
  "updatedAt": "2026-12-14T09:30:00Z",
  "publishedBy": "0f4e...",
  "publishedAt": "2026-11-21T02:00:00Z",
  "supersededBy": null,
  "supersededAt": null,
  "snapshotDurationWorkdays": null,
  "snapshotPercentPlan": null,
```

```
  "snapshotPercentActual": null,
  "snapshotAsOfDate": null,
  "cancelledBy": null,
  "cancelledAt": null
}
```

8.5.6 GET /api/v1/projects/{projectId}/timeline

The aggregate read is the primary timeline screen. It returns the current published version, the active draft if one exists, and the number of versions. If the project has no timeline, it returns 404 TIMELINE_NOT_FOUND. A non-member receives 403 FORBIDDEN.

```
{
  "timelineId": "e9b1...",
  "projectId": "8f4c...",
  "versionCount": 2,
  "current": { "...version object..." },
  "draft": { "...version object or null..." }
}
```

8.5.7 Create initial timeline

Table 52 - Initial timeline create fields

Field	Type	Required	Validation
startDate	date	Yes	ISO date inside the holiday coverage horizon.
endDate	date	Yes	On or after startDate; inside the holiday coverage horizon; at least one working day.
requestId	uuid	Yes	Client-generated idempotency key. Unique; a replay returns the original timeline/draft.
revisionNotes	string	No	Max 1,000 chars. For the initial version this is a baseline note.

```
POST /api/v1/projects/8f4c.../timeline
Authorization: Bearer <member token>
Content-Type: application/json

{
  "startDate": "2026-11-02",
  "endDate": "2027-03-31",
  "requestId": "1d7c6a42-5b9e-4f0a-8c31-6e2d9b4a7f10",
  "revisionNotes": "Initial approved baseline."
}

HTTP/1.1 201 Created

{
  "id": "c1a2...",
  "timelineId": "e9b1...",
  "projectId": "8f4c...",
  "versionNumber": 1,
  "lifecycleStatus": "DRAFT",
  "startDate": "2026-11-02",
  "endDate": "2027-03-31",
  "durationWorkdays": 95,
  "operationalStatus": "OPEN",
  "percentActual": 0,
  "percentPlan": 0.00,
  "planAsOfDate": "2026-10-19",
  "planWarning": null,
  "revisionNotes": "Initial approved baseline.",
  "createdBy": "0f4e...",
  "createdAt": "2026-10-19T08:20:00Z",
  "publishedBy": null,
  "publishedAt": null,
  "supersededAt": null,
```

```
  "cancelledAt": null
}
```

- Creating a timeline when one already exists returns 409 TIMELINE_EXISTS. A cancelled initial draft removes the identity, so the project can create a new timeline later.
- The initial version is DRAFT. It is not current until published; publish carries forward OPEN and 0 because there is no current version.
- The range must lie inside the holiday coverage horizon; otherwise the request returns 409 HOLIDAY_COVERAGE_REQUIRED.
- durationWorkdays uses the current holiday table. If the range has no working day, the request returns 400 NO_WORKING_DAYS.
- The project must be ACTIVE. An archived project returns 409 PROJECT_ARCHIVED.
- The requestId is unique. A replay returns the original timeline/draft with 200 OK and Idempotent-Replay: true.

8.5.8 Edit and cancel the draft

```
PUT /api/v1/projects/{projectId}/timeline/draft
{
  "startDate": "2026-11-09",
  "endDate": "2027-04-15",
  "revisionNotes": "Adjusted kick-off by one week.",
  "expectedDraftUpdatedAt": "2026-10-19T08:20:00Z"
}

DELETE /api/v1/projects/{projectId}/timeline/draft
HTTP/1.1 204 No Content
```

- Only a DRAFT can be edited. Editing a published version returns 409 INVALID_TIMELINE_STATE.
- The draft update replaces planned dates and revision notes only. Operational status and actual completion are not accepted as draft fields; they are carried forward at publish.
- If expectedDraftUpdatedAt is supplied and the draft changed since it was read, the endpoint returns 409 DRAFT_MODIFIED.
- Dates must stay inside the holiday coverage horizon and contain at least one working day.
- Cancelling a draft sets lifecycle_status=CANCELLED and cancelled_at. The current published version is unaffected. A new revision can be created afterwards.
- If the cancelled draft is the only version and no version has ever been published, the timeline identity is deleted in the same transaction. This is the recovery path for an initial timeline created by mistake.
- There is no hard delete for a timeline that has a published version.
- If the project is archived, edit/cancel/publish writes return 409 PROJECT_ARCHIVED.

8.5.9 Publish the draft

```
POST /api/v1/projects/{projectId}/timeline/draft/publish
Authorization: Bearer <member token>
Content-Type: application/json

{
  "requestId": "7e2b9a10-3c4d-4e5f-8a6b-1c2d3e4f5a60",
  "expectedDraftUpdatedAt": "2026-10-19T08:20:00Z",
  "expectedCurrentUpdatedAt": null
}

HTTP/1.1 200 OK

{
  "id": "c1a2...",
  "versionNumber": 1,
```

```
  "lifecycleStatus": "PUBLISHED",
  "startDate": "2026-11-02",
  "endDate": "2027-03-31",
  "durationWorkdays": 95,
  "operationalStatus": "OPEN",
  "percentActual": 0,
  "percentPlan": 0.00,
  "planAsOfDate": "2026-10-19",
  "publishedBy": "0f4e...",
  "publishedAt": "2026-10-19T08:25:00Z"
}
```

- Publishing runs in one transaction and locks the timeline row. Status and progress updates (TL10/TL11) take the same lock, so a concurrent progress update either commits before publish reads the current values (and is carried forward) or waits until publish commits.
- If there is no current version, the draft becomes PUBLISHED with operational status OPEN and actual completion 0 carried forward.
- If a current version exists, its operational status and actual completion are carried forward to the new version at publish time; the old current version is snapshotted and becomes SUPERSEDED. A draft does not carry operational values, so it cannot overwrite current progress.
- When a current version exists, expectedCurrentUpdatedAt is required and must match the current version's updatedAt; a missing, null, or mismatched value returns 409 CURRENT_VERSION_CHANGED. expectedDraftUpdatedAt is optional and a mismatch returns 409 DRAFT_MODIFIED.
- The partial unique indexes guarantee that two concurrent publishes cannot leave two current versions.
- If there is no draft, the endpoint returns 404 DRAFT_NOT_FOUND.
- A publish retry is detected by publish_request_id before any draft-exists or timestamp checks: when a version already has this publish_request_id, the endpoint returns that version with 200 and Idempotent-Replay: true.
- Publishing applies the draft's planned dates and revision notes.
- Publish does not re-reject a draft whose range became zero-working-day after a holiday edit; reads return planWarning=NO_WORKING_DAYS so the operator can correct the dates in a new revision.

8.5.10 Create a revision

Table 53 - Timeline revision request fields

Field	Type	Required	Validation
startDate	date	No	If omitted, copy current planned start. If supplied, on or before endDate, inside the horizon, at least one working day.
endDate	date	No	If omitted, copy current planned end. If supplied, on or after startDate and inside the horizon.
requestId	uuid	Yes	Client-generated idempotency key. Unique; a replay returns the original draft.
revisionNotes	string	No	Max 1,000 chars. Recommended.

```
POST /api/v1/projects/8f4c.../timeline/revisions
{
  "startDate": "2026-11-02",
  "endDate": "2027-04-15",
  "requestId": "9c4e2f18-7a6b-4d3c-8e5f-0a1b2c3d4e5f",
  "revisionNotes": "Extended UAT by two weeks."
}

HTTP/1.1 201 Created

{
  "id": "d2b3...",
  "versionNumber": 2,
  "lifecycleStatus": "DRAFT",
```

```
  "startDate": "2026-11-02",
  "endDate": "2027-04-15",
  "operationalStatus": "OPEN",
  "percentActual": 0,
  "revisionNotes": "Extended UAT by two weeks.",
  "publishedAt": null
}
```

- A revision copies the current published version's planned dates and notes, then applies supplied date overrides. Operational status and actual completion are not copied into the draft: the DRAFT row stores OPEN/0 and the draft response reports those stored defaults. The current operational values remain on the current version and are carried forward when the draft is published.
- The current version remains active and can still receive status/progress updates while the draft exists. Those updates are carried forward when the draft is published.
- Only one draft may exist. A second revision request returns 409 DRAFT_EXISTS.
- A revision requires a current published version. If none exists, return 409 NO_CURRENT_VERSION.
- When the draft is published, it becomes current and the previous current becomes SUPERSEDED with a snapshot.
- The requestId is unique. A replay returns the original draft with 200 OK and Idempotent-Replay: true.

8.5.11 Version history

GET /api/v1/projects/{projectId}/timeline/versions returns a page envelope, newest first. It supports page, size (default 50, maximum 100), and sort=versionNumber,desc. There is no 200-version cap. Each item includes lifecycle status, dates, durationWorkdays, percentPlan, planAsOfDate, planWarning, and the audit fields. For SUPERSEDED and CANCELLED versions, duration and percentages come from the stored snapshot, so later holiday corrections do not rewrite history. Current and draft values are calculated live.

```
GET /api/v1/projects/{projectId}/timeline/versions
Authorization: Bearer <member token>

HTTP/1.1 200 OK
{
  "content": [
    {
      "id": "d2b3...",
      "versionNumber": 2,
      "lifecycleStatus": "DRAFT",
      "startDate": "2026-11-02",
      "endDate": "2027-04-15",
      "durationWorkdays": 106,
      "percentPlan": 0.00,
      "planAsOfDate": "2026-10-19"
    },
    {
      "id": "c1a2...",
      "versionNumber": 1,
      "lifecycleStatus": "PUBLISHED",
      "startDate": "2026-11-02",
      "endDate": "2027-03-31",
      "durationWorkdays": 95,
      "percentPlan": 0.00,
      "planAsOfDate": "2026-10-19"
    }
  ],
  "page": 0,
  "size": 50,
  "totalElements": 2,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

8.5.12 Current status and actual progress

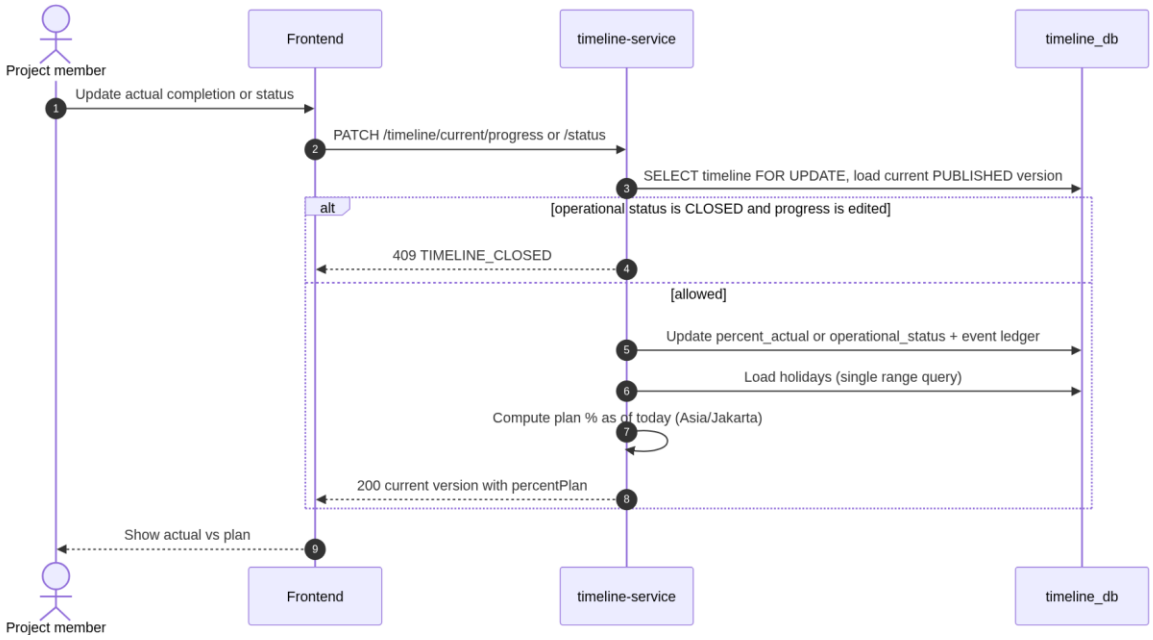


Figure 17 - Current timeline status and progress update sequence

```
PATCH /api/v1/projects/{projectId}/timeline/current/status
{ "status": "CLOSED" }

PATCH /api/v1/projects/{projectId}/timeline/current/progress
{ "percentActual": 60 }
```

- Only the current PUBLISHED version can be updated through these endpoints.
- Changing status to CLOSED sets percentActual to 100 in the same transaction.
- Changing progress does not automatically change status. The user can close the timeline separately. A progress edit while CLOSED returns 409 TIMELINE_CLOSED.
- Allowed status transitions are the same as tasks: OPEN -> IN_PROGRESS, OPEN -> CLOSED, IN_PROGRESS -> CLOSED, CLOSED -> IN_PROGRESS.
- percentPlan is recalculated on every response. It is not affected by operational status.
- Every status and progress change writes a STATUS_CHANGED or PROGRESS_CHANGED row to timeline_version_event with the actor and the previous/new value.
- If there is no current version, return 409 NO_CURRENT_VERSION.

8.5.13 Timeline errors

Table 54 - Timeline error cases

Condition	HTTP	Code
Project already has a timeline	409	TIMELINE_EXISTS
Project has no timeline	404	TIMELINE_NOT_FOUND
No draft exists	404	DRAFT_NOT_FOUND
A draft already exists	409	DRAFT_EXISTS
Revision without current version	409	NO_CURRENT_VERSION
Current version changed after revision started	409	CURRENT_VERSION_CHANGED
Draft changed since it was loaded	409	DRAFT_MODIFIED
Timeline dates outside holiday coverage	409	HOLIDAY_COVERAGE_REQUIRED
Progress edit on a CLOSED timeline	409	TIMELINE_CLOSED
Status CLOSED submitted with actual below 100	400	INVALID_COMPLETION_FOR_CLOSED

Condition	HTTP	Code
Edit/publish a non-draft	409	INVALID_TIMELINE_STATE
Invalid operational status transition	409	INVALID_STATUS_TRANSITION
Date range invalid	400	INVALID_DATE_RANGE
Range contains no working day	400	NO_WORKING_DAYS
Archived project write	409	PROJECT_ARCHIVED
Non-member access	403	FORBIDDEN
user-service unavailable	503	INTERNAL_DEPENDENCY_UNAVAILABLE

8.6 Holiday Management

8.6.1 Purpose and actors

Holidays are shared calendar data owned by timeline-service. ADMIN can add, update, and delete holidays. Every authenticated user can read the calendar so the frontend can explain why a timeline duration or plan percentage changed.

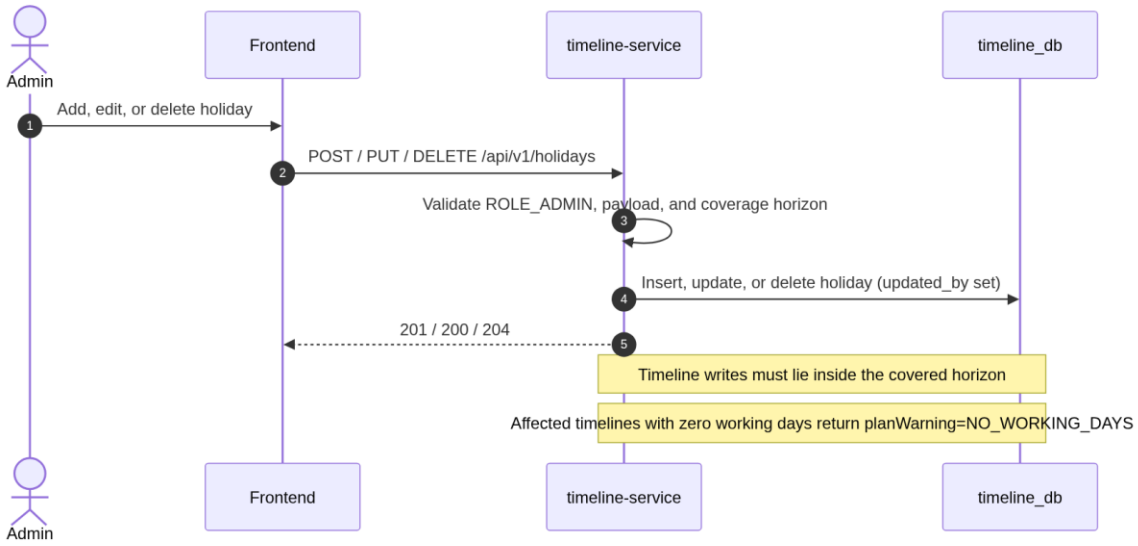


Figure 18 - Holiday administration sequence

8.6.2 Holiday endpoints

Table 55 - Holiday API endpoints

#	Method	Path	Role	Purpose
H1	GET	/api/v1/holidays?year=2026&page=0&size=100	Authenticated	Paginated holidays for a year, sorted by date.
H2	POST	/api/v1/holidays	ADMIN	Create a holiday.
H3	PUT	/api/v1/holidays/{id}	ADMIN	Update a holiday.
H4	DELETE	/api/v1/holidays/{id}	ADMIN	Delete a holiday.

8.6.3 Holiday fields and example

Table 56 - Holiday request fields

Field	Type	Required	Validation
date	date	Yes	ISO date, unique.
name	string	Yes	1-150 chars.
type	enum	Yes	NATIONAL or JOINT_LEAVE.

```
POST /api/v1/holidays
Authorization: Bearer <ADMIN token>
Content-Type: application/json

{
  "date": "2026-12-28",
  "name": "Cuti bersama akhir tahun 2026",
  "type": "JOINT_LEAVE"
}

HTTP/1.1 201 Created
```

```
{
  "id": "5a6b...",
  "date": "2026-12-28",
  "name": "Cuti bersama akhir tahun 2026",
  "type": "JOINT_LEAVE",
  "createdAt": "2026-10-19T08:30:00Z",
  "updatedAt": "2026-10-19T08:30:00Z"
}
```

```
GET /api/v1/holidays?year=2026&page=0&size=100
Authorization: Bearer <any authenticated user>

HTTP/1.1 200 OK
{
  "content": [
    { "id": "1...", "date": "2026-01-01", "name": "Tahun Baru 2026 Masehi", "type": "NATIONAL" },
    { "id": "2...", "date": "2026-01-16", "name": "Isra Mikraj Nabi Muhammad SAW", "type": "NATIONAL" }
  ],
  "page": 0,
  "size": 100,
  "totalElements": 2,
  "totalPages": 1,
  "first": true,
  "last": true
}
```

8.6.4 Holiday rules and errors

- Both holiday types reduce working days. This matches the Indonesian practice of cuti bersama.
- A holiday on a Saturday or Sunday is allowed and stored; it simply does not reduce the working-day count further.
- Holiday changes affect current timeline duration and plan percentage on the next read. Superseded and cancelled versions keep their snapshots.
- Every holiday write records created_by or updated_by. The holiday list is paginated with the standard page envelope.
- The coverage horizon is derived from the holiday table: currently 2025-01-01 to 2027-12-31. Timeline creates and edits outside it return 409 HOLIDAY_COVERAGE_REQUIRED.
- A holiday edit can reduce an existing timeline to zero working days; reads return durationWorkdays=0, percentPlan=0.00, and planWarning=NO_WORKING_DAYS.
- The 2025-2027 seed is loaded by Flyway and can be corrected by an admin.
- Deleting a holiday is a hard delete. The change is immediate.
- The year query parameter is optional; if omitted, the current year in Asia/Jakarta is used.

Table 57 - Holiday error cases

Condition	HTTP	Code
Duplicate holiday date	409	HOLIDAY_DATE_TAKEN
Holiday not found	404	HOLIDAY_NOT_FOUND
Invalid type or date	400	VALIDATION_ERROR
Non-admin create/update/delete	403	FORBIDDEN

8.7 Internal Service APIs

Internal APIs are available only on the private service network and require X-Internal-Key-Id and X-Internal-Api-Key. Each caller key is scoped to the endpoints in Section 7.8. These contracts are deliberately small and are not exposed in the public OpenAPI documentation used by the frontend.

Table 58 - Internal service API endpoints

#	Method	Path	Owner	Caller	Purpose
I1	GET	/internal/users/{userId}	user-service	auth-service, task-service	Profile and enabled state for password-target validation or PIC snapshot.
I2	GET	/internal/users/{userId}/auth-context	user-service	auth-service	Enabled state, roles, username, and full name during login/refresh.
I3	POST	/internal/users/bootstrap	user-service	auth-service	Idempotently ensure the bootstrap ADMIN profile and role exist.
I4	GET	/internal/projects/{projectId}/access?userId=...	user-service	task-service, timeline-service	Check project existence, membership, admin flag, and project status.
I5	POST	/internal/users/{userId}/revoke-tokens	auth-service	user-service	Revoke all active refresh-token families after deactivation or role change.

8.7.1 GET /internal/users/{userId}/auth-context

```
GET /internal/users/2b2e.../auth-context
X-Internal-Key-Id: auth-service-2026-10
X-Internal-Api-Key: <secret>

HTTP/1.1 200 OK

{
  "userId": "2b2e...",
  "username": "budi.santoso",
  "fullName": "Budi Santoso",
  "roles": ["USER"],
  "enabled": true
}
```

auth-service calls this endpoint after a successful local BCrypt verification and during refresh. It returns the current enabled state and roles so deactivation and role changes take effect at the next login or refresh. The password hash is never involved.

8.7.2 POST /internal/users/bootstrap

```
POST /internal/users/bootstrap
X-Internal-Key-Id: auth-service-2026-10
X-Internal-Api-Key: <secret>
Content-Type: application/json

{
  "username": "admin",
  "email": "admin@example.com",
  "fullName": "System Administrator"
}

HTTP/1.1 200 OK
{
  "userId": "0f4e...",
```

```
  "username": "admin",
  "roles": ["ADMIN"],
  "enabled": true,
  "created": true
}
```

The endpoint is idempotent by username. If the profile already exists, it returns the existing user with created=false. auth-service uses it once at bootstrap to ensure the profile and ADMIN role exist before storing the credential.

8.7.3 GET /internal/projects/{projectId}/access

```
GET /internal/projects/8f4c.../access?userId=2b2e...
X-Internal-Key-Id: task-service-2026-10
X-Internal-Api-Key: <secret>

HTTP/1.1 200 OK
{
  "projectId": "8f4c...",
  "userId": "2b2e...",
  "allowed": true,
  "member": true,
  "admin": false,
  "projectStatus": "ACTIVE",
  "projectCode": "PRJ-001"
}
```

- Unknown project returns 404 PROJECT_NOT_FOUND. Unknown user returns 404 USER_NOT_FOUND.
- A known user who is not a member returns 200 with allowed=false, member=false. The caller maps this to 403.
- An ADMIN returns allowed=true and admin=true even without a project_member row.
- Task PIC validation requires member=true. An ADMIN who is not a member is not eligible to be PIC.
- projectStatus is ACTIVE or ARCHIVED; callers use it to block writes.

8.7.4 GET /internal/users/{userId}

```
GET /internal/users/2b2e...
X-Internal-Key-Id: task-service-2026-10
X-Internal-Api-Key: <secret>

HTTP/1.1 200 OK
{
  "id": "2b2e...",
  "username": "budi.santoso",
  "fullName": "Budi Santoso",
  "enabled": true
}
```

task-service uses this endpoint to validate that the PIC exists, is enabled, and to capture the full-name snapshot. It calls the access endpoint separately to confirm project membership.

8.7.5 POST /internal/users/{userId}/revoke-tokens

```
POST /internal/users/2b2e.../revoke-tokens
X-Internal-Key-Id: user-service-2026-10
X-Internal-Api-Key: <secret>
Content-Type: application/json

{ "reason": "DEACTIVATED" }

HTTP/1.1 204 No Content
```

user-service calls this endpoint after deactivating a user or changing roles. auth-service revokes every active refresh-token family for the user. If the call fails, the refresh flow still rejects the user because auth-service checks the enabled state and roles with user-service on every refresh; the revoke is an immediate-cleanup control, not the only guard.

8.7.6 Internal API failure handling

Table 59 - Internal API failure handling

Condition	Caller response	Code
Missing/invalid internal key	401 from the owning service	INTERNAL_UNAUTHORIZED
Valid key used outside its scope	403 from the owning service	INTERNAL_FORBIDDEN
user-service timeout or connection refused	503 to the browser	INTERNAL_DEPENDENCY_UNAVAILABLE
user-service returns 5xx	503 to the browser	INTERNAL_DEPENDENCY_UNAVAILABLE
user-service returns 404 for project/user	404 with the corresponding public code	PROJECT_NOT_FOUND / USER_NOT_FOUND

9. Frontend Design

9.1 Technology choice

The frontend is a React 19 single-page application written in TypeScript and built with Vite. Ant Design 5 provides the enterprise table, form, date-picker, drawer, modal, and grid components that fit a project-management back office. TanStack Query handles server state and cache invalidation; React Hook Form and Zod handle form state and validation; Axios handles JWT and refresh behaviour.

Table 60 - Frontend technology choices

Concern	Choice	Reason
Framework	React 19 + TypeScript 5	Largest ecosystem, strong typing, component reuse.
Build	Vite 7	Fast development server and optimized static build.
UI kit	Ant Design 5	Best-fit tables, forms, date pickers, drawers, and responsive grid.
Routing	React Router 7	Nested layouts, protected routes, project tabs.
Server state	TanStack Query 5	Caching, background refetch, mutation invalidation.
Forms	React Hook Form + Zod	Minimal re-renders and shared validation rules.
HTTP	Axios	Interceptors for bearer token, refresh-once, request ID, and error mapping.
Dates	Day.js	Lightweight date formatting and Asia/Jakarta display.
Testing	Vitest + React Testing Library + MSW	Fast component tests with mocked APIs.

9.2 Pages and routes

Table 61 - Frontend routes

Route	Page	Access	Main content
/login	Login	Public	Username/password form, inline error, redirect after login.
/	Dashboard	Authenticated	Project summary cards, my open tasks, current timeline summaries.
/projects	Projects	Authenticated	Admin: all projects and create action. User: member projects only.
/projects/:projectId/tasks	Project Tasks	Project member / ADMIN	Filterable task table, create/edit drawer, status and progress actions.
/projects/:projectId/timeline	Project Timeline	Project member / ADMIN	Current vs plan, draft editor, version history, publish/cancel/revision actions.
/projects/:projectId/members	Project Members	Project member / ADMIN	Member list; ADMIN can add/remove.
/users	Users	ADMIN	User table, create/edit drawer, role assignment, activate/deactivate, password reset.
/holidays	Holidays	ADMIN write / authenticated read	Year filter, holiday table, create/edit/delete.
/profile	Profile	Authenticated	Own profile and change password.
*	Not Found / Forbidden	Any	Friendly 403/404 page with return-to-dashboard action.

9.3 Responsive behaviour for laptop and tablet

Table 62 - Responsive layout rules

Area	Laptop (>= 1200 px)	Tablet (768-1199 px)
App shell	Expanded 240 px sidebar + 64 px header.	Collapsed 72 px sidebar; labels shown in tooltip. Full navigation remains reachable.

Area	Laptop (>= 1200 px)	Tablet (768-1199 px)
Content width	Max-width 1440 px with 24 px gutters.	Fluid width with 16 px gutters.
Task table	All columns visible; sticky header; inline actions.	Horizontal scroll with sticky first column; actions in a compact menu.
Forms	Two-column layout in a 520 px drawer.	Single-column layout in a full-width drawer.
Timeline	Version cards side by side with progress bars and date range.	Stacked cards; horizontal date bar scrolls inside its card.
Dashboard	4 summary cards + two-column panels.	2 summary cards per row; panels stack.
Modals	Centered, 640 px maximum.	Near-full-width with 16 px margins.
Touch targets	Mouse-sized controls acceptable.	Minimum 44 x 44 px for primary actions and table row actions.
Typography	14 px base, 24-30 px page title.	14 px base, 22-26 px page title.
Navigation	Sidebar always visible.	Sidebar can expand as an overlay drawer; no horizontal page scroll.

- The supported primary viewports are laptop (1366 x 768 and above) and tablet landscape/portrait (768-1199 px).
- A phone layout is not a release-1 target, but the grid and drawer patterns should remain usable below 768 px.
- Tables never force the whole page wider than the viewport; only the table container scrolls horizontally.
- All primary actions remain visible on tablet without hover-only affordances.
- Charts and timeline bars use responsive containers and reflow on resize.

9.4 Application structure

```
src/
  app/
    App.tsx                 route tree and providers
    queryClient.ts          TanStack Query defaults
    theme.ts                Ant Design theme tokens
  auth/
    AuthProvider.tsx        current user, access token in memory
    RequireAuth.tsx         route guard
    RequireRole.tsx         ADMIN guard
    axios.ts                axios instances per service + refresh interceptor
  pages/
    LoginPage.tsx
    DashboardPage.tsx
    ProjectsPage.tsx
    ProjectTasksPage.tsx
    ProjectTimelinePage.tsx
    ProjectMembersPage.tsx
    UsersPage.tsx
    HolidaysPage.tsx
    ProfilePage.tsx
  features/
    tasks/                  api.ts, hooks.ts, TaskTable.tsx, TaskFormDrawer.tsx
    timeline/               api.ts, hooks.ts, TimelinePanel.tsx, DraftEditor.tsx, VersionHistory.tsx
    projects/               api.ts, hooks.ts, ProjectTable.tsx, ProjectFormDrawer.tsx
    users/                  api.ts, hooks.ts, UserTable.tsx, UserFormDrawer.tsx
    holidays/               api.ts, hooks.ts, HolidayTable.tsx, HolidayFormDrawer.tsx
  components/
    AppShell.tsx
    PageHeader.tsx
    ErrorState.tsx
    EmptyState.tsx
    ConfirmAction.tsx
    ProgressCell.tsx
  lib/
    dates.ts                Asia/Jakarta formatting
    errors.ts               Problem Details mapping
    validation.ts           Zod schemas
  types/
    api.ts                  shared response and request types
```

9.5 Authentication and API layer

- The access token is stored only in memory in AuthProvider. It is never written to localStorage or sessionStorage.
- On app load, the SPA calls POST /api/v1/auth/refresh with the HttpOnly cookie. A successful response restores the session; a 401 sends the user to login.
- There is one Axios instance per service, each with the configured base URL. A shared response interceptor detects 401, performs one refresh, and retries the original request once.
- Concurrent 401 responses share a single in-flight refresh promise. Cross-tab refresh is serialized with the Web Locks API and a BroadcastChannel fallback, so two tabs cannot rotate the same token simultaneously.
- Logout calls auth-service, clears the in-memory token, clears TanStack Query cache, and navigates to /login.
- Every request sends X-Request-Id generated with crypto.randomUUID() for traceability. Create and publish bodies also send a separate requestId UUID so retries are idempotent.
- The app treats 409 CURRENT_VERSION_CHANGED or DRAFT_MODIFIED as a conflict: it reloads the current version, shows a diff-style notice, and asks the user to reapply the change.
- Password change and admin password reset call auth-service. On success the SPA clears the session and sends the user to login because all refresh tokens are revoked.
- Route guards use the role from /auth/me. The backend remains authoritative.

```
// Conceptual refresh interceptor
response interceptor:
  if error.status != 401 or request already retried: reject
  if request.url is /auth/login or /auth/refresh: reject
  await singleFlightRefresh()
  retry original request with new Authorization header
```

9.6 Timeline screen behaviour

The timeline screen is the most important interaction. It must make the difference between the active published version and the editable draft obvious.

Table 63 - Timeline screen states

State	UI treatment
No timeline	Empty state with a Create Timeline button. Opens a date-range form and explains the draft/publish flow.
Draft only (never published)	Amber 'Draft - not active' banner. Publish and Cancel Draft actions. Plan % is labelled 'Preview'. Actual % and status show as 'starts at OPEN / 0%' and are not editable on the draft.
Published current, no draft	Green 'Current vN' badge. Show actual vs plan progress. Actions: Revise Timeline, Update Status, Update Actual %.
Published current + draft	Current remains fully visible; a separate amber draft card shows proposed dates and notes only. Actual/status are read from the current version and are carried forward at publish. Actions: Edit Draft, Publish Draft, Cancel Draft.
Superseded history	Paginated version history table; each row shows version, lifecycle, dates, snapshot duration, snapshot plan %, snapshot actual %, as-of date, actor, and notes.
Zero working days	Amber warning banner: 'This range has no working days after the latest holiday changes. Edit the dates or review the holiday calendar.' Plan % shows 0.00.
Archived project	Read-only banner; all mutation actions disabled with tooltip 'Project is archived'.

- Publish and Cancel Draft require explicit confirmation modals that explain the effect on the current version.
- Publishing a revision explains that the current operational status and actual % will be carried forward, while the previous version's values are snapshotted.
- Publish sends the loaded expectedDraftUpdatedAt and, when a current version exists, expectedCurrentUpdatedAt; a 409 triggers the reload-and-reapply flow instead of a blind retry.

- Cancelling the only draft of a never-published timeline warns that the timeline will be removed and can be created again.
- Progress is displayed with two numbers: Actual % (manual) and Plan % (calculated), plus the difference.
- The plan-as-of date is shown next to Plan % so the user understands the snapshot. For past versions, it is the stored snapshot date.
- Duration is shown in working days with a tooltip: 'Excludes weekends and Indonesian holidays'.
- The version history is never hidden; it is paginated and revisions remain auditable.
- Conflict responses (409 CURRENT_VERSION_CHANGED / DRAFT_MODIFIED) do not discard the user's input; the screen offers a reload-and-reapply flow.

9.7 Loading, empty, and error states

- Every page has a skeleton or spinner for the initial load and a retry action on failure.
- Empty states explain the next action, for example 'No tasks yet. Create the first task.'
- Problem Details errors are shown in a notification and, for validation, mapped to form fields.
- 403 pages explain that the user is not a member of the project and offer a return to Projects.
- Optimistic updates are limited to status/progress toggles; all other mutations wait for the server and invalidate the relevant query.
- Query keys are scoped by project ID and filters so switching projects does not show stale data.

9.8 Accessibility and UX

- All form controls have labels; validation errors are announced and associated with fields.
- Primary actions are reachable by keyboard; focus is trapped in drawers/modals and returned on close.
- Color is not the only signal for status: badges include text labels.
- Contrast targets WCAG 2.1 AA.
- Date inputs use the Ant Design DatePicker with explicit yyyy-MM-dd display.
- Destructive actions require confirmation and are visually distinct.

10. Non-Functional Requirements

10.1 Performance

Table 64 - Performance targets

Metric	Target	Measurement / notes
Login response	p95 <= 800 ms	Includes BCrypt cost 12 and the internal auth-context call.
Authenticated API read	p95 <= 500 ms	Single-service request, page size 20, 10,000 rows per project.
Task list	p95 <= 500 ms	Indexed project_id filters and pagination.
Timeline aggregate	p95 <= 500 ms	Loads at most one current and one draft; one indexed holiday range query.
Timeline calculation	<= 50 ms for the 3-year API horizon	Single date pass with a holiday set; pure calculator benchmarked at 10 years <= 150 ms.
Frontend first contentful paint	<= 2 s on a typical laptop/tablet network	Static build, route-level code splitting, Ant Design tree-shaking.
Page interaction	<= 100 ms for local state changes	Server calls are asynchronous with skeletons.

10.2 Scalability and availability

- All services are stateless except for auth refresh-token state in PostgreSQL; they can be scaled horizontally behind a future load balancer or gateway.
- HikariCP pool default 10 connections per service; tune to database capacity.
- Release 1 targets a single organisation with up to 200 users, 100 projects, and 50,000 tasks. The schema and pagination support larger data sets, but capacity testing is required before that claim.
- A service instance failure affects only its capability. The frontend shows an error and retry action; no partial write is attempted.
- The local Docker Compose stack is single-instance. Production high availability is an infrastructure concern outside release 1.

10.3 Security requirements

Table 65 - Security requirements

Area	Requirement
Authentication	RS256 JWT access token validated via cached JWKS, rotating refresh-token family, BCrypt passwords, disabled-account enforcement.
Credential ownership	Password hashes, lock state, refresh tokens, and signing keys live in auth_db/auth-service only. user-service never stores or accepts a password.
Authorization	ADMIN/USER role checks plus live project-membership checks for every project-scoped request, re-checked immediately before commit on writes.
Transport	HTTPS in production; HSTS at the edge.
Input validation	Bean Validation on every request DTO; allow-listed sort fields; escaped search input.
Output safety	No password hashes, refresh tokens, internal secrets, or signing keys in responses or logs.
CORS	Explicit origin allow-list; credentials only for auth-service; no wildcard with credentials.
CSRF	SameSite=Lax refresh cookie, allow-listed CORS, no state-changing GET. Add a CSRF token before exposing the app to untrusted browser extensions or cross-site contexts.
Secrets	Environment variables or a secrets manager; never committed. The RSA private key exists only in auth-service. Internal caller keys are per-service and at least 256 bits.
Dependencies	Dependency and container scanning in CI; no critical/high vulnerabilities at release.
Audit	Actor UUIDs and timestamps on writes, plus a timeline_version_event ledger for timeline lifecycle changes.

10.4 Observability

- Each service exposes separate liveness (/actuator/health/liveness) and readiness (/actuator/health/readiness) groups. Readiness includes database connectivity and completed Flyway migrations; liveness does not depend on the database.
- Micrometer HTTP server and client metrics are mandatory in every service, including request count, latency percentiles, error rate, internal dependency latency, refresh reuse detections, and timeline publish conflicts.
- Structured logs include timestamp, service, level, requestId, traceId, spanId, method, path, status, and duration. Tokens, passwords, and refresh hashes are redacted.
- W3C traceparent is propagated on every internal call through OpenTelemetry/Micrometer Tracing. X-Request-Id is also propagated and returned in Problem Details.
- Metrics are exposed on the management port on the internal network and scraped by the deployment monitoring stack.
- Log retention and central aggregation are deployment concerns; JSON logs are recommended in production.

10.5 Browser and device support

- Latest two major versions of Chrome, Edge, Firefox, and Safari.
- Laptop: 1366 x 768 and above. Tablet: 768 x 1024 and 1024 x 768. Phone is best effort, not a release target.
- No Internet Explorer support.
- The UI uses responsive CSS and Ant Design breakpoints; no separate tablet application.

10.6 Data volume and retention

- Expected release-1 volume: <= 200 users, <= 100 projects, <= 50,000 tasks, <= 1,000 timeline versions, <= 100 holidays per year.
- Tasks are paginated and indexed by project, status, PIC, and dates.
- Timeline version history is paginated with default size 50 and a maximum of 100; there is no version cap.
- timeline_version_event is append-only and indexed by timeline_version_id and occurred_at. It grows with lifecycle actions, not with page views.
- Refresh tokens are retained until expiry plus a short grace period for reuse detection; a maintenance job can purge expired rows.
- Database backups are daily with 7-day retention in the target environment. Restore testing is an operational task.

10.7 Maintainability

- Package-by-feature inside each service, with controller, service, repository, domain, and DTO boundaries.
- Constructor injection, immutable request/response records, and explicit mappers.
- No business logic in controllers or entities beyond simple invariants.
- The working-day and plan-progress calculators are pure, deterministic, and heavily unit-tested.
- Configuration is externalised and environment-specific; no secrets in source.
- Database changes are Flyway migrations and reviewed like code.

11. Testing Strategy and Definition of Done

11.1 Test principles

- Unit tests are mandatory and are written with the feature, not after it.
- Business calculations (working days, plan completion, duration, status transitions) are pure and covered exhaustively.
- Repository tests use real PostgreSQL through Testcontainers, not H2, because partial indexes, advisory locks, CHECK constraints, and generated columns are part of the contract.
- Controller tests verify HTTP status, validation, security, and Problem Details shape.
- Integration tests verify the cross-service seams with WireMock for user-service, including the final access re-check before commit.
- Concurrency tests run parallel refresh rotation, publish, revision-create, and last-admin updates against real PostgreSQL to prove the conditional update, row locks, partial unique indexes, and advisory lock.
- Idempotency tests replay the same requestId for project, task, timeline, revision, and publish and assert that exactly one row exists and the original resource is returned.
- Frontend tests verify user-visible behaviour, not implementation details, including cross-tab refresh serialization in a simulated two-tab test.
- The 100% line coverage gate applies to production business code with explicitly documented exclusions. Branch coverage is gated for calculators and state machines, and mutation testing guards the formulas.

11.2 Backend test layers

Table 66 - Backend test layers

Layer	Tools	Scope	Examples
Pure unit	JUnit 5, AssertJ	Calculators, value objects, validators, mappers	Working-day count, plan %, duration, date range, status transition.
Service unit	JUnit 5, Mockito, AssertJ	Service orchestration with mocked repositories and internal clients	Create task, PIC validation, publish draft, last-admin guard, refresh rotation.
Web slice	Spring Boot Test, MockMvc, Spring Security Test	Controller, validation, JSON, security filters	401/403, 400 field errors, 201 Location header, pagination.
Repository	DataJpaTest, Testcontainers PostgreSQL	Mappings, constraints, indexes, generated columns, locks	Duplicate member, one draft per timeline, generated duration, CLOSED=>100 CHECK, advisory lock.
Integration	SpringBootTest, Testcontainers, WireMock	Full service flow with real DB and stubbed dependency	Login -> task access -> timeline publish; holiday change -> plan %; idempotent replay; carry-forward publish.
Concurrency	SpringBootTest, Testcontainers, ExecutorService	Parallel transactions and conflict mapping	Double refresh, double publish, concurrent publish vs status/progress update, revision race, last-admin race, final access re-check.
Mutation	PIT (pitest) on domain/calculator packages	Formula and state-machine strength	Working-day count, plan %, duration, status/version transitions.
Contract	OpenAPI comparison (optional)	Generated OpenAPI matches approved paths/schemas	CI diff for breaking changes.

11.3 Required unit-test scenarios

Table 67 - Required backend unit and slice scenarios

Area	Required cases
Working-day calculator	Same-day range; weekend-only range rejected on create; holiday on weekday; holiday on weekend; range crossing year boundary; leap day; horizon boundaries; start/end inclusive; 10-year pure benchmark; zero working days after holiday edit returns warning.

Area	Required cases
Plan completion	Before start = 0; on start = daily increment; weekend/holiday no increment; end = 100; after end = 100; rounding half-up; 2-decimal output; zero working days returns 0.00 + NO_WORKING_DAYS and never divides by zero; snapshot frozen on supersede.
Task duration	Same-day task = 1; range with weekend; range with holiday (holiday still counted); end before start rejected; 10-year maximum.
Task status	All allowed transitions; closing sets 100; progress edit on CLOSED returns 409 TASK_CLOSED; PUT CLOSED with <100 returns 400; reopen keeps value; invalid transition rejected.
Timeline lifecycle	Initial draft; publish initial; edit draft; cancel draft; cancel initial draft deletes timeline identity; cancel revision keeps history; revise; publish revision carries forward current actual/status; snapshot old current; expected current timestamp required on publish; expected updated-at conflicts; one-draft/one-current constraints; immutable published dates; draft stores OPEN/0; zero-workday publish warning.
Timeline access	Member allowed; non-member 403; admin allowed; archived project read-only; unknown project 404; final access re-check rolls back a write when membership is removed.
Holiday	Create/update/delete; duplicate date 409; year filter pagination; weekend holiday allowed; both types excluded; coverage horizon derivation; zero-workday warning after edit.
Users and roles	Unique username/email; last-admin advisory lock under concurrency; self-deactivation; role replacement; disabled login; password policy; token revocation on role/status change; profile without credential cannot log in.
Projects and members	Unique code; idempotent create; member add idempotency; removal; archived project; non-member visibility.
Auth	Login success/failure/disabled/locked; atomic refresh rotation; token-family reuse revocation; cross-tab single flight; logout idempotency; role refresh; JWKS unknown-kid refresh; password reset/change revokes families; deactivation revokes families.
Idempotency	Replay of project/task/timeline/revision/publish requestId returns the original resource, adds no row, and sets Idempotent-Replay: true.
Internal security	Missing key 401; wrong key 401; valid key outside scope 403; cross-database connection denied for every service role.

11.4 Frontend test layers

Table 68 - Frontend test layers

Layer	Tools	Scope
Unit	Vitest	Date formatting, error mapping, Zod schemas, progress formatting, query-key builders.
Component	React Testing Library + MSW	Login form, project table, task table/form, timeline draft/publish, zero-working-day warning, user and holiday forms, role guards.
Integration	RTL + MSW + MemoryRouter	Login -> project -> task create; draft -> publish -> revise; 401 -> refresh -> retry; cross-tab refresh lock; 409 conflict -> reload and reapply; 403 page.
Accessibility	jest-axe or axe-core (optional)	Form labels, dialog focus, table headers, color-independent status.
End-to-end	Playwright (optional, phase 2)	Smoke: login, create task, create/publish/revise timeline, admin user creation.

11.5 Coverage gate

The approved acceptance criterion is 100% line coverage of production business code. Coverage is a floor, not a substitute for meaningful assertions. The build fails if a gated scope is below its threshold. Boundary and presentational code has a separate, lower gate so the business-code target is not diluted by generated or glue code.

Table 69 - Coverage gates and exclusions

Scope	Line gate	Branch / mutation	Exclusions

Scope	Line gate	Branch / mutation	Exclusions
Backend core: domain, service, calculator, validator, mapper	100% lines (JaCoCo check bound to Maven verify)	Branch >= 95% on calculators and state machines; PIT mutation score >= 90% on calculator/domain packages	None. This is the gated business-code scope.
Backend boundary: controller, repository adapter, security, config	>= 90% lines, reported (advisory gate)	Covered by web slice, repository, and integration tests	Application bootstrap, generated OpenAPI interfaces, Flyway migrations, Swagger config.
Frontend features, auth, and lib	100% lines (Vitest per-directory threshold)	Branch >= 90% on timeline state and auth refresh logic	Type-only files, MSW handlers, test setup, generated assets.
Frontend shared presentational components	>= 90% lines (Vitest per-directory threshold)	Behaviour covered through feature tests	main.tsx, vite-env.d.ts, generated assets.

- JaCoCo, Vitest, and PIT reports are published as CI artifacts.
- Branch coverage is reported for all code and gated at 95% for calculators/state machines and 90% for frontend timeline/auth logic.
- PIT mutation testing runs on the working-day, plan-completion, duration, and timeline state-machine packages; the mutation score gate is 90%.
- Excluded classes must be listed in the build configuration and reviewed; adding an exclusion to hide untested logic is not acceptable.
- Tests must assert outcomes, errors, and side effects, not only execute lines.
- Concurrency, idempotency, and cross-service integration tests are required in addition to line coverage because they cannot be replaced by unit mocks.

11.6 Feature acceptance matrix

Table 70 - Feature acceptance matrix

Feature	Acceptance evidence
Login / refresh / logout	Unit tests for RS256/JWKS validation, token rotation, family reuse revocation; MockMvc tests for 200/401/403/423; frontend login, cross-tab refresh, and password-reset revocation tests.
Users / roles	CRUD and guard tests; last-admin advisory lock; self-deactivation; role/status token revocation; profile-without-credential login; frontend user form and role guard tests.
Projects / members	Visibility tests for admin vs user vs non-member; idempotent create; member add/remove; archive behaviour; final access re-check tests.
Tasks	Duration and validation tests; PIC membership tests; status/progress tests including CLOSED rules; idempotent create; list filter/pagination tests; frontend table/form tests.
Timeline	Draft/publish/revise/cancel tests; initial-draft identity deletion; carry-forward and snapshots; one-draft/one-published constraints; expected-timestamp conflicts; zero-working-day warning; idempotent publish; history pagination; frontend wizard and conflict tests.
Holidays	Seed verification; CRUD tests; duplicate date; weekend holiday; horizon derivation and HOLIDAY_COVERAGE_REQUIRED; zero-workday warning; plan calculation effect.
Idempotency and concurrency	Parallel refresh, publish, revision, and last-admin tests; replay tests for all create/publish requestId values; WireMock final access re-check.
Database isolation	Per-service role grants; REVOKE CONNECT on other databases; cross-database connection denial test.
Responsive UI	Component tests at laptop/tablet widths; manual viewport check in Chrome and Edge.
Deployment	Docker Compose up; liveness/readiness; Flyway migrations; auth-service bootstrap admin; smoke flow.

11.7 CI pipeline

```
Backend (per service):
  mvn -B clean verify
    -> unit + slice + repository + integration + concurrency tests
    -> JaCoCo core business-code 100% line check
    -> PIT mutation check (calculator/domain packages, score >= 90%)
    -> OpenAPI generation smoke test
    -> cross-database denial test
```

```
Frontend:
  npm ci
  npm run lint
  npm run test:coverage
  npm run build
  -> Vitest per-directory line thresholds
  -> production bundle

Compose smoke:
  docker compose up -d --build
  wait for liveness/readiness endpoints
  run smoke script: login -> list projects -> create task -> create/publish/revise timeline -> replay a
create
  docker compose down -v
```

11.8 Definition of Done

- Every in-scope endpoint in Section 8 is implemented and matches the documented contract.
- Every business rule in Section 6 is implemented and covered by automated tests.
- Backend Maven verify passes with the 100% core business-code line gate, the calculator/state-machine branch gate, and the PIT mutation gate.
- Frontend lint, tests, per-directory coverage gates, and production build pass.
- Docker Compose starts all services, runs Flyway, seeds roles and holidays, and bootstraps the admin through auth-service.
- Liveness and readiness probes pass, and Micrometer/OpenTelemetry metrics and traces are visible.
- The smoke flow works: login, project access, task create/update, timeline draft/publish/revision, and an idempotent create replay.
- Cross-database access is denied for every service role; per-caller internal keys and scopes are enforced.
- No critical or high dependency/container vulnerabilities at release review.
- OpenAPI documents are generated and reviewed against this design.
- The technical design document is updated for any approved change.

12. Error Handling and Observability

12.1 Backend error model

Each service has a @RestControllerAdvice that converts known exceptions into Problem Details. Unknown exceptions are logged with the request ID and returned as a generic 500 without a stack trace, SQL fragment, or internal class name.

Table 71 - Backend exception mapping

Exception	HTTP	Code	Handling
MethodArgumentNotValidException	400	VALIDATION_ERROR	Field errors are mapped to errors[].
ConstraintViolationException	400	VALIDATION_ERROR	Query/path parameter violations are mapped to field errors.
BusinessRuleException	400	Specific code	Used for PIC_NOT_MEMBER, INVALID_DATE_RANGE, NO_WORKING_DAYS, INVALID_COMPLETION_FOR_CLOSED.
ResourceNotFoundException	404	Specific code	Task, project, user, holiday, timeline, draft.
ConflictException	409	Specific code	Duplicates, last admin, archived project, lifecycle conflicts, holiday coverage, closed-resource writes, expected-timestamp conflicts.
AccountLockedException	423	AUTH_ACCOUNT_LOCKED	Credential locked after five failed logins.
AccessDeniedException	403	FORBIDDEN	Role or project access denied, including the final pre-commit re-check.
AuthenticationException	401	UNAUTHORIZED	Missing, invalid, or expired access token.
InternalDependencyException	503	INTERNAL_DEPENDENCY_UNAVAILABLE	Internal REST timeout or 5xx.
Exception	500	INTERNAL_ERROR	Logged; generic detail returned.

12.2 Validation errors

```
{
  "type": "https://prolean.local/errors/validation-error",
  "title": "Validation failed",
  "status": 400,
  "detail": "One or more fields are invalid.",
  "instance": "/api/v1/projects/8f4c.../tasks",
  "code": "VALIDATION_ERROR",
  "requestId": "01J...",
  "timestamp": "2026-10-19T10:15:30Z",
  "errors": [
    { "field": "endDate", "message": "must be on or after startDate" },
    { "field": "percentComplete", "message": "must be between 0 and 100" }
  ]
}
```

- Field names use the JSON property names the client sent, not Java field names.
- Errors are deterministic and stable enough for frontend field mapping.
- No exception stack trace is returned to the client.

12.3 Dependency failures

- task-service and timeline-service fail closed: if user-service access cannot be confirmed, the request returns 503 rather than allowing access.
- auth-service returns 503 if user-service is unavailable during login or refresh.
- Internal GET calls use a 2-second connect timeout and a 3-second read timeout. There is no retry for writes and at most one retry for idempotent reads if the failure is a connection reset.

- The frontend shows a retryable error notification and keeps the user on the current page.
- A future release can add Resilience4j circuit breakers and a short-lived membership cache, but v1 favours correctness over availability.

12.4 Logging

- Log at INFO for successful state changes, WARN for expected business conflicts, and ERROR for unexpected failures.
- Include requestId, traceId, spanId, service, method, path, status, duration, and actor userId where available.
- Never log passwords, access tokens, refresh tokens, token hashes, Authorization headers, RSA private keys, or internal caller secrets.
- SQL logging is disabled in production; Flyway and migration logs are kept.
- The frontend logs only sanitized error metadata in development; production logging is disabled unless a monitoring integration is added.

12.5 Health endpoints

Table 72 - Operational endpoints

Endpoint	Purpose	Exposure
/actuator/health/liveness	Process is up; no database dependency.	Docker/Kubernetes liveness probe; internal network.
/actuator/health/readiness	Database connectivity and completed Flyway migrations.	Docker/Kubernetes readiness probe; internal network.
/actuator/info	Build and version information.	Internal network.
/actuator/prometheus	Micrometer metrics for scraping.	Internal monitoring network only.
/swagger-ui.html	API documentation.	Local/dev only; disabled or protected in production.

13. Deployment and Operations

13.1 Docker Compose topology

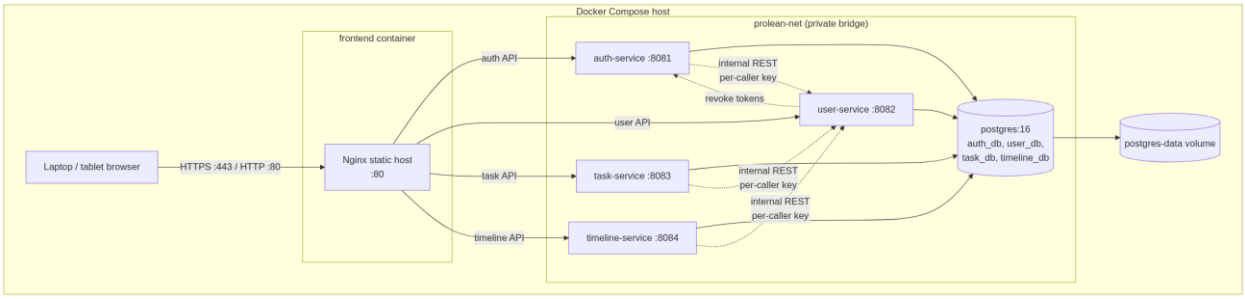


Figure 19 - Docker Compose deployment topology

Table 73 - Docker Compose services

Container	Image / build	Port	Depends on	Health check
postgres	postgres:16	5432 (optional host publish)	-	pg_isready
auth-service	Maven build -> JRE 21	8081	postgres healthy	/actuator/health/readiness
user-service	Maven build -> JRE 21	8082	postgres healthy	/actuator/health/readiness
task-service	Maven build -> JRE 21	8083	postgres healthy, user-service ready	/actuator/health/readiness
timeline-service	Maven build -> JRE 21	8084	postgres healthy, user-service ready	/actuator/health/readiness
frontend	Node build -> Nginx 1.27	80 / 3000	four services healthy	GET /

13.2 Environment configuration

Table 74 - Environment variables

Variable	Used by	Example / note
SPRING_DATASOURCE_URL	Each service	jdbc:postgresql://postgres:5432/auth_db (service-specific).
SPRING_DATASOURCE_USERNAME / PASSWORD	Each service	Dedicated database role per service; no shared superuser.
JWT_ACTIVE_KID	auth-service	Key ID placed in the JOSE header, for example 2026-10.
JWT_PRIVATE_KEY_PEM	auth-service	RSA private key from a secret manager. Never committed or logged.
JWT_RETIRED_PUBLIC_KEYS	auth-service	JSON array of {kid, publicKeyPem} for retired keys; published in JWKS until their access tokens expire.
AUTH_JWKS_URL	user-service, task-service, timeline-service	http://auth-service:8081/.well-known/jwks.json.
JWT_ACCESS_TTL_SECONDS	auth-service	900.
JWT_REFRESH_TTL_DAYS	auth-service	7.
JWT_ISSUER / JWT_AUDIENCE	All services	prolean-auth / prolean-api.
INTERNAL_KEY_ID / INTERNAL_KEY_SECRET	Each service	This service's caller identity. Two IDs are active during rotation.
INTERNAL_ALLOWED_CALLERS	Each receiving service	JSON map of accepted keyId -> {caller, secret}; scopes are defined in code/configuration.
USER_SERVICE_URL	auth-service, task-service, timeline-service	http://user-service:8082.
AUTH_SERVICE_URL	user-service	http://auth-service:8081.

Variable	Used by	Example / note
CORS_ALLOWED_ORIGINS	All services	http://localhost:5173,http://localhost:3000.
BOOTSTRAP_ADMIN_USERNAME	auth-service	admin.
BOOTSTRAP_ADMIN_PASSWORD	auth-service	Required on first boot if no active admin credential exists; never logged.
BOOTSTRAP_ADMIN_EMAIL	auth-service	admin@example.com.
TZ	All services	Asia/Jakarta for business date calculations.
SPRING_PROFILES_ACTIVE	All services	local, dev, or prod.
OTEL_EXPORTER_OTLP_ENDPOINT	All services	Collector endpoint; traces and metrics.
VITE_AUTH_API_URL	frontend build/runtime	http://localhost:8081/api/v1.
VITE_USER_API_URL	frontend build/runtime	http://localhost:8082/api/v1.
VITE_TASK_API_URL	frontend build/runtime	http://localhost:8083/api/v1.
VITE_TIMELINE_API_URL	frontend build/runtime	http://localhost:8084/api/v1.

13.3 Database initialisation and migrations

1. The postgres container starts with an init script that creates auth_db, user_db, task_db, timeline_db and four service-specific database roles.
2. The init script runs REVOKE CONNECT ON DATABASE <other_db> FROM PUBLIC and grants CONNECT only to the owning role. Each role also gets USAGE/CREATE only on its own schema.
3. Each service connects with its own role and runs Flyway migrations on startup.
4. user-service seeds roles ADMIN and USER. It has no password column and never receives a password.
5. auth-service seeds no credential by default. On first startup, if no active admin credential exists and BOOTSTRAP_ADMIN_PASSWORD is set, it calls user-service POST /internal/users/bootstrap and then stores a BCrypt hash in auth_credential.
6. timeline-service seeds the 2025-2027 Indonesian holiday calendar in its V2 migration.
7. Flyway failures stop the service; the readiness probe fails and the Compose stack reports the failure.
8. No service reads or writes another service's database. A cross-database connection test runs in CI for every role and must fail.

13.4 Local startup and URLs

```
# Build and start everything
docker compose up -d --build

# Check readiness
curl http://localhost:8081/actuator/health/readiness
curl http://localhost:8082/actuator/health/readiness
curl http://localhost:8083/actuator/health/readiness
curl http://localhost:8084/actuator/health/readiness

# Public keys
curl http://localhost:8081/.well-known/jwks.json

# Frontend
http://localhost:3000

# API documentation (dev only)
http://localhost:8081/swagger-ui.html
http://localhost:8082/swagger-ui.html
http://localhost:8083/swagger-ui.html
http://localhost:8084/swagger-ui.html
```

13.5 Production notes

- Terminate TLS at Nginx or a managed load balancer. Redirect HTTP to HTTPS.
- Block /internal/** and /actuator/** from the public edge; expose only the four external API prefixes, /.well-known/jwks.json, and the frontend.
- Use a secrets manager or orchestration secrets for the RSA private key, per-caller internal keys, database passwords, and the bootstrap admin password.
- The RSA private key is configured only in auth-service. Other services need only the JWKS URL.
- Use per-caller internal keys with rotation overlap and, where the network supports it, mTLS between services plus Kubernetes network policies or equivalent segmentation.
- Set Secure cookies in production; the local profile may disable Secure only for HTTP development.
- Use a managed PostgreSQL service with automated backups and point-in-time recovery for production.
- Run at least two replicas of each stateless service behind a load balancer if availability requirements grow. Refresh-token rotation stays correct because the conditional update is atomic in PostgreSQL.
- API gateway remains optional; if added, Kong or Spring Cloud Gateway sits in front of the four services, and the frontend base URLs change to the gateway URL.
- No sticky sessions are required because all services are stateless apart from database state.

13.6 Backup and recovery

- Daily logical backup per database (pg_dump) with 7-day retention in release 1.
- Restore procedure is documented and tested before production use.
- Timeline and task data are business-critical; the PostgreSQL volume must not be the only copy. The timeline_version_event ledger and snapshot columns make the timeline state reconstructable.
- Refresh tokens can be lost without business data loss; users simply log in again.

14. Risks, Limitations, and Future Work

14.1 Risks and mitigations

Table 75 - Risk register

Risk	Likelihood	Impact	Mitigation
100% line coverage encourages low-value tests.	Medium	Medium	Scope the 100% gate to core business code, add branch and mutation gates for calculators/state machines, and review tests for meaningful assertions.
Live project access checks add a user-service dependency to task/timeline requests.	Medium	High	Timeouts, fail-closed 503, readiness probes. Add a short-lived cache or circuit breaker only after measuring.
No API gateway means four origins, four CORS configurations, and four base URLs.	High	Low	Environment-based configuration, Nginx edge, one shared CORS property template. Add Kong/SCG later if needed.
Indonesian holiday data can change or be corrected.	Medium	Medium	Seed from official SKB, admin CRUD, representative tests, snapshots for past versions, and visible planAsOfDate/planWarning.
The holiday coverage horizon (2025-2027) blocks timelines beyond it.	Medium	Medium	Clear 409 HOLIDAY_COVERAGE_REQUIRED error and admin workflow to add future holiday years before planning them.
Timezone mistakes around 'today' and weekends/holidays.	Medium	High	Use date-only business values, Asia/Jakarta explicitly, and pure calculators with boundary tests.
Concurrent timeline publishes could create two current versions.	Low	High	Timeline row lock, expected-timestamp checks, and partial unique indexes on DRAFT and PUBLISHED.
Removed project members remain PIC on historical tasks.	Medium	Low	Store PIC name snapshot; require reassignment before editing such tasks; document the rule.
RSA key rotation or JWKS cache staleness can reject valid tokens.	Low	High	Publish retired public keys until tokens expire, cache by kid, rate-limit JWKS refresh, and test rotation in CI.
Strict reuse detection can log out a user when an old refresh token is replayed.	Medium	Medium	Cross-tab single-flight lock, atomic rotation, clear session-expired messaging, and reuse metrics.
Task hard delete loses history.	Medium	Medium	Confirmation modal; timeline history and event ledger cover schedule changes. Release 1 explicitly accepts task hard delete.
No optimistic locking on task edits.	Medium	Medium	Last-write-wins in v1; add ETag/If-Match or a version column in a future release. Timeline versions are protected.
Frontend 100% coverage on features is expensive.	High	Medium	Exclude entrypoints/types/mocks, gate shared components at 90%, test feature behaviour, and keep components small.
Scope creep into Gantt, comments, attachments, or dynamic roles.	High	Medium	This document explicitly lists out-of-scope items; changes require a new design version.
The final access re-check cannot eliminate the last millisecond race.	Low	Medium	Membership changes are rare; the window is one internal round trip; document and monitor 403 rollbacks.
Login throttling is per credential, not global.	Medium	Medium	Five-failure lockout per credential, failed-login metrics; add gateway-level rate limiting before public exposure.

14.2 Known limitations

- One timeline per project, with no activity-level Gantt rows or dependencies.
- Fixed ADMIN and USER roles; no custom permissions or project-level roles.
- No email, comments, attachments, notifications, or approval workflow.
- Task hard delete is allowed; there is no soft delete or trash.
- No full cross-entity audit trail; timeline lifecycle events and actor columns are stored.
- No full-text search engine; task/project search uses case-insensitive LIKE on indexed/short fields.
- No optimistic locking on task edits; timeline lifecycle writes do use optimistic checks and row locks.
- No API gateway, backend-wide rate limiting, or circuit breaker. OpenTelemetry tracing and Micrometer metrics are included.
- Timeline creates and edits are limited to the holiday coverage horizon (2025-2027 until more holiday data is added).

- Internal traffic uses per-caller keys; mTLS and network policies are documented for production but are not implemented in the Compose profile.
- No mobile-native app; tablet web is supported.
- No reporting/export endpoints.

14.3 Future work

- Activity-level timeline and Gantt dependencies, with optional task-to-timeline links.
- Dynamic roles/permissions and a project-manager role.
- Soft delete and restore for tasks; a full cross-entity audit log.
- Notifications, comments, attachments, and an approval workflow.
- MFA, gateway-level rate limiting, and finer session controls.
- API gateway (Kong or Spring Cloud Gateway) with centralized authentication, rate limiting, and observability.
- Reporting, CSV/Excel export, and dashboard metrics.
- Membership cache or event-driven invalidation if access-check latency becomes material.
- Optimistic locking/ETags for task edits.
- mTLS between services and automated RSA/JWKS key rotation.

15. Approval Checklist

Please review the checklist below. Approval confirms that implementation can start in the coding folder. If any item is not approved, return this document with the requested change before coding.

Table 76 - Design approval checklist

#	Review item	Approved?
1	Four-service boundary: auth, user/project, task, timeline.	
2	No API gateway in v1; Kong/APISIX/Traefik/Spring Cloud Gateway comparison accepted.	
3	ADMIN/USER roles and project-membership access model with pre-commit re-check.	
4	One versioned timeline per project; draft-then-publish workflow; publish carries forward current status/actual % and snapshots the previous version.	
5	Task duration = calendar days; timeline duration = working days excluding weekends and Indonesian holidays.	
6	Plan completion formula, as-of semantics, rounding, and zero-working-day warning.	
7	Indonesian holiday seed for 2025-2027, holiday coverage horizon, and admin CRUD.	
8	PostgreSQL one instance with one database per service, no cross-service joins, and per-role grants with CONNECT revoked on other databases.	
9	RS256 JWT with JWKS key rotation, rotating refresh cookie with token families and reuse detection, and per-caller internal API keys.	
10	Credentials and password hashes owned only by auth-service; user-service owns profiles, roles, and membership.	
11	API endpoints, request/response shapes, status codes, error codes, and idempotent requestId handling.	
12	Timeline audit ledger, snapshots, and expected-timestamp conflict handling.	
13	React + TypeScript + Vite + Ant Design responsive frontend with cross-tab refresh locking.	
14	Docker Compose deployment, environment configuration, and observability (metrics, tracing, probes).	
15	Coverage gates: 100% lines on core business code, branch gates on calculators/state machines, PIT mutation gate, documented exclusions.	
16	Out-of-scope list and known limitations.	

15.1 Next steps after approval

1. Create the implementation scaffold under /coding with one Maven module per service and the frontend workspace.
2. Implement shared API conventions (Problem Details, request ID, validation) and the database migrations.
3. Implement user-service and auth-service first, including RS256/JWKS, credential ownership, token families, bootstrap admin, roles, refresh rotation, and access checks.
4. Implement task-service and timeline-service with their calculators, timeline versioning, audit ledger, and tests.
5. Implement the React application against the published API contracts, including cross-tab refresh locking and conflict handling.
6. Run the scoped coverage gates, mutation tests, Docker Compose smoke flow, and acceptance checklist.
7. Request final acceptance and record the approved document version.

Approval statement

I approve this technical design and authorise implementation to begin in the /coding folder. Changes after approval will be recorded as a new document version.

Table 77 - Final sign-off

Name	Role	Signature	Date
	Product Owner / Stakeholder		
	Engineering Lead		

Appendix A. Indonesian Holiday Seed Data 2025-2027

The seed below follows the Indonesian Joint Ministerial Decrees (SKB 3 Menteri) for 2025, 2026, and 2027. Both national holidays and joint leave (cuti bersama) are excluded from timeline working days. The same data is provided in docs/holidays-id-2025-2027.csv for implementation.

Sources: SKB 2025 (Kepmenag 1017/2024, Kepmenaker 2/2024, KemenPANRB 2/2024); SKB 2026 (1497/2025, 2/2025, 5/2025); SKB 2027 (1205/2026, 3/2026, 2/2026). Official announcements are listed in Appendix D.

A.1 2025

Table 78 - Indonesian holidays and joint leave - 2025

Date	Day	Type	Holiday
01 Jan 2025	Wednesday	National	Tahun Baru 2025 Masehi
27 Jan 2025	Monday	National	Isra Mikraj Nabi Muhammad SAW
28 Jan 2025	Tuesday	Joint leave	Cuti bersama Tahun Baru Imlek 2576 Kongzili
29 Jan 2025	Wednesday	National	Tahun Baru Imlek 2576 Kongzili
28 Mar 2025	Friday	Joint leave	Cuti bersama Hari Suci Nyepi
29 Mar 2025	Saturday	National	Hari Suci Nyepi (Tahun Baru Saka 1947)
31 Mar 2025	Monday	National	Idul Fitri 1446 H - hari pertama
01 Apr 2025	Tuesday	National	Idul Fitri 1446 H - hari kedua
02 Apr 2025	Wednesday	Joint leave	Cuti bersama Idul Fitri 1446 H
03 Apr 2025	Thursday	Joint leave	Cuti bersama Idul Fitri 1446 H
04 Apr 2025	Friday	Joint leave	Cuti bersama Idul Fitri 1446 H
07 Apr 2025	Monday	Joint leave	Cuti bersama Idul Fitri 1446 H
18 Apr 2025	Friday	National	Wafat Yesus Kristus
20 Apr 2025	Sunday	National	Kebangkitan Yesus Kristus (Paskah)
01 May 2025	Thursday	National	Hari Buruh Internasional
12 May 2025	Monday	National	Hari Raya Waisak 2569 BE
13 May 2025	Tuesday	Joint leave	Cuti bersama Hari Raya Waisak 2569 BE
29 May 2025	Thursday	National	Kenaikan Yesus Kristus
30 May 2025	Friday	Joint leave	Cuti bersama Kenaikan Yesus Kristus
01 Jun 2025	Sunday	National	Hari Lahir Pancasila
06 Jun 2025	Friday	National	Idul Adha 1446 H
09 Jun 2025	Monday	Joint leave	Cuti bersama Idul Adha 1446 H
27 Jun 2025	Friday	National	1 Muharam Tahun Baru Islam 1447 H
17 Aug 2025	Sunday	National	Proklamasi Kemerdekaan
05 Sep 2025	Friday	National	Maulid Nabi Muhammad SAW
25 Dec 2025	Thursday	National	Kelahiran Yesus Kristus
26 Dec 2025	Friday	Joint leave	Cuti bersama Kelahiran Yesus Kristus

A.2 2026

Table 79 - Indonesian holidays and joint leave - 2026

Date	Day	Type	Holiday
01 Jan 2026	Thursday	National	Tahun Baru 2026 Masehi
16 Jan 2026	Friday	National	Isra Mikraj Nabi Muhammad SAW
16 Feb 2026	Monday	Joint leave	Cuti bersama Tahun Baru Imlek 2577 Kongzili
17 Feb 2026	Tuesday	National	Tahun Baru Imlek 2577 Kongzili
18 Mar 2026	Wednesday	Joint leave	Cuti bersama Hari Suci Nyepi
19 Mar 2026	Thursday	National	Hari Suci Nyepi (Tahun Baru Saka 1948)
20 Mar 2026	Friday	Joint leave	Cuti bersama Idul Fitri 1447 H
21 Mar 2026	Saturday	National	Idul Fitri 1447 H - hari pertama
22 Mar 2026	Sunday	National	Idul Fitri 1447 H - hari kedua
23 Mar 2026	Monday	Joint leave	Cuti bersama Idul Fitri 1447 H
24 Mar 2026	Tuesday	Joint leave	Cuti bersama Idul Fitri 1447 H
03 Apr 2026	Friday	National	Wafat Yesus Kristus
05 Apr 2026	Sunday	National	Kebangkitan Yesus Kristus (Paskah)
01 May 2026	Friday	National	Hari Buruh Internasional
14 May 2026	Thursday	National	Kenaikan Yesus Kristus
15 May 2026	Friday	Joint leave	Cuti bersama Kenaikan Yesus Kristus
27 May 2026	Wednesday	National	Idul Adha 1447 H
28 May 2026	Thursday	Joint leave	Cuti bersama Idul Adha 1447 H
31 May 2026	Sunday	National	Hari Raya Waisak 2570 BE
01 Jun 2026	Monday	National	Hari Lahir Pancasila
16 Jun 2026	Tuesday	National	1 Muharam Tahun Baru Islam 1448 H
17 Aug 2026	Monday	National	Proklamasi Kemerdekaan
25 Aug 2026	Tuesday	National	Maulid Nabi Muhammad SAW
24 Dec 2026	Thursday	Joint leave	Cuti bersama Kelahiran Yesus Kristus
25 Dec 2026	Friday	National	Kelahiran Yesus Kristus

A.3 2027

Table 80 - Indonesian holidays and joint leave - 2027

Date	Day	Type	Holiday
01 Jan 2027	Friday	National	Tahun Baru 2027 Masehi
05 Jan 2027	Tuesday	National	Isra Mikraj Nabi Muhammad SAW 1448 H
05 Feb 2027	Friday	Joint leave	Cuti bersama Tahun Baru Imlek 2578 Kongzili
06 Feb 2027	Saturday	National	Tahun Baru Imlek 2578 Kongzili
08 Mar 2027	Monday	National	Hari Suci Nyepi (Tahun Baru Saka 1949)
09 Mar 2027	Tuesday	Joint leave	Cuti bersama Idul Fitri 1448 H
10 Mar 2027	Wednesday	National	Idul Fitri 1448 H - hari pertama
11 Mar 2027	Thursday	National	Idul Fitri 1448 H - hari kedua
12 Mar 2027	Friday	Joint leave	Cuti bersama Idul Fitri 1448 H
15 Mar 2027	Monday	Joint leave	Cuti bersama Idul Fitri 1448 H
25 Mar 2027	Thursday	Joint leave	Cuti bersama Wafat Yesus Kristus
26 Mar 2027	Friday	National	Wafat Yesus Kristus
28 Mar 2027	Sunday	National	Kebangkitan Yesus Kristus (Paskah)
01 May 2027	Saturday	National	Hari Buruh Internasional
06 May 2027	Thursday	National	Kenaikan Yesus Kristus
17 May 2027	Monday	National	Idul Adha 1448 H
18 May 2027	Tuesday	Joint leave	Cuti bersama Idul Adha 1448 H
19 May 2027	Wednesday	Joint leave	Cuti bersama Hari Raya Waisak 2571 BE
20 May 2027	Thursday	National	Hari Raya Waisak 2571 BE
01 Jun 2027	Tuesday	National	Hari Lahir Pancasila
06 Jun 2027	Sunday	National	1 Muharam Tahun Baru Islam 1449 H
15 Aug 2027	Sunday	National	Maulid Nabi Muhammad SAW
17 Aug 2027	Tuesday	National	Proklamasi Kemerdekaan
24 Dec 2027	Friday	Joint leave	Cuti bersama Kelahiran Yesus Kristus
25 Dec 2027	Saturday	National	Kelahiran Yesus Kristus
26 Dec 2027	Sunday	National	Isra Mikraj Nabi Muhammad SAW 1449 H

Appendix B. Calculation Examples

B.1 Task duration

Table 81 - Task duration examples

Start	End	Calendar days inclusive	Result
2026-11-02 (Mon)	2026-11-02 (Mon)	1	1 day
2026-11-02 (Mon)	2026-11-06 (Fri)	5	5 days
2026-11-02 (Mon)	2026-11-09 (Mon)	8	8 days (includes weekend)
2026-12-24 (Thu)	2026-12-28 (Mon)	5	5 days (holiday still counted)

B.2 Timeline working days and plan completion

Timeline 16-27 March 2026. The range contains the weekend 21-22 March, Nyepi on 19 March, and joint leave on 18, 20, 23, and 24 March. The working days are 16, 17, 25, 26, and 27 March: 5 days. Daily increment = 100 / 5 = 20.00%.

Table 82 - Timeline plan completion example

As-of	Reason	Elapsed working days	Plan %
2026-03-15	Sunday before start	0	0.00%
2026-03-16	Monday	1	20.00%
2026-03-17	Tuesday	2	40.00%
2026-03-18	Joint leave	2	40.00%
2026-03-19	Nyepi	2	40.00%
2026-03-20	Joint leave	2	40.00%
2026-03-21	Saturday	2	40.00%
2026-03-22	Sunday	2	40.00%
2026-03-23	Joint leave	2	40.00%
2026-03-24	Joint leave	2	40.00%
2026-03-25	Wednesday	3	60.00%
2026-03-26	Thursday	4	80.00%
2026-03-27	Friday, end	5	100.00%
2026-03-30	After end	5 (capped)	100.00%

B.3 Timeline revision scenario

Table 83 - Timeline versioning scenario

Event	Version	Lifecycle	Dates	Effect
Create initial timeline	v1	DRAFT	02 Nov 2026 - 31 Mar 2027	Not active. Plan % preview only. Actual/status are not editable on the draft.
Publish v1	v1	PUBLISHED	02 Nov 2026 - 31 Mar 2027	v1 is current with OPEN / 0% carried forward.
Update actual progress	v1	PUBLISHED	unchanged	percentActual changes; percentPlan recalculates. Event ledger records the change.
Create revision	v2	DRAFT	02 Nov 2026 - 15 Apr 2027	Copies planned dates only; v1 status/actual are carried forward at publish; v1 remains current.

Event	Version	Lifecycle	Dates	Effect
Publish v2	v2	PUBLISHED	02 Nov 2026 - 15 Apr 2027	v2 current with v1's current status/actual carried forward; v1 becomes SUPERSEDED with a snapshot.
View v1 history	v1	SUPERSEDED	02 Nov 2026 - 31 Mar 2027	Returns snapshot duration/plan/actual and snapshot_as_of_date; immutable.
Cancel v2 draft before publish	v2	CANCELLED	02 Nov 2026 - 15 Apr 2027	v1 remains current; v2 stays in history. If v2 were the only draft and v1 never existed, the timeline identity would be removed.

B.4 Rounding

Table 84 - Plan percentage rounding examples

Scenario	Raw value	Rounded (HALF_UP, 2 decimals)
1 of 3 working days	33.333...	33.33%
2 of 3 working days	66.666...	66.67%
1 of 6 working days	16.666...	16.67%
5 of 7 working days	71.428...	71.43%
After end	capped	100.00%
Before start	0	0.00%

B.5 Zero working days after a holiday edit

A timeline for 04-05 May 2026 is created while both days are working days: durationWorkdays = 2 and the daily increment is 50.00%. An administrator later adds 04 and 05 May 2026 as joint leave. The timeline is not deleted and the holiday edit is not rejected. Every read of that timeline now returns durationWorkdays = 0, percentPlan = 0.00, and planWarning = NO_WORKING_DAYS. Creating or editing a timeline with those dates after the holiday change would be rejected with 400 NO_WORKING_DAYS, so the operator recovers by editing the planned dates in a new draft or revision.

Appendix C. Glossary

Table 85 - Glossary

Term	Meaning
Access token	Short-lived RS256-signed JWT sent in the Authorization header and validated through cached JWKS public keys.
Refresh token	Opaque long-lived token stored in an HttpOnly cookie and hashed in auth_db. Belongs to a token family.
Token family	All refresh tokens descended from one login. Reuse of any revoked token revokes the whole family.
JWKS	JSON Web Key Set served by auth-service at /.well-known/jwks.json; resource services cache the public keys.
Credential	auth_db.auth_credential row holding the BCrypt password hash and login lock state. Owned only by auth-service.
requestId	Client-generated UUID on create/publish requests; a unique constraint makes retries idempotent.
Idempotent-Replay	Response header on a replayed requestId; the original resource is returned and the body is not re-applied.
ADMIN	Global role that can manage users, roles, projects, members, holidays, tasks, and timelines.
USER	Global role that can access only projects where they are a member.
Project member	A user linked to a project; membership is the access boundary.
PIC	Person in charge of a task. Must be an active project member.
Working day	Monday-Friday excluding Indonesian national holidays and joint leave.
Calendar day	Every date in the range, including weekends and holidays.
Duration	Task: calendar days inclusive. Timeline: working days inclusive.
Plan completion	Expected completion based on elapsed working days / total working days.
Actual completion	Manually entered completion percentage on the current published timeline.
Timeline version	An ordered, auditable planned schedule with lifecycle status.
Draft	Editable timeline version holding planned dates and notes; not active until published.
Current version	The single PUBLISHED version used as the project's active timeline.
Superseded	A previously published version replaced by a newer published version; keeps a frozen snapshot.
Snapshot	Duration, plan %, actual %, and as-of date frozen when a version is superseded or cancelled.
planWarning	NO_WORKING_DAYS when a timeline range has no working days; percentPlan is then 0.00.
Holiday coverage horizon	Minimum to maximum seeded holiday year; timeline writes outside it are rejected with HOLIDAY_COVERAGE_REQUIRED.
Timeline version event	Append-only ledger row recording create, update, publish, supersede, cancel, status, and progress changes.
Joint leave	Indonesian cuti bersama; excluded from working days like a national holiday.
Problem Details	RFC 7807/9457 JSON error format used by all services.
CORS	Browser cross-origin policy; each service uses an explicit origin allow-list.

Appendix D. References

1. Sekretariat Kabinet RI. Government Establishes National Holidays, Mass Leaves for 2025. https://setkab.go.id/en/govt-establishes-national-holidays-mass-leaves-for-2025/
2. Kementerian Koordinator Bidang Pembangunan Manusia dan Kebudayaan. SKB Libur Nasional dan Cuti Bersama Tahun 2026. https://www.kemenkopmk.go.id/
3. Sekretariat Negara RI. Inilah SKB 3 Menteri Libur Nasional dan Cuti Bersama 2026. https://www.setneg.go.id/
4. Kementerian Koordinator Bidang Pembangunan Manusia dan Kebudayaan. SKB Libur Nasional dan Cuti Bersama Tahun 2027. https://www.kemenkopmk.go.id/
5. Sekretariat Negara RI. Inilah SKB 3 Menteri Libur Nasional dan Cuti Bersama 2027. https://www.setneg.go.id/
6. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/
7. Spring Security Reference. https://docs.spring.io/spring-security/reference/
8. PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/
9. React Documentation. https://react.dev/

10. Ant Design 5 Documentation. https://ant.design/
11. OpenAPI Specification 3.1. https://spec.openapis.org/oas/v3.1.0
12. RFC 9457 Problem Details for HTTP APIs. https://www.rfc-editor.org/rfc/rfc9457
13. OWASP Application Security Verification Standard. https://owasp.org/www-project-application-security-verification-standard/
14. Kong Gateway Documentation. https://docs.konghq.com/
15. Apache APISIX Documentation. https://apisix.apache.org/docs/
16. Traefik Documentation. https://doc.traefik.io/traefik/
17. Spring Cloud Gateway Documentation. https://docs.spring.io/spring-cloud-gateway/reference/